Reti Sequenziali Asincrone

Reti Logiche T

Ingegneria Informatica

Rete sequenziale

Es. **cassaforte:**

$z = 1$ (porta aperta) se e solo se la **sequenza** degli ingressi $(\boldsymbol{x_0}, \boldsymbol{x_1})$ è nell'ordine:

$(\mathbf{1}, \mathbf{1}) - (\mathbf{0}, \mathbf{1}) - (\mathbf{1}, \mathbf{1})$

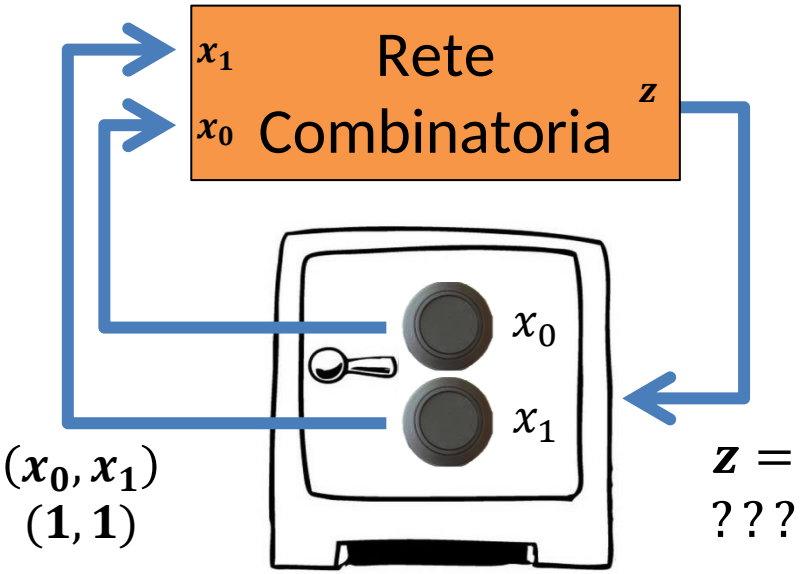


Tabella della Verità (?)

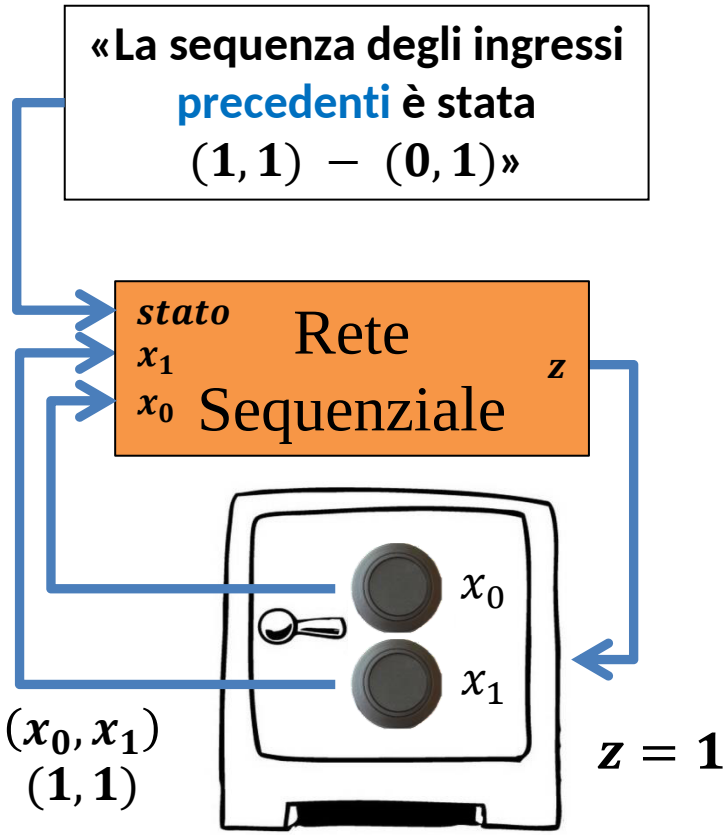
x_0	x_1	z
0	0	0
0	1	0
1	0	0
1	1	**dipende dagli ingressi precedenti…**

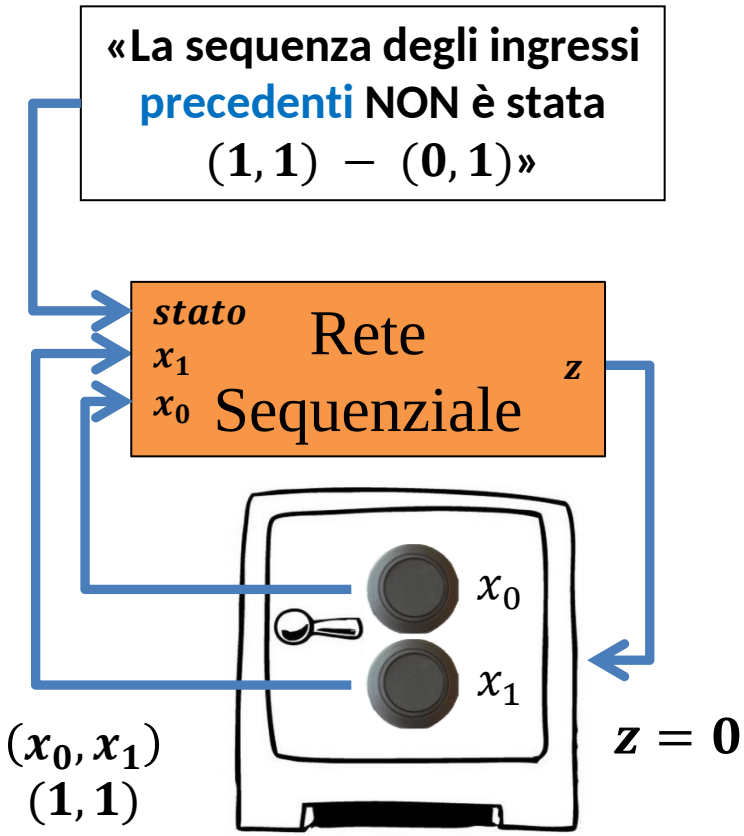
A una stessa configurazione binaria degli ingressi corrispondono uscite diverse

La tabella della verità **<u>non è più sufficiente</u>** per esprimere il **comportamento** della rete

Rete sequenziale

Gli ingressi attuali non determinano univocamente l'uscita: è necessario un **riassunto** della storia passata, detto **«stato»** della rete



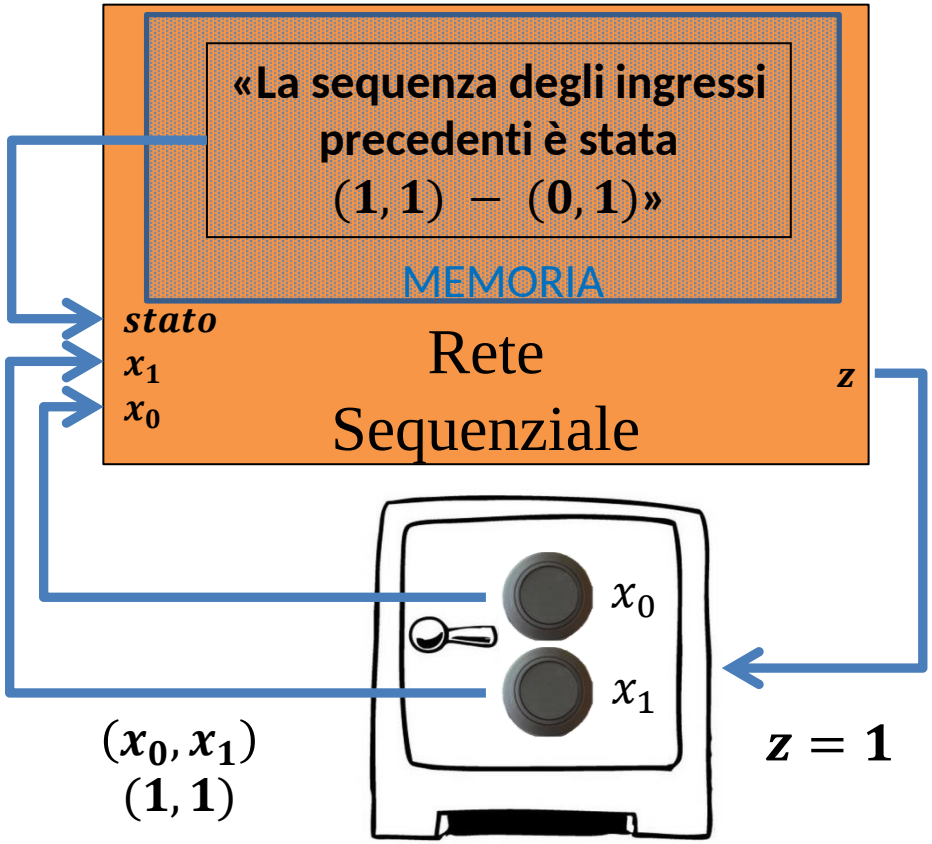


Da dove arrivano questi ingressi aggiuntivi di stato?

Ho bisogno di una **memoria** in cui mantenere questa informazione

Memoria

Lo stato è quindi un'informazione che la rete sequenziale **memorizza al suo interno**.



Cosa deve ricordarsi la rete del nostro esempio?

Stati

Gli stati necessari sono tutti i riassunti «**utili**» della storia passata degli ingressi.
«Utile» = che permette di disambiguare ingressi futuri,
che cambia il comportamento futuro della rete

«Non ho visto simboli della sequenza»

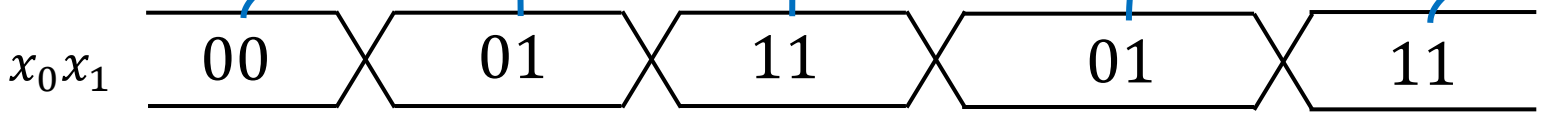
Non è importante sapere quali e quanti simboli fuori sequenza ho visto, il comportamento futuro è comunque attendere il primo (1,1) e non aprire la cassaforte quando viene ricevuto

«Ho visto il primo $(\mathbf{1},\mathbf{1})$ della sequenza»

Sapere se ho visto il primo simbolo della sequenza è «utile», perché se dopo arriva (0,1) la rete si comporterà diversamente da prima

«La sequenza degli ingressi precedenti è stata $(\mathbf{1},\mathbf{1})\ -\ (\mathbf{0},\mathbf{1})$»

«Sequenza riconosciuta»

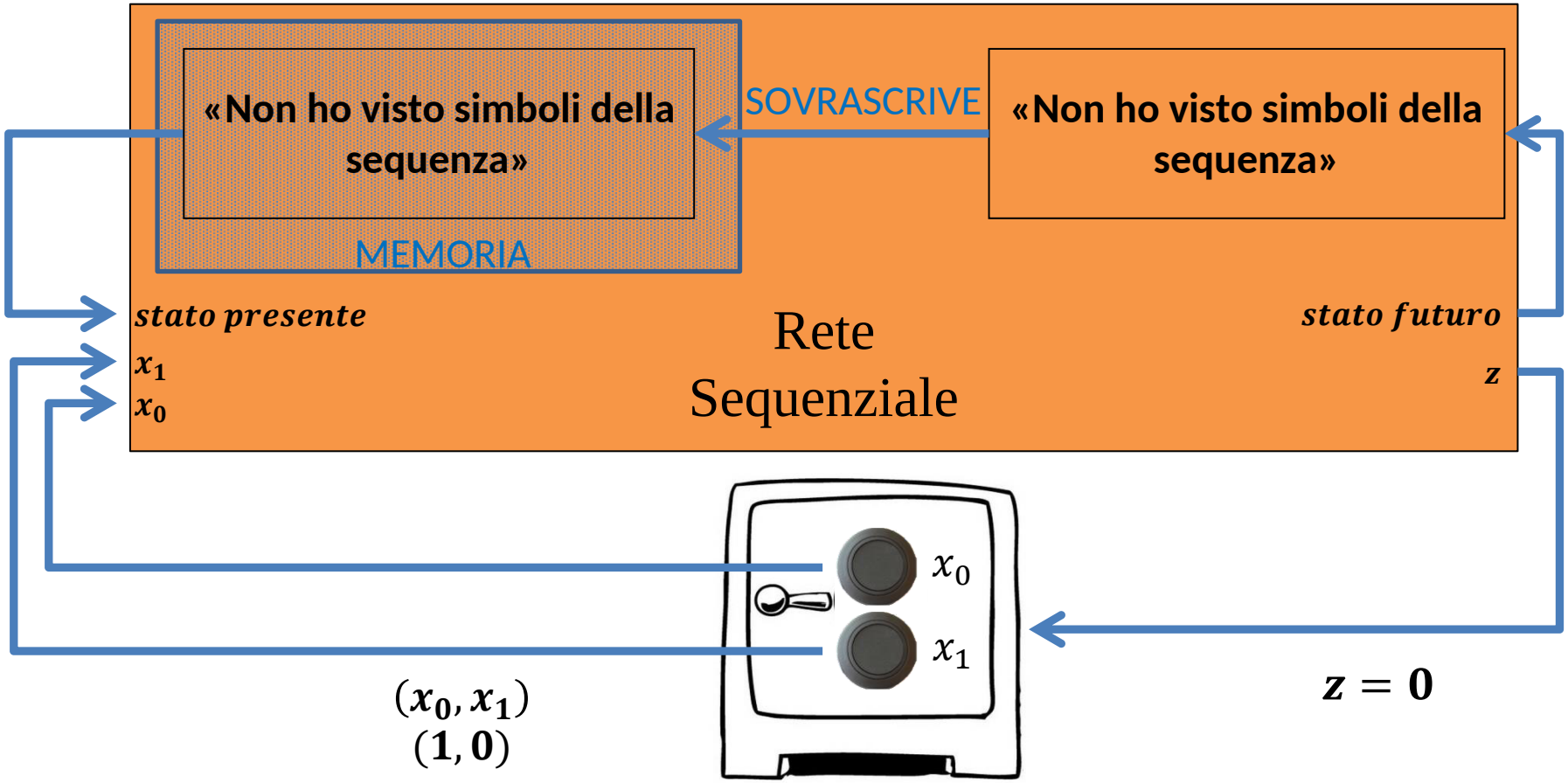


Stato presente e futuro

Ogni volta che cambiano gli ingressi, la rete, oltre a decidere il valore dell'uscita, deve anche decidere **se è successo qualcosa che «vale la pena ricordare», ovvero qualcosa che influenzerà il comportamento futuro della rete.**

Se è così, **deve cambiare stato** per memorizzare il nuovo evento.

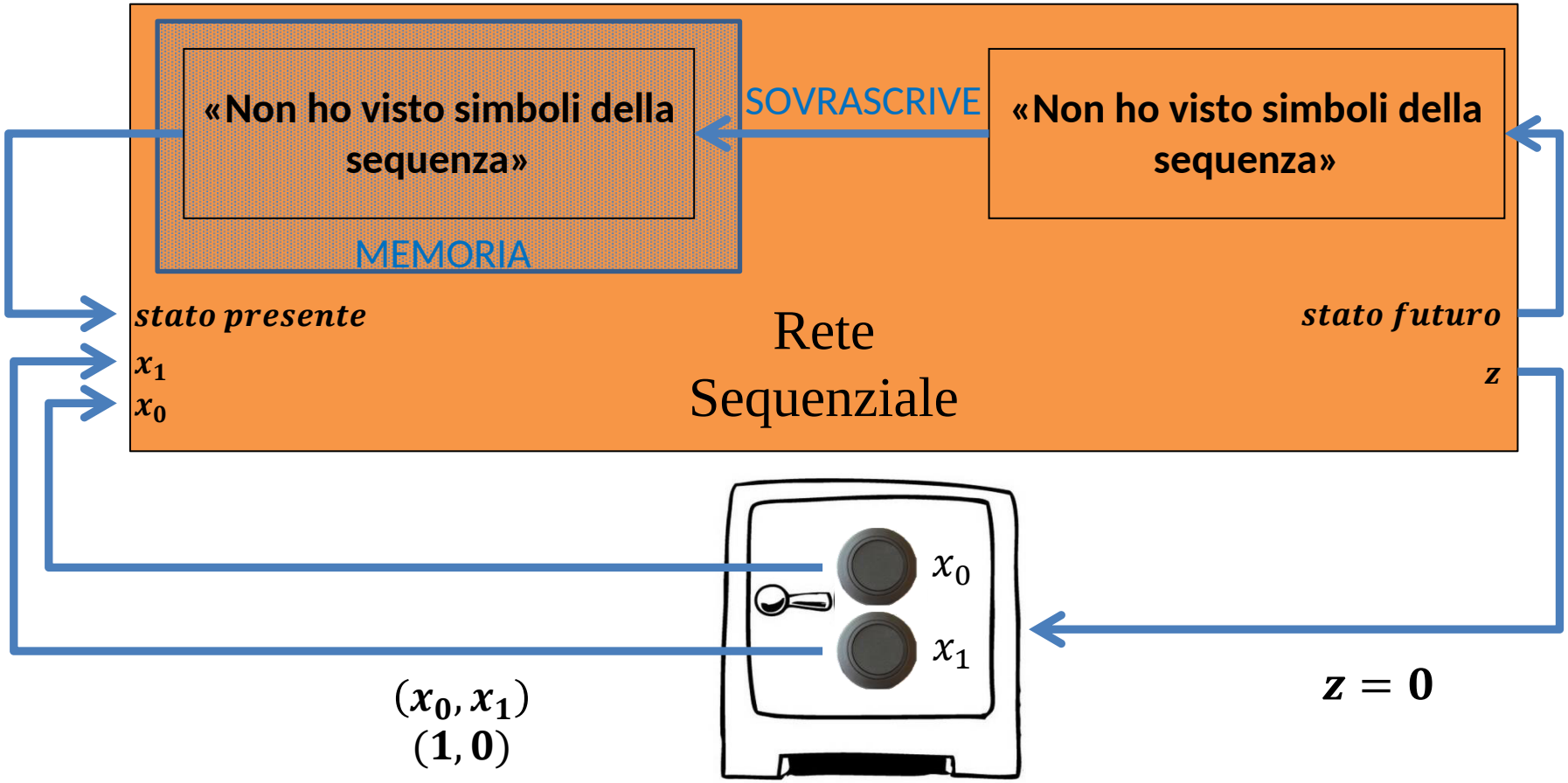
La rete calcola quindi lo **stato futuro**, ovvero il nuovo stato che riassume la storia tenendo in considerazione anche l'ultimo ingresso visto.



Rete Sequenziale Asincrona (RSA)

Esistono due tipi di reti sequenziali: **asincrone** e **sincrone**.

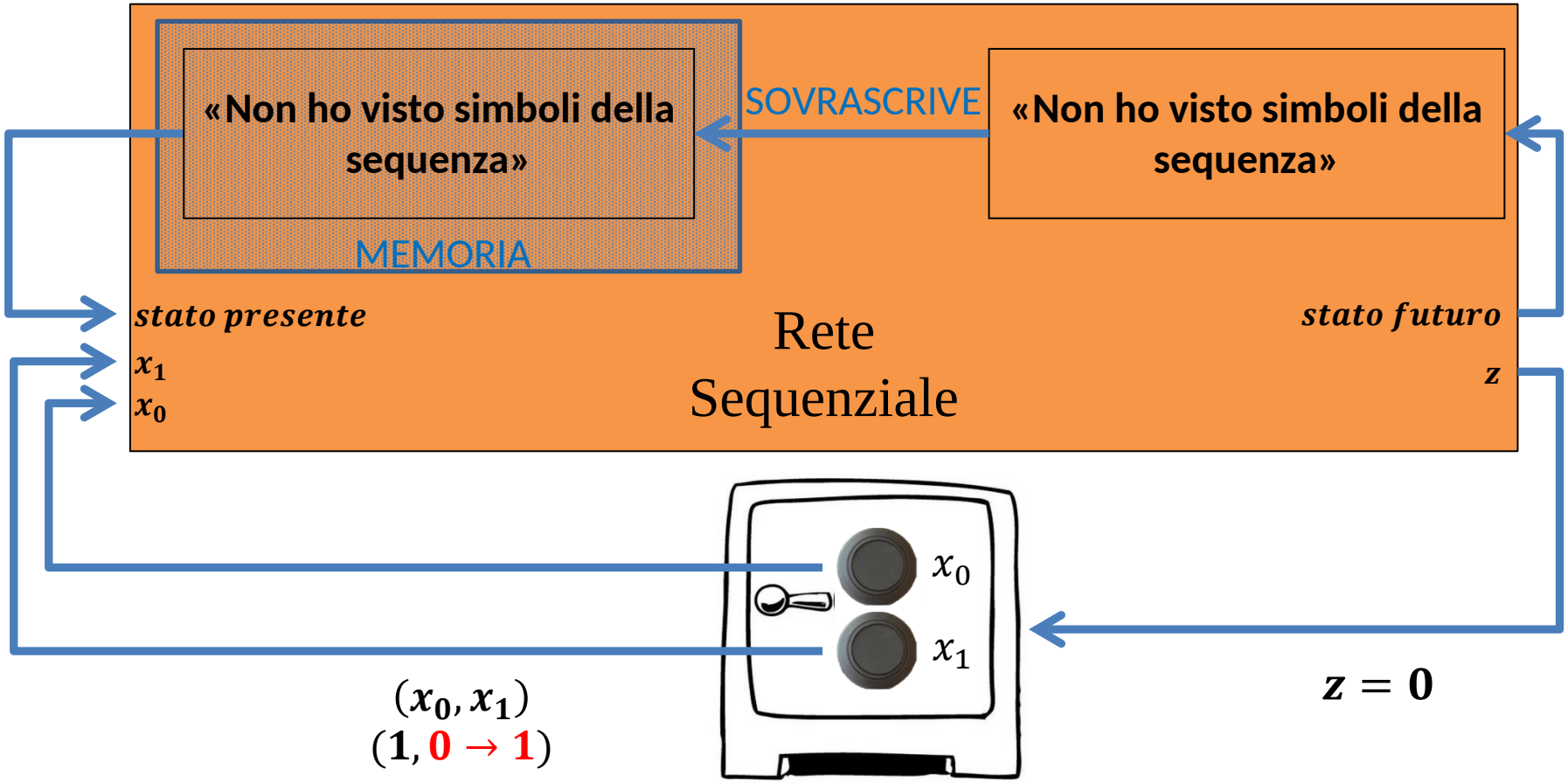
In una rete asincrona, lo stato futuro sovrascrive lo stato presente **non appena è stato calcolato**.



Stato presente e futuro

Sequenza tipica di lavoro di una rete sequenziale **asincrona**

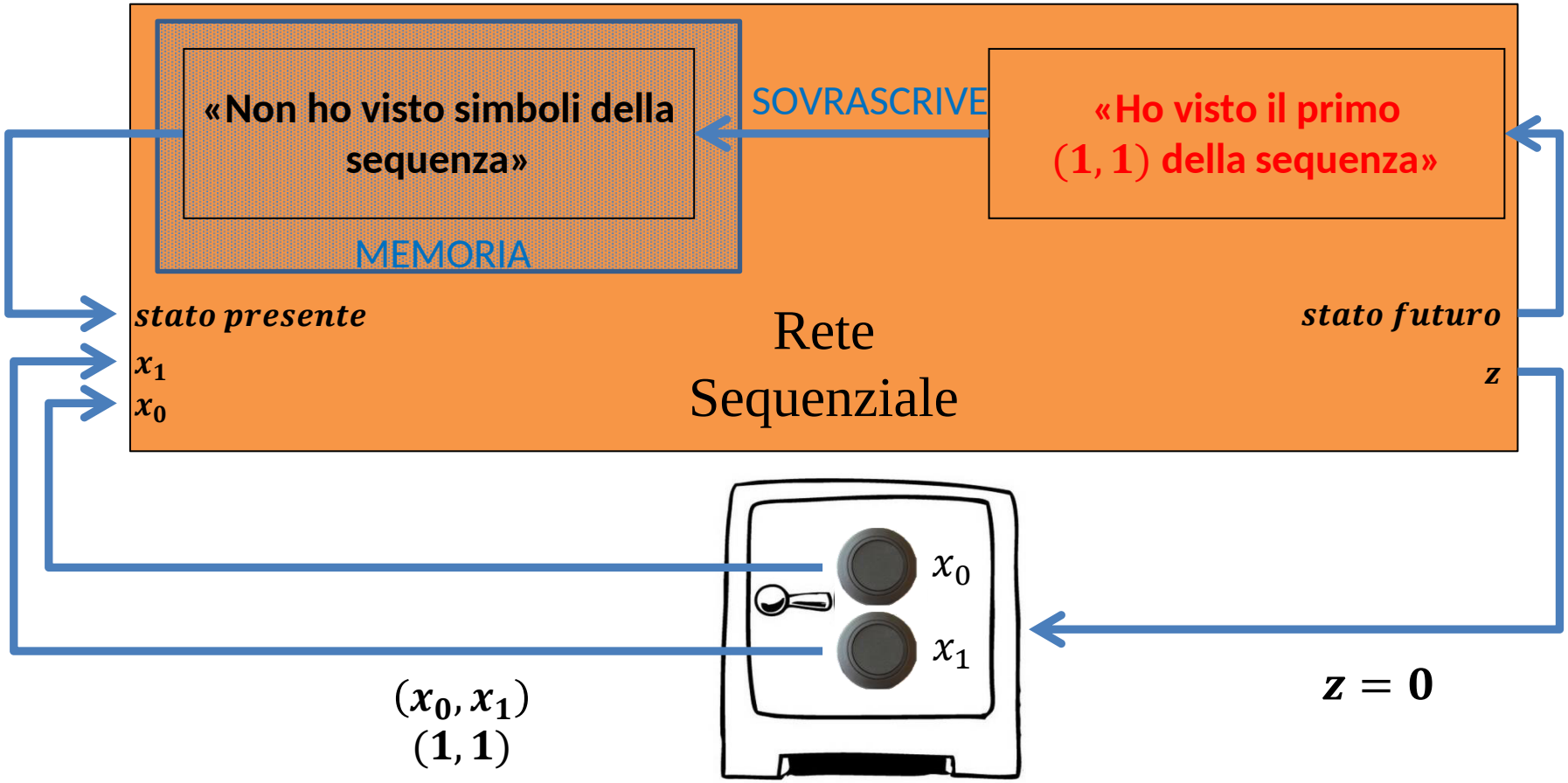
1. Partendo da una situazione **stabile**, ovvero stato presente = stato futuro e ingresso costante, **cambia un ingresso**



Stato presente e futuro

Sequenza tipica di lavoro di una rete sequenziale **asincrona**

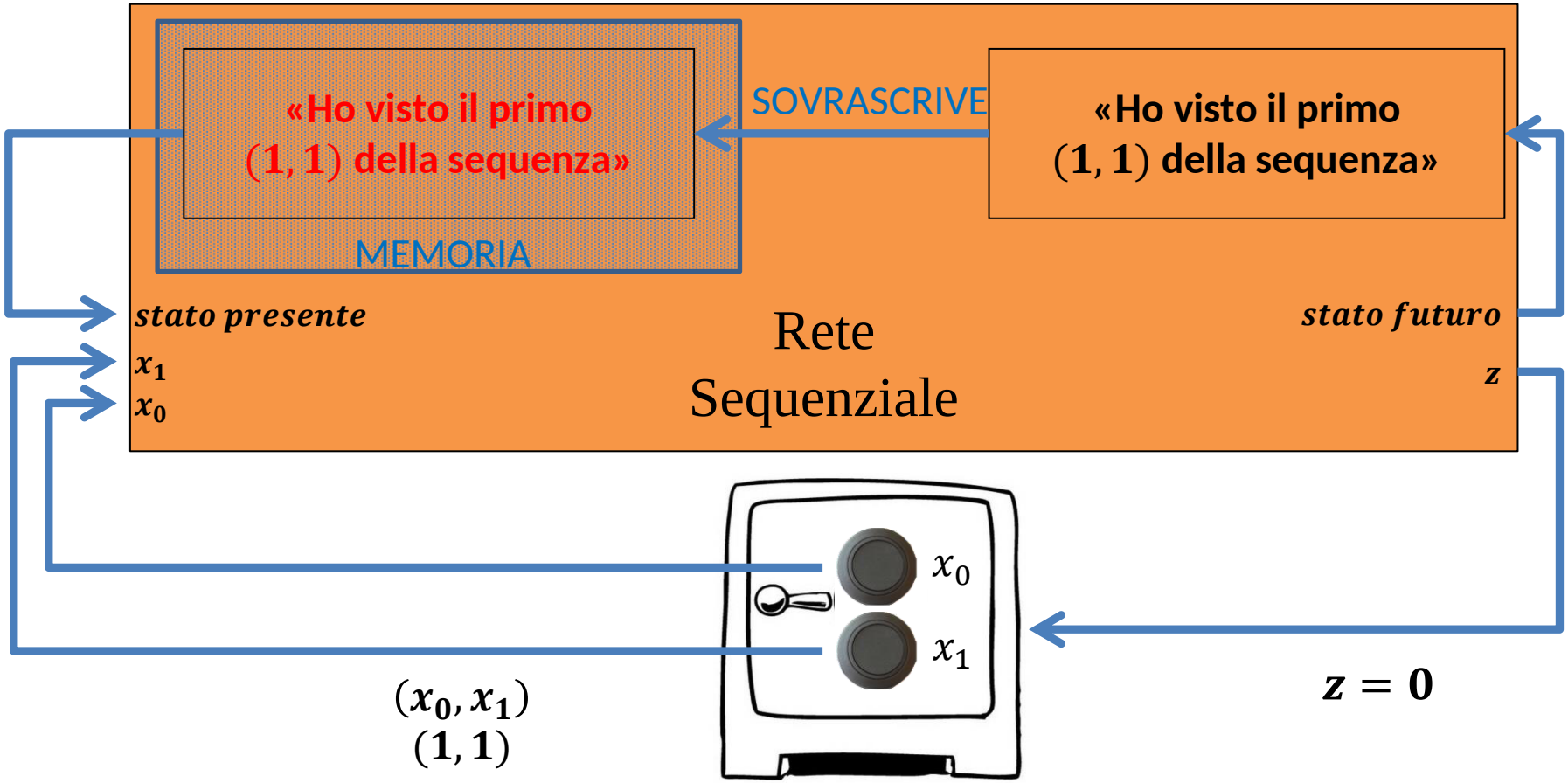
2. Dopo un tempo pari al ritardo della rete, sono disponibili **un nuovo stato futuro e una nuova uscita**



Stato presente e futuro

Sequenza tipica di lavoro di una rete sequenziale **asincrona**

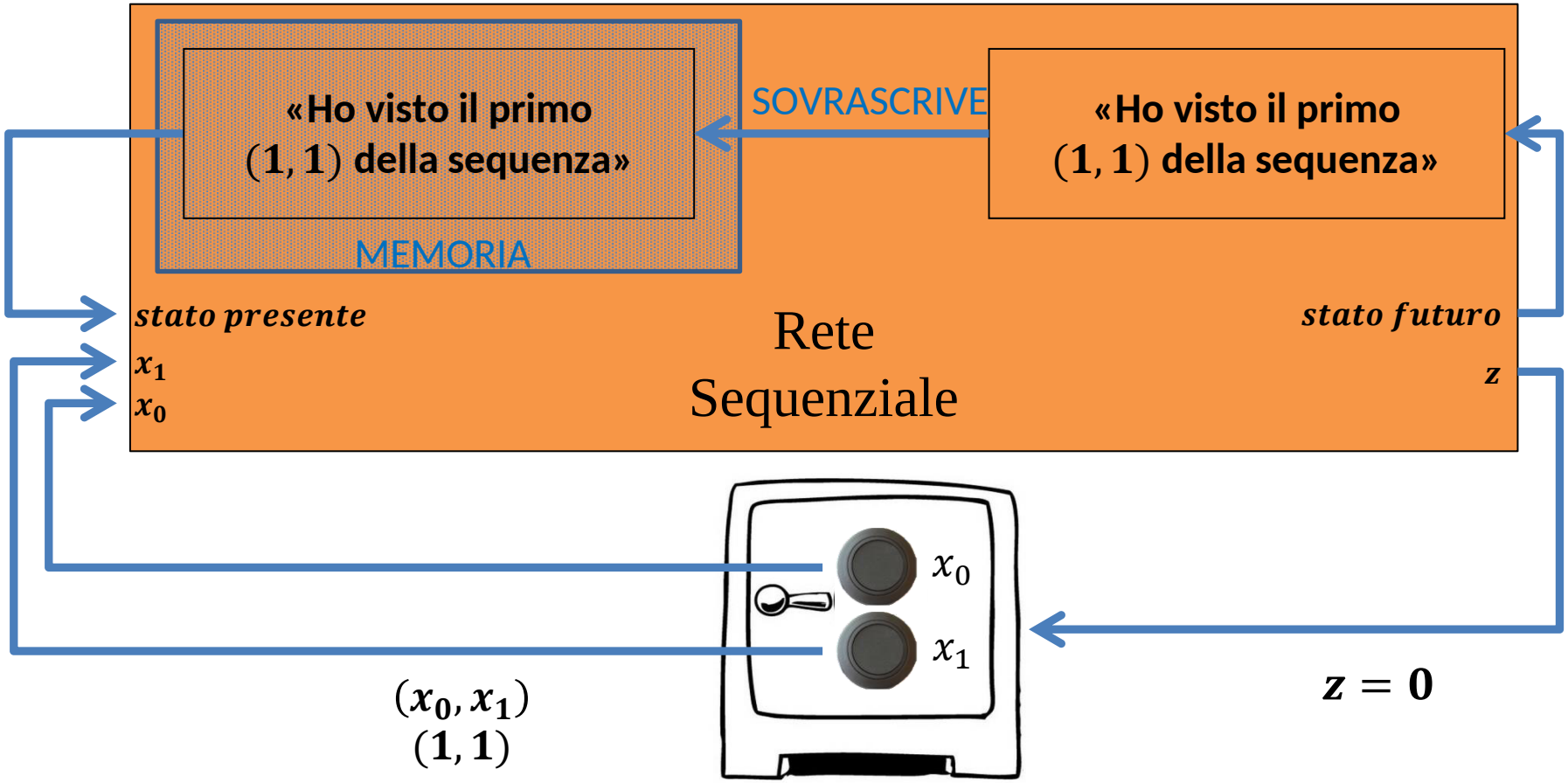
3. Lo stato futuro **diventa stato presente** sovrascrivendo il precedente nella memoria della rete



Stato presente e futuro

Sequenza tipica di lavoro di una rete sequenziale **asincrona**

4. **La rete raggiunge una nuova stabilità**, ovvero lo stato futuro calcolato a partire dal nuovo ingresso e dal nuovo stato presente è lo stesso stato, e attende il prossimo cambiamento degli ingressi.



Codifica degli stati

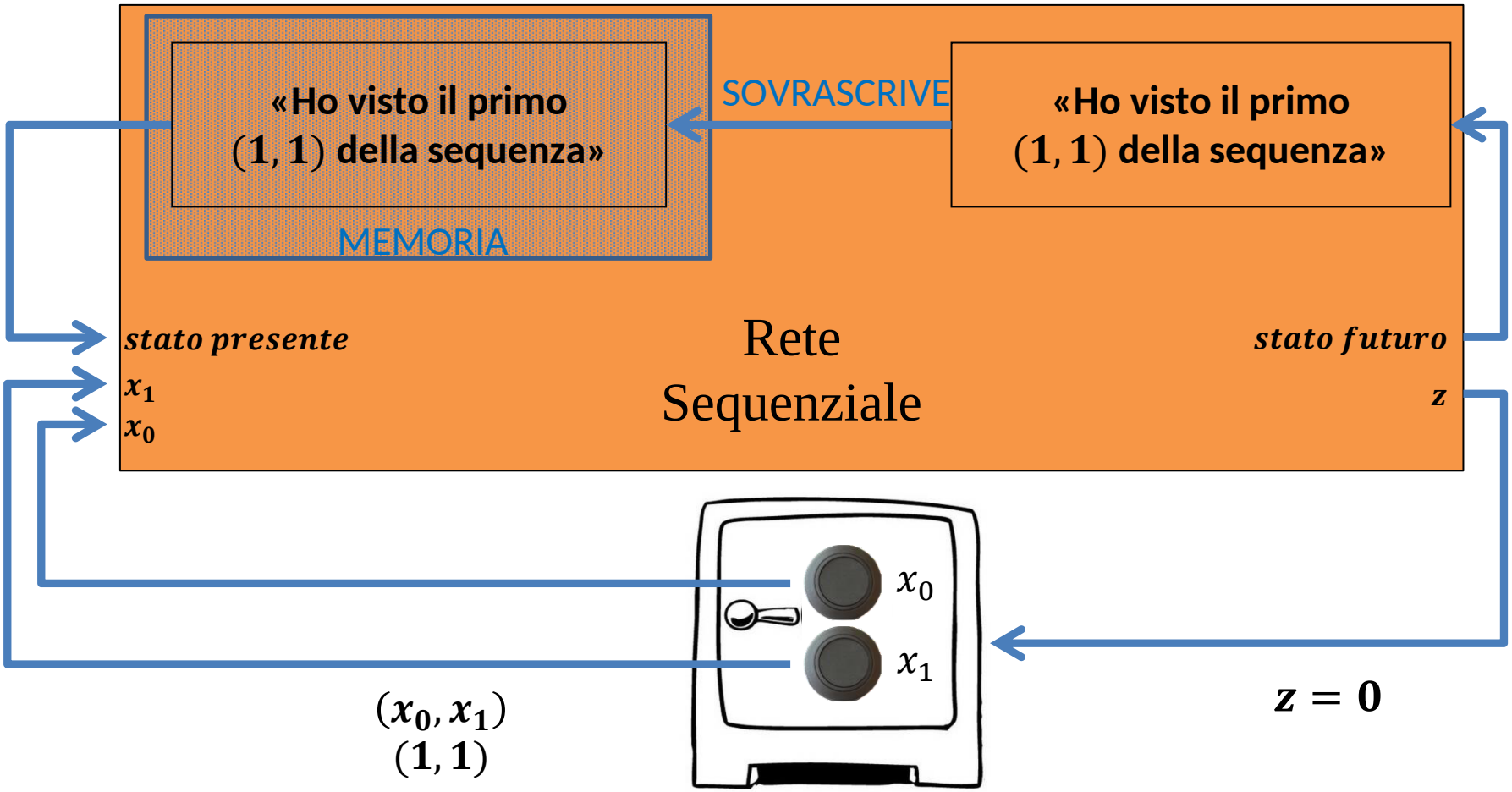
- Come rappresento gli stati in una macchina digitale?
- Come tutto il resto: **attraverso segnali binari organizzati in un codice**
- In una rete sequenziale con K stati ci saranno quindi (almeno) $k = \lceil \log_2 K \rceil$ bit per rappresentare lo stato
- Per esempio (**scelta arbitraria**, vedremo che non tutte sono valide):

y_1	y_0	$stato$
0	0	**«Non ho visto simboli della sequenza»**
0	1	**«Ho visto il primo $(\mathbf{1},\mathbf{1})$ della sequenza»**
1	1	**«La sequenza degli ingressi precedenti è stata $(\mathbf{1},\mathbf{1}) - (\mathbf{0},\mathbf{1})$»**
1	0	**«Sequenza riconosciuta»**

Stato come codice binario

Gli ingressi di stato presente e le uscite di stato futuro sono quindi rappresentati ognuno da k segnali binari

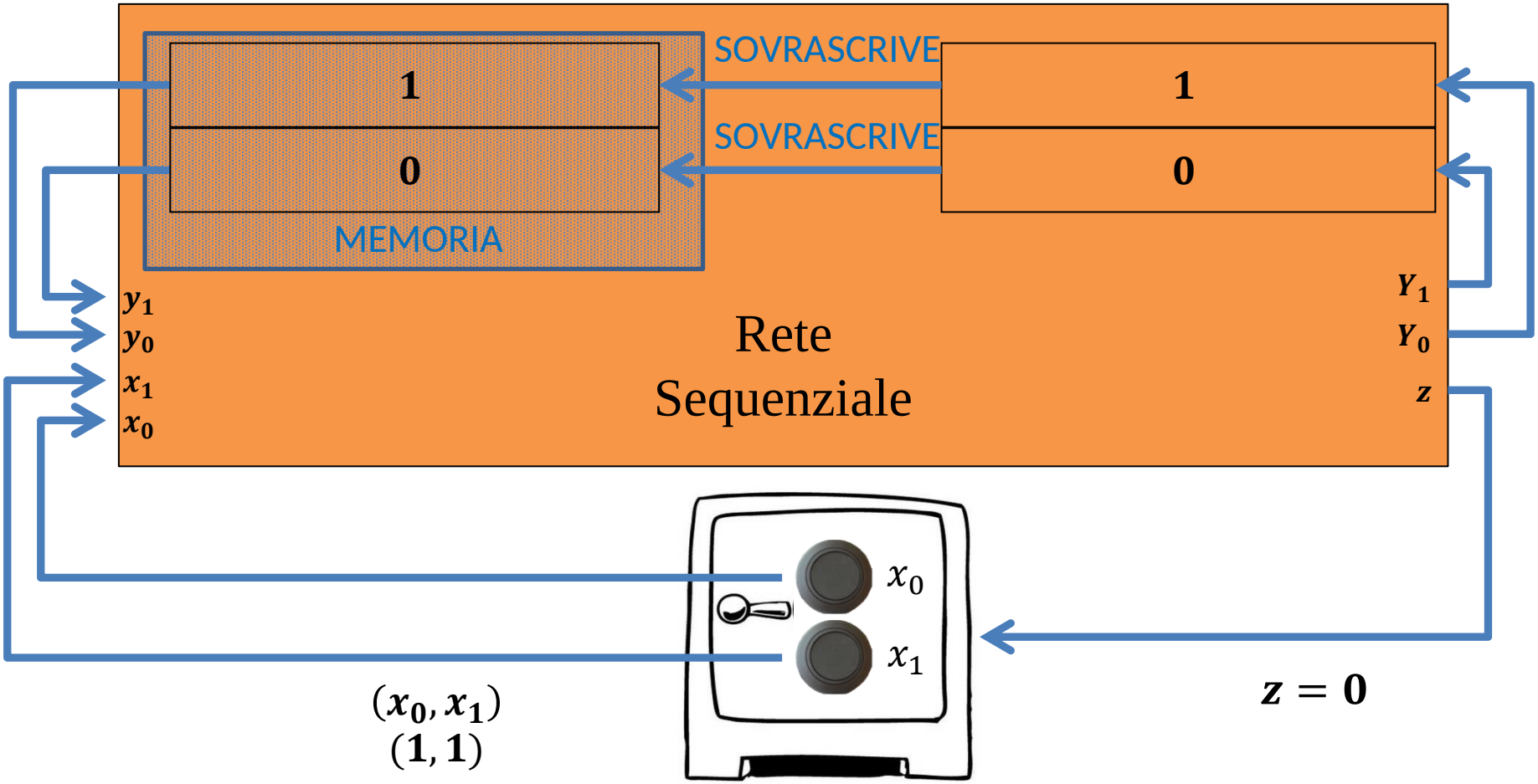
Tipicamente useremo $y_{k-1}, \ldots, y_1, y_0$ per lo stato presente e $Y_{k-1}, \ldots, Y_1, Y_0$ per lo stato futuro.



Stato come codice binario

Gli ingressi di stato presente e le uscite di stato futuro sono quindi rappresentate ognuno da k segnali binari

Tipicamente useremo $y_{k-1}, \ldots, y_1, y_0$ per lo stato presente e $Y_{k-1}, \ldots, Y_1, Y_0$ per lo stato futuro.



Rete Sequenziale Asincrona (RSA)

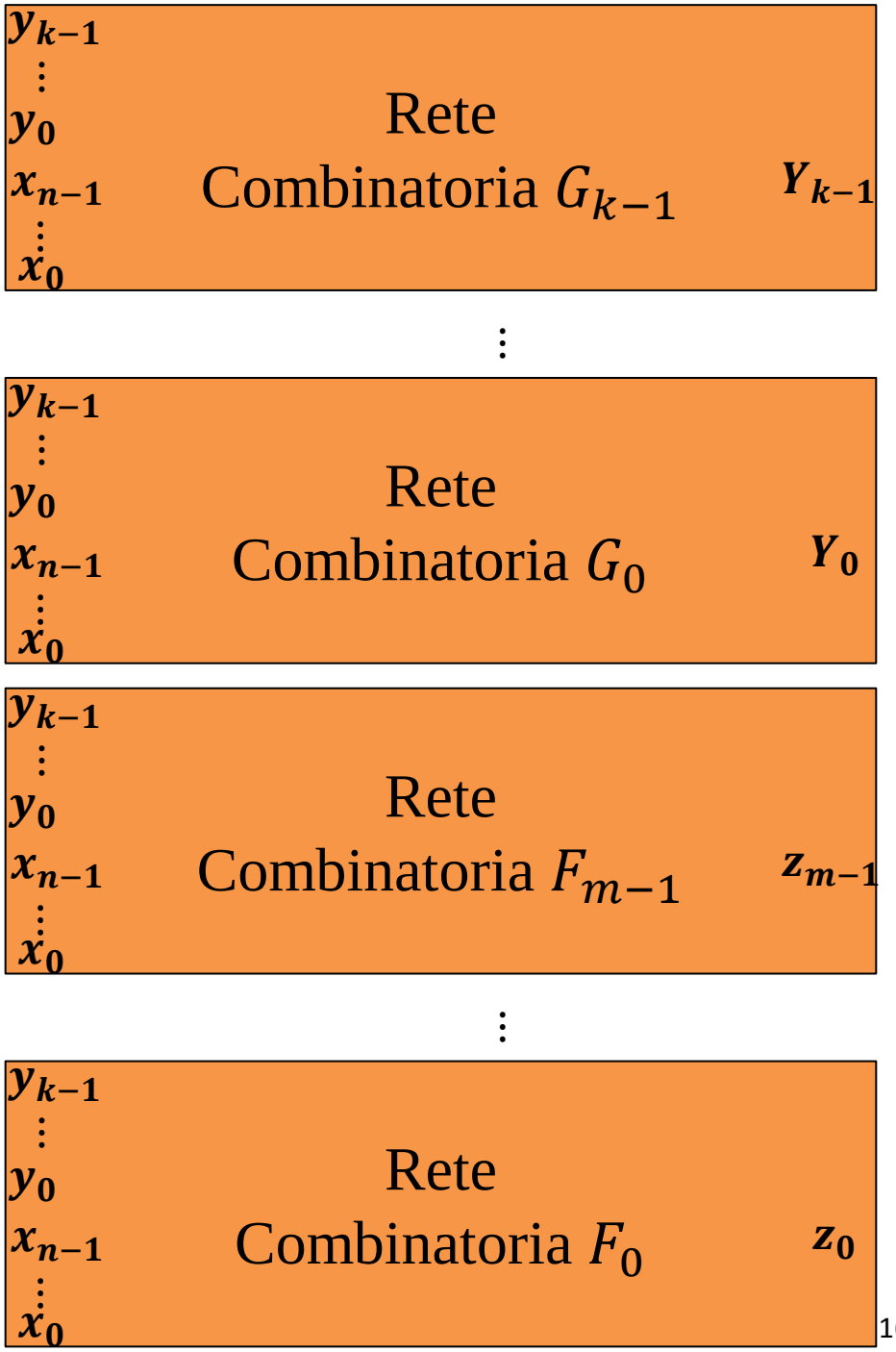
- Riassumendo, una **rete sequenziale asincrona** deve calcolare in ogni istante due insiemi di **segnali binari**
  - **m segnali di uscita della rete**
  - **k segnali di stato futuro**
- Avendo in ingresso due insiemi di segnali
  - **n segnali di ingresso**
  - **k segnali di stato presente**
- Una volta che ho disponibile lo stato presente che riassume la storia, le uscite della rete dipendono solo dagli ingressi correnti e dallo stato presente. Sono quindi **funzioni combinatorie** di questi segnali.

Rete Sequenziale Asincrona (RSA)

- Quindi una RSA è composta da **$m + k$ reti combinatorie ognuna con $n + k$ ingressi**, sintetizzabili con una delle metodologie viste fin ora
- Indicheremo con F le funzioni che calcolano i bit di uscita e con G le funzioni che calcolano i bit di stato futuro

$$z_i = F_i(x_0, \dots, x_{n-1}, y_0, \dots, y_{k-1})$$
$$Y_i = G_i(x_0, \dots, x_{n-1}, y_0, \dots, y_{k-1})$$

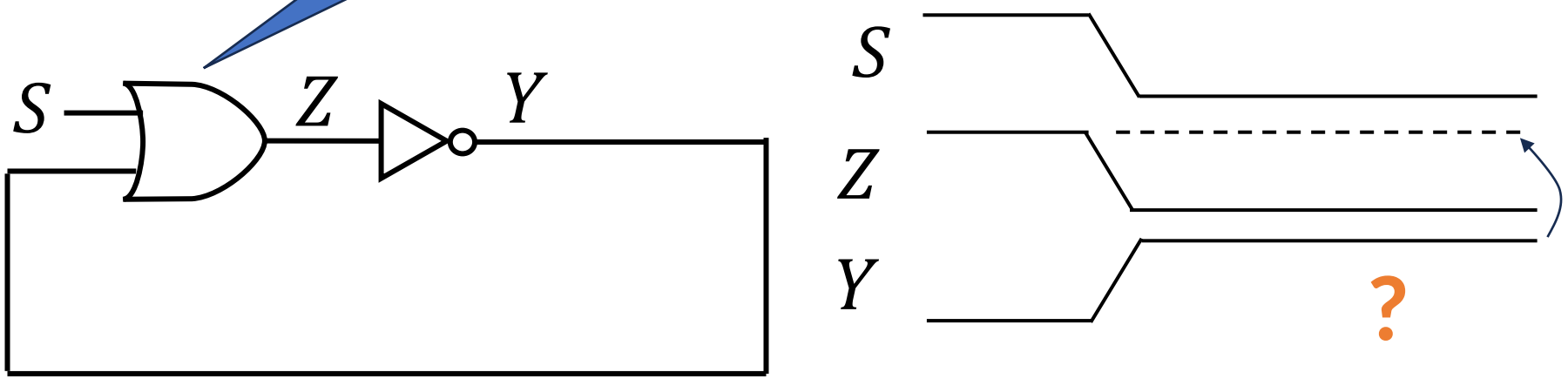
=



Retroazione diretta

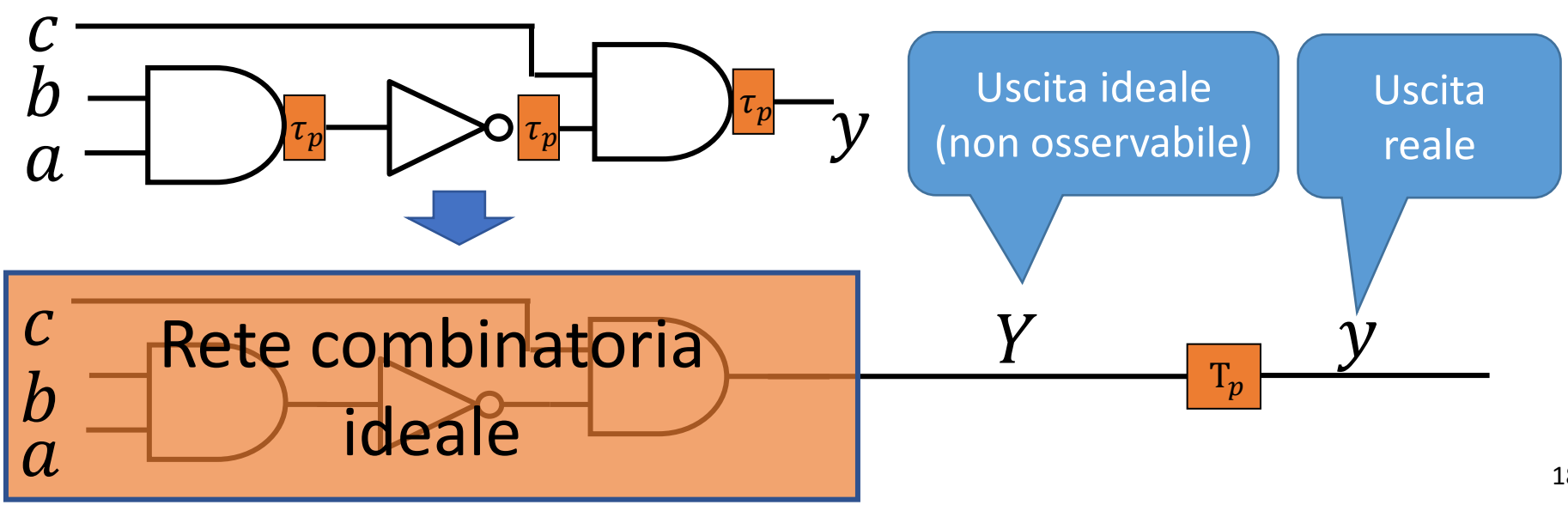
- E' possibile collegare l'uscita di una rete combinatoria in retroazione diretta su uno dei suoi ingressi?
- Trattando i gate come pure relazioni matematiche specificate dalle loro tabelle della verità sembrerebbe impossibile.

Quando S=0, questo OR è come un cortocircuito (E5 – identità). Quindi Z=Y. Ma Y=Z'. Contraddizione



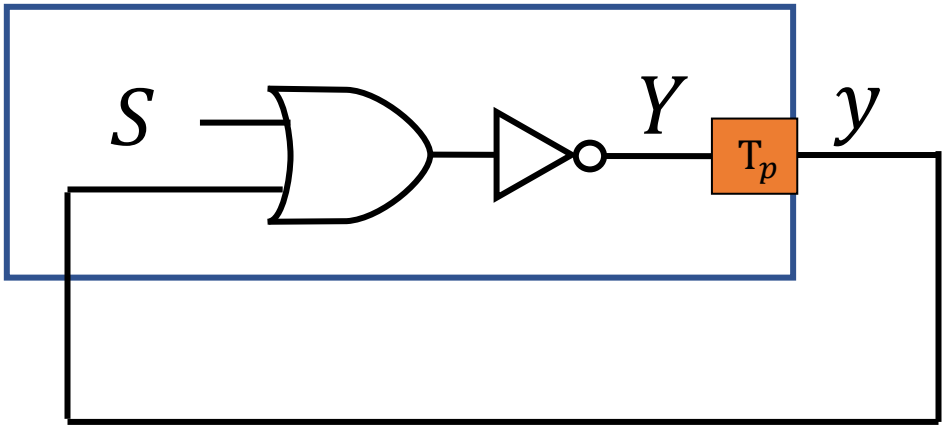
Ritardo di una rete combinatoria

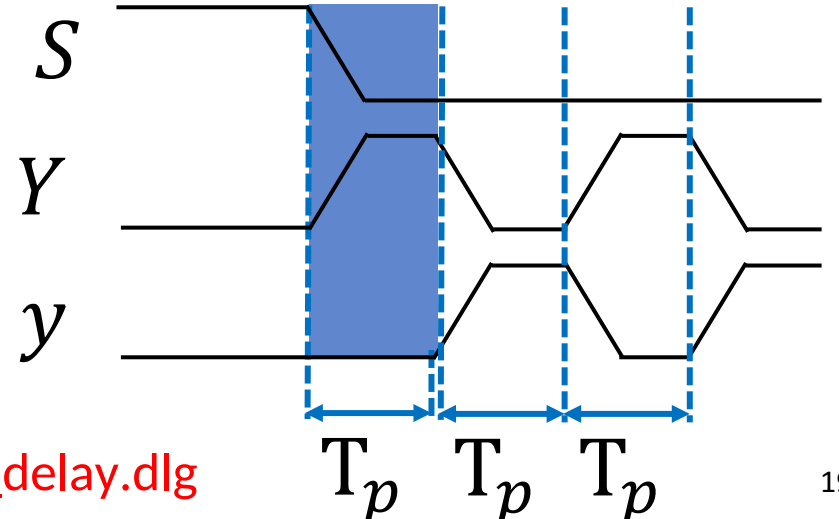
- Abbiamo già visto che ogni gate reale ha un **ritardo di propagazione:** quando cambia un ingresso di un gate, **l'uscita non cambia istantaneamente**, ma dopo un tempo τ_p che dipende dalla tecnologia utilizzata
- Abbiamo anche definito il ritardo complessivo di una rete combinatoria come il ritardo di caso peggiore $N_{casc} * \tau_p = \mathrm{T}_p$
- Possiamo quindi immaginare di avere in uscita dalla rete **un segnale ideale**, che cambia istantaneamente al variare degli ingressi, e **un segnale reale**, che cambia dopo il ritardo complessivo della rete.



Retroazione diretta con ritardi

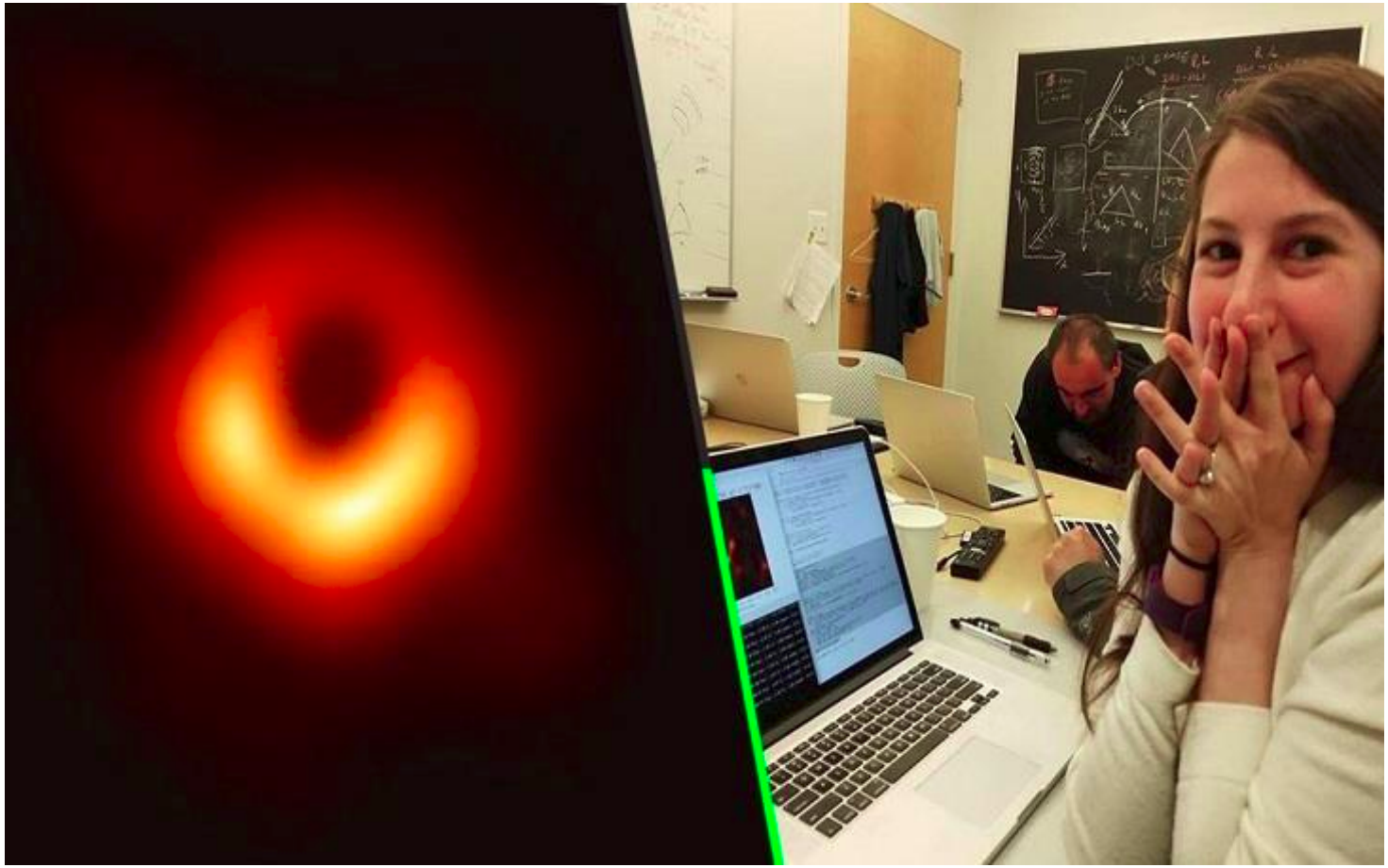
- Un altro modo di vedere il ritardo di propagazione: al variare di un ingresso che provoca la variazione dell'uscita di una rete combinatoria, **la rete ricorda il precedente valore dell'uscita per un tempo T_p**, prima di sostituirla col nuovo valore.
- Il tempo di propagazione agisce da brevissima **«memoria»** del precedente valore di uscita.
- Grazie ai ritardi della rete, è possibile connettere reti combinatorie in **retroazione diretta** garantendo che la rete calcoli correttamente la nuova uscita





Esempio con Digital: 6_delay.dlg

Ritardo = memoria

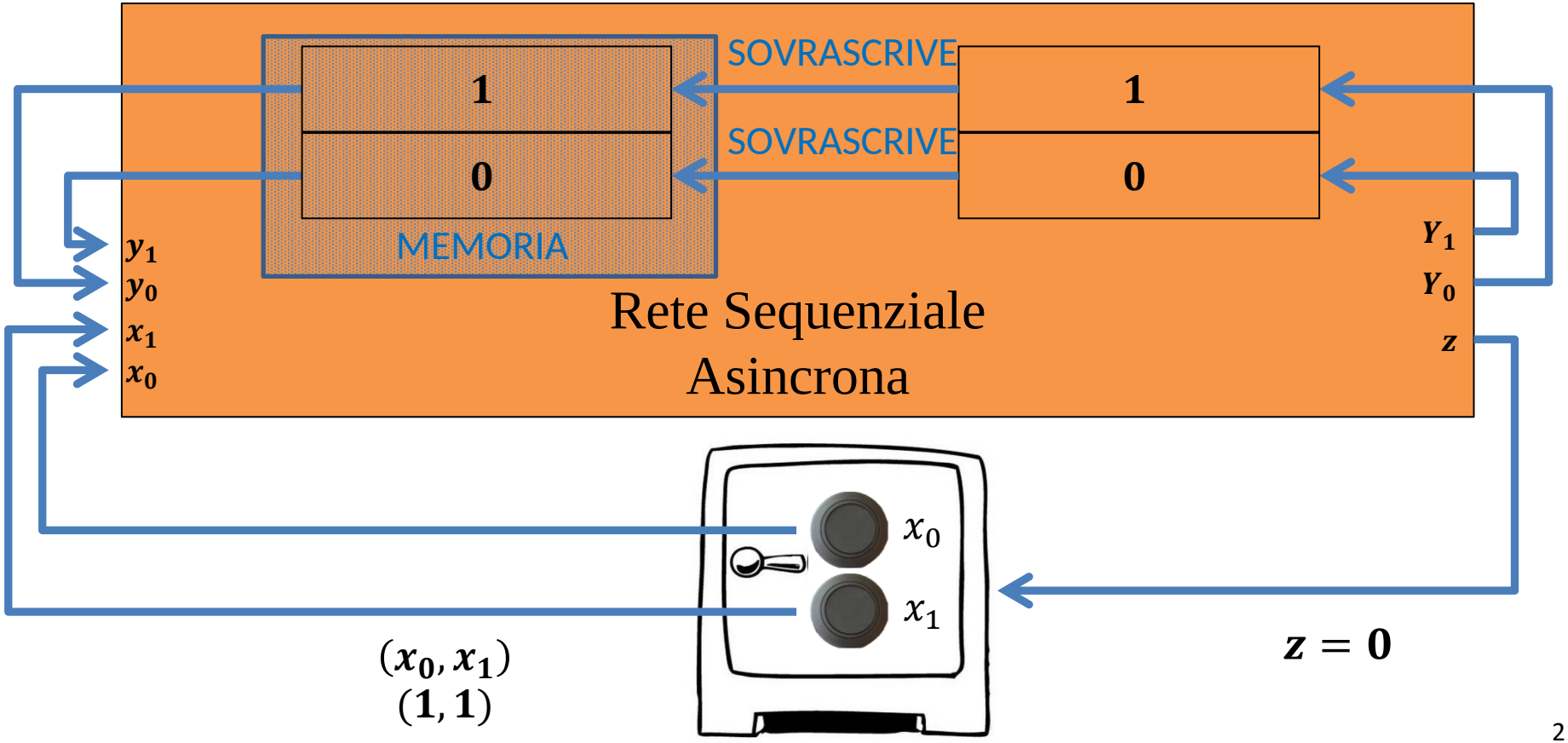


La prima immagine di un buco nero ritrae qualcosa distante 55 milioni di anni luce da noi. Ovvero stiamo vedendo **adesso** come appariva **55 milioni di anni fa**, grazie al **ritardo** di propagazione della luce. Rendere disponibile il passato nel presente non è forse la definizione di memoria?

«Memoria» in una RSA

In una RSA, lo stato futuro appena disponibile diventa stato presente.

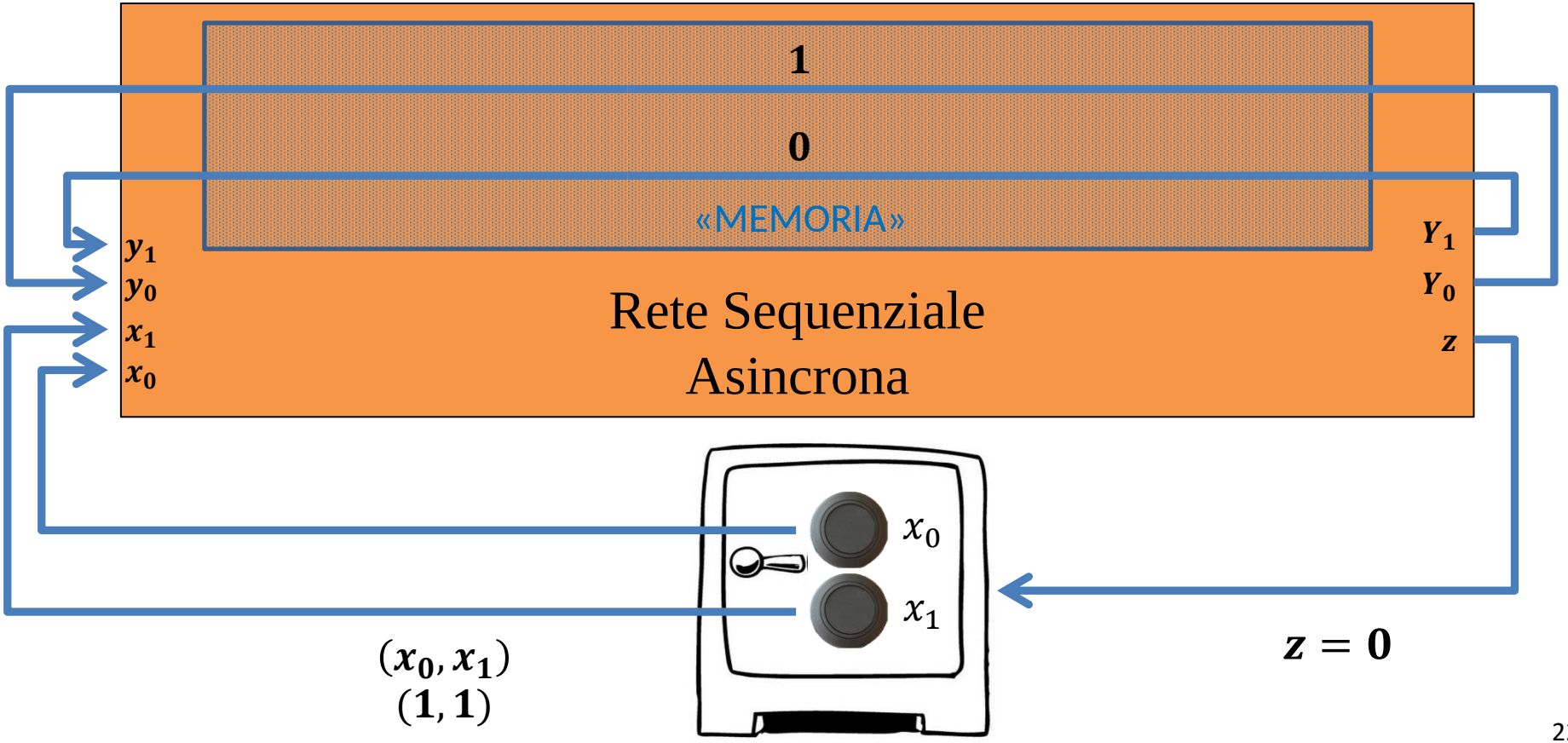
La «memoria» in una RSA deve quindi ricordare lo stato presente solo per un tempo molto breve, al fine di permettere alle reti che calcolano lo stato futuro di lavorare con ingressi costanti.



«Memoria» in una RSA

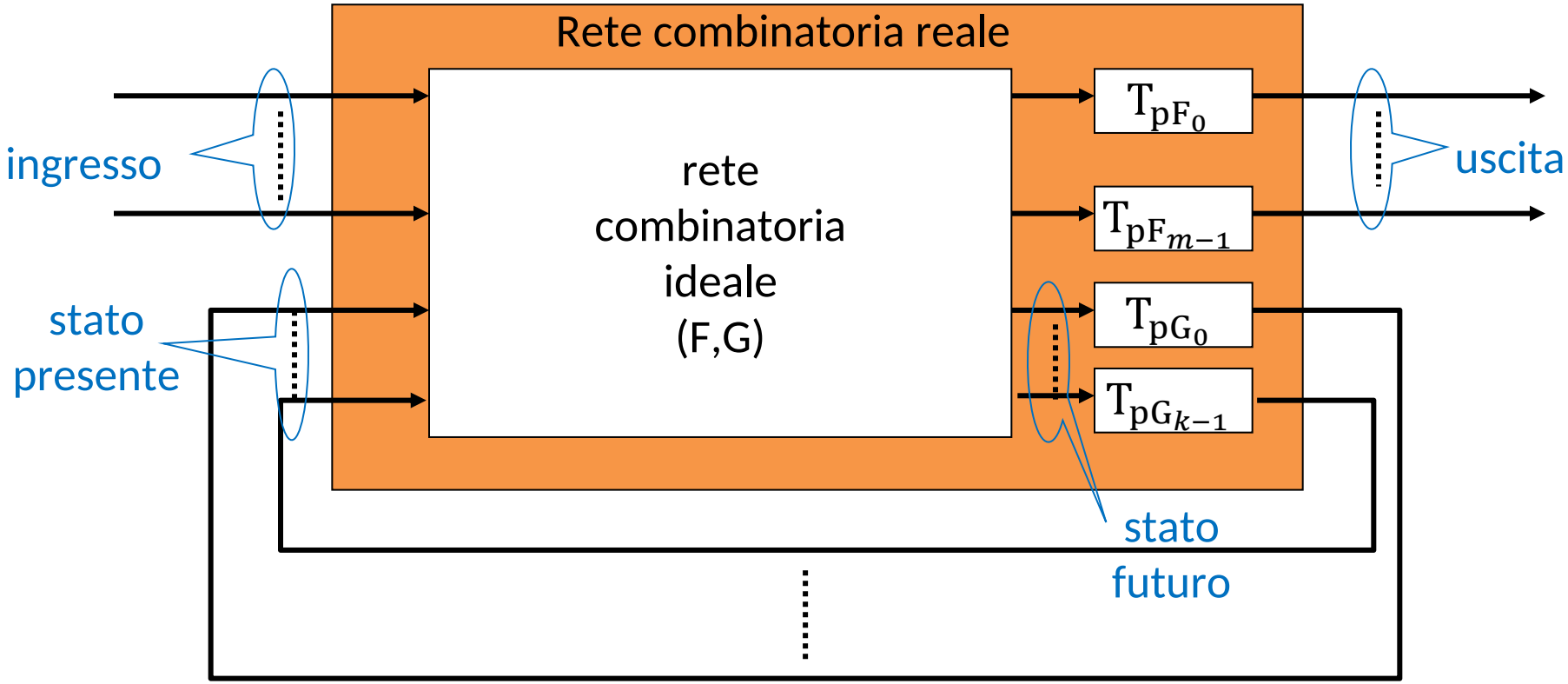
Il **ritardo delle reti combinatorie** che calcolano lo stato futuro è sufficiente a realizzare questa «memoria» di una RSA.

Possiamo quindi connettere i segnali di stato futuro/presente in **retroazione diretta**. Il valore «memorizzato» dallo stato è quello che leggo sui segnali in retroazione.



Struttura generale di una RSA

- **Ogni rete combinatoria con <u>anelli di retroazione diretta</u> è quindi in realtà una RSA**
- La rete ha k bit di stato se ci sono k segnali in retroazione
- I ritardi della rete combinatoria G agiscono da «memoria»
- Quindi lo stato futuro non è in realtà osservabile. Lo è solo quando diventa stato presente.



La FSM (Finite State Machine)

Una RSA è un caso particolare di Automa (o Macchina) a stati finiti M, ovvero un sistema matematico

$$\boldsymbol{FSM} = \{\boldsymbol{I}, \boldsymbol{U}, \boldsymbol{S}, \boldsymbol{F}, \boldsymbol{G}\}$$

formato da 3 **insiemi**

$\boldsymbol{I}$: $\{\boldsymbol{i_1}, \boldsymbol{i_2}, \dots, \boldsymbol{i_N}\}$ *alfabeto di ingresso*

$\boldsymbol{U}$: $\{\boldsymbol{u_1}, \boldsymbol{u_2}, \dots, \boldsymbol{u_M}\}$ *alfabeto di uscita*

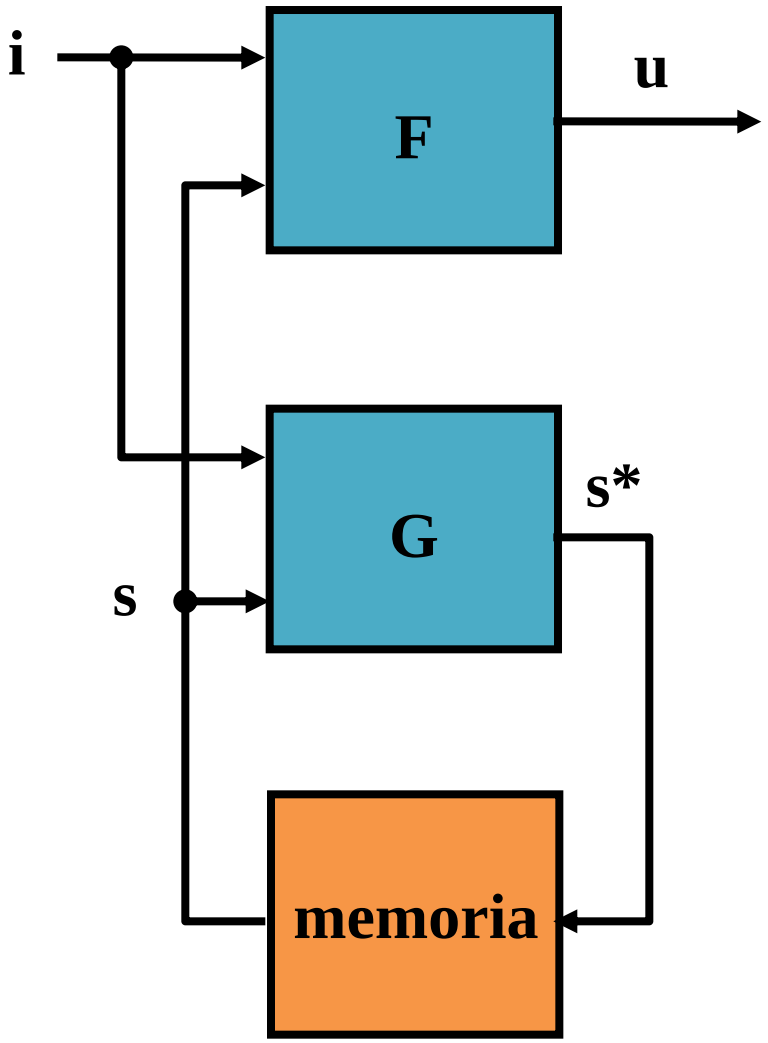
$\boldsymbol{S}$: $\{\boldsymbol{s_1}, \boldsymbol{s_2}, \dots, \boldsymbol{s_K}\}$ *insieme degli stati*

e da 2 **funzioni**

$\boldsymbol{F} : \boldsymbol{S} \times \boldsymbol{I} \rightarrow \boldsymbol{U}$ *funzione di uscita*

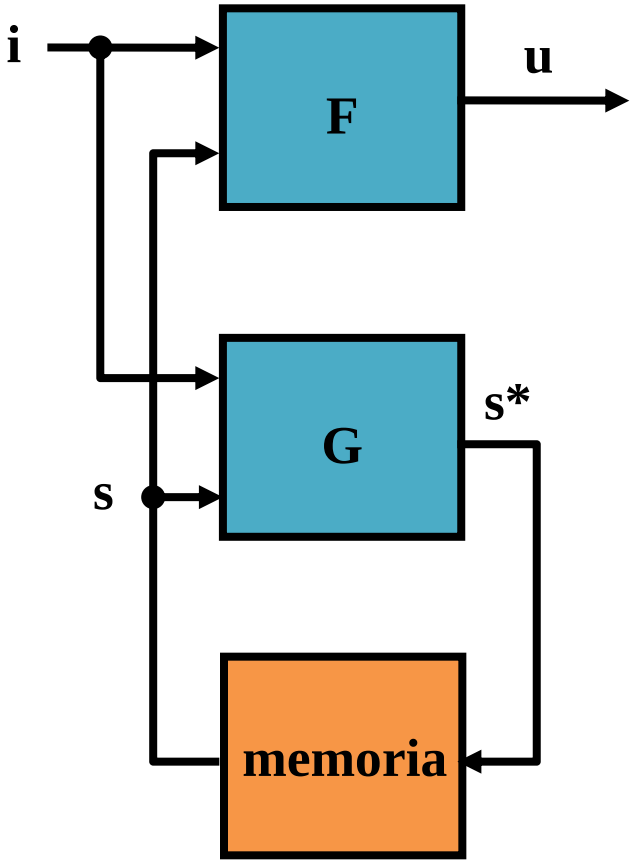
$\boldsymbol{G} : \boldsymbol{S} \times \boldsymbol{I} \rightarrow \boldsymbol{S}$ *funzione di aggiornamento dello stato interno*

Nella realizzazione occorre una memoria *che mantenga il "vecchio stato" s fino a quando non è necessario sostituirlo con il "nuovo stato" s**

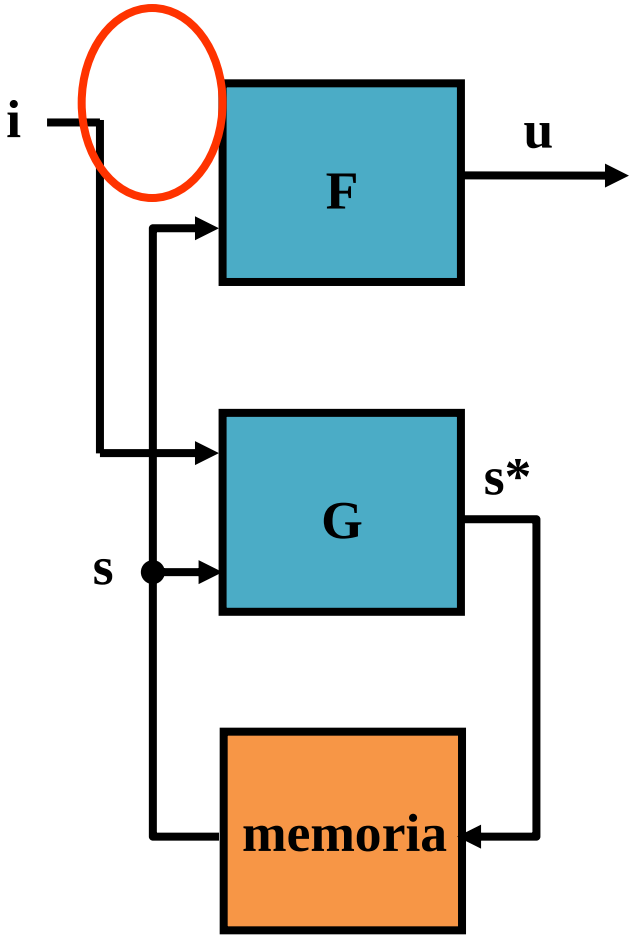


Automa di Mealy e Automa di Moore

- **Automa di Mealy**: quello che abbiamo descritto nella slide precedente
- **Automa di Moore**: caso particolare dell'automa di Mealy, in cui l'uscita dipende solo dallo **stato presente**



Mealy
$F : S \times I \rightarrow U$



Moore
$F : S \rightarrow U$

COMPORTAMENTO E STRUTTURA

Reti Combinatorie

Comportamento
(cosa fa la rete)

Tabella della verità

w	x	y	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
…	…	…	…

sintesi

analisi

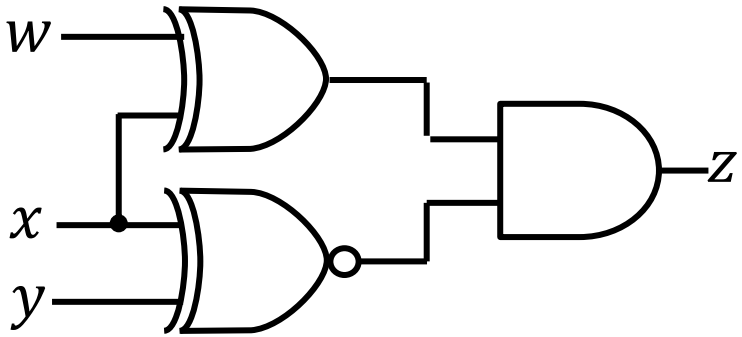
Struttura
(come è realizzata la rete)

Espressione

$$z = (x \equiv y)(x \oplus w)$$

o, <u>in modo equivalente</u>,

Schema logico



Reti Sequenziali Asincrone

Comportamento
(cosa fa la rete)

Grafo degli stati

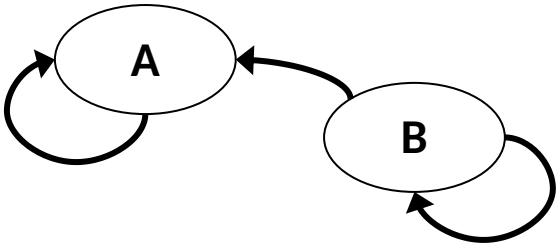


Tabella di flusso

	00	01	11
A	$A, 0$	$A, 0$	$B, -$
B	$B, 1$	$A, -$	$B, 1$
C	$A, -$	$C, 1$	$C, 1$
D	$A, -$	$-, -$	$C, -$
…	…	…	…

sintesi

analisi

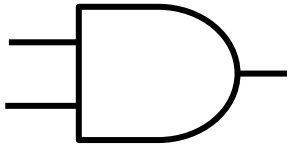
Struttura
(come è realizzata la rete)

Espressioni

$$F_i = (x_0 + y)x_1$$

o, <u>in modo equivalente</u>,

Schemi logici

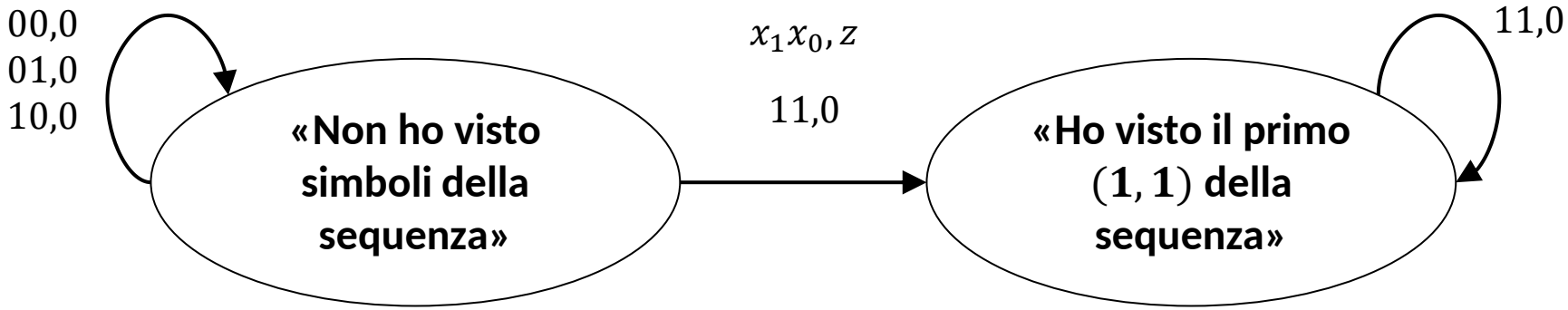


delle $m + k$ reti combinatorie che la compongono

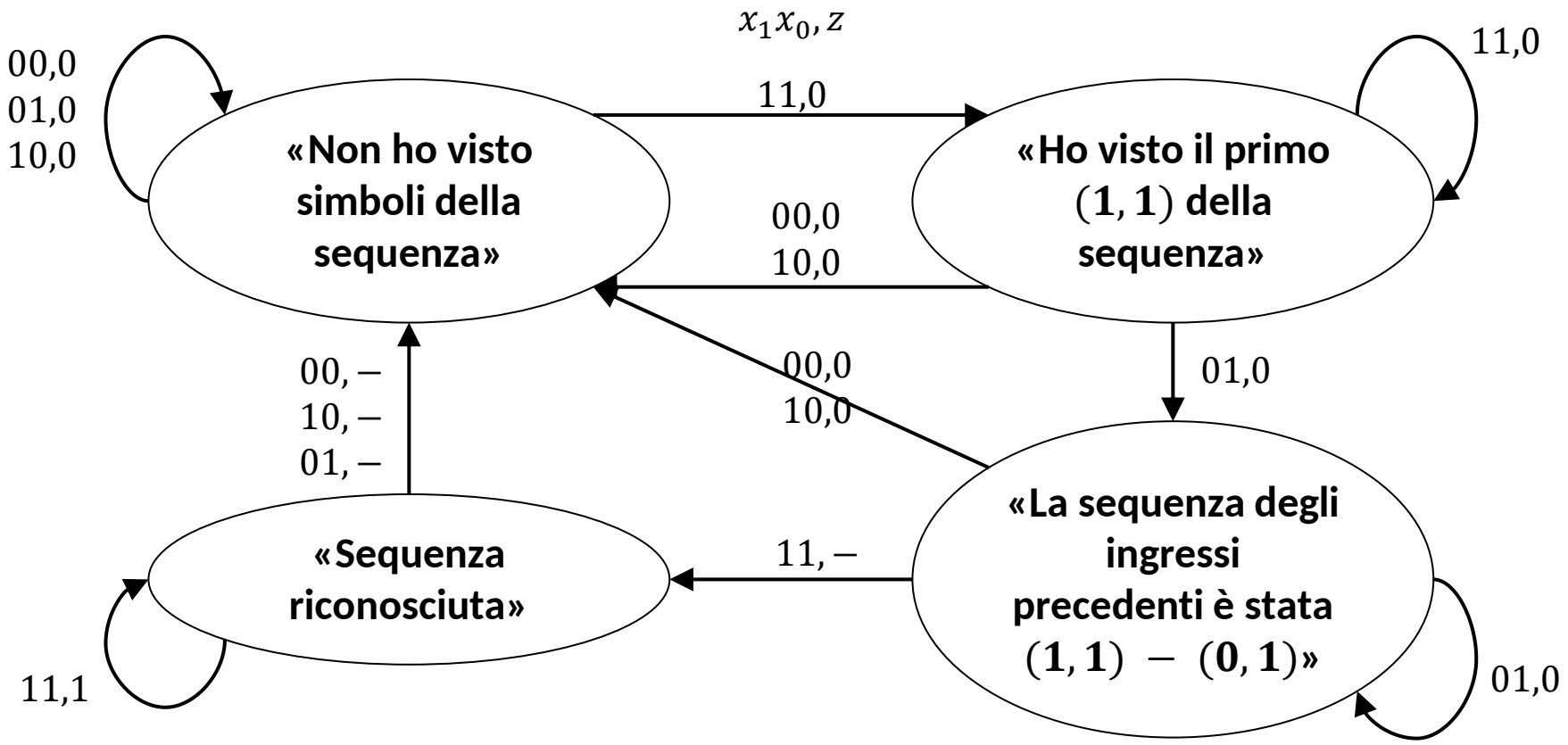
Il grafo degli stati

- Grafo (o diagramma) degli stati: grafo ad archi orientati in cui
  - **ogni nodo rappresenta lo stato presente**
  - **ogni arco una transizione da stato presente a stato futuro** in corrispondenza di una o più configurazioni binarie degli ingressi
- Da ogni nodo devono uscire tanti rami quante sono le **configurazioni binarie possibili degli ingressi** (N.B.: configurazioni **possibili**, ci torneremo)
- **Stato stabile**: nodo con un arco che ritorna sul nodo da cui parte
- Nel caso di un automa di Mealy, si indica il valore dell'uscita su ciascuna transizione, in quanto l'uscita dipende dallo stato presente **e dall'ingresso corrente**

- Come si legge la parte di grafo sottostante? Se lo stato presente, memorizzato nei segnali di stato in retroazione, è **«Non ho visto simboli della sequenza»**, e gli ingressi valgono «00», «01», o «10», allora lo stato futuro è lo stato presente e l'uscita vale «0». Quando invece gli ingressi assumono valore «11», lo stato futuro è **«Ho visto il primo $(\mathbf{1},\mathbf{1})$ della sequenza»**, e l'uscita continua a valere «0». Quando, dopo un breve tempo T_p , lo stato futuro diventa stato presente, la rete rimane in quello stato fintantoché gli ingressi valgono «11», e l'uscita continua a valere «0».

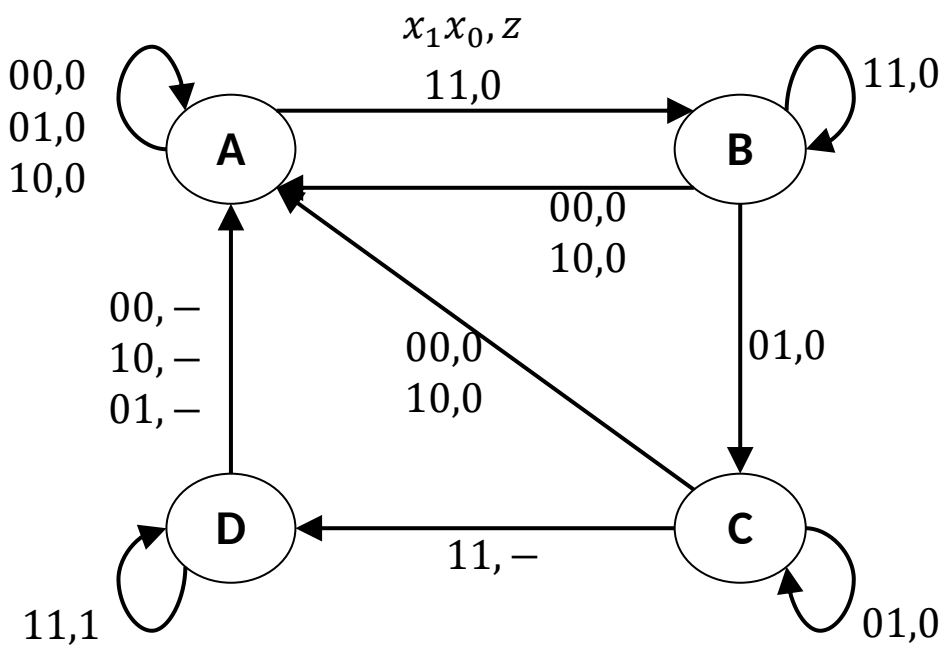


Grafo della cassaforte (Mealy)



Nomi simbolici per gli stati

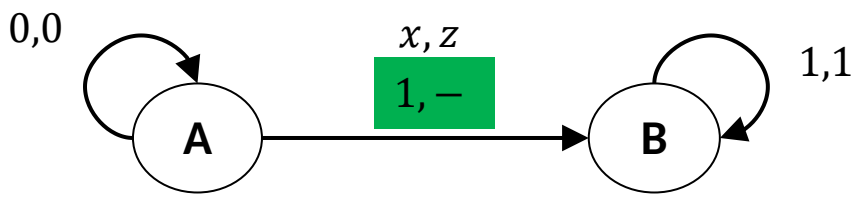
- Tipicamente, per rendere il grafo più compatto e gestibile, non si riporta la descrizione di cosa «memorizza» lo stato, ma un nome simbolico A,B,C,… e si lascia l'interpretazione sul significato al contesto
- Noi riporteremo anche una tabella col «significato» degli stati



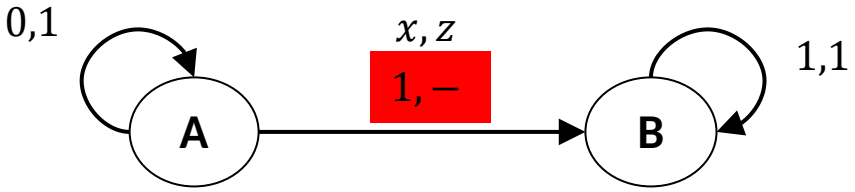
Simbolo	Riassunto
A	**«Non ho visto simboli della sequenza»**
B	**«Ho visto il primo $(1,1)$ della sequenza»**
C	**«La sequenza degli ingressi precedenti è stata $(1,1) - (0,1)$»**
D	**«Sequenza riconosciuta»**

Indifferenza sull'uscita

- Quando inserisco una condizione di indifferenza nelle transizioni tra stato presente e stato futuro nel caso di automi di Mealy?
- **Quando l'uscita cambia valore tra la stabilità nello stato presente e quella nello stato futuro**
- Perché non mi interessa il valore dell'uscita in questo caso? Perché la transizione dura il tempo brevissimo del ritardo T_p sull'anello di retroazione, e salvo casi particolari non è importante se l'uscita mantiene il vecchio valore per qualche ns o assume il nuovo valore qualche ns prima



OK: L'uscita è diversa tra stato di partenza e di arrivo, non è importante quale valore abbia durante la transizione

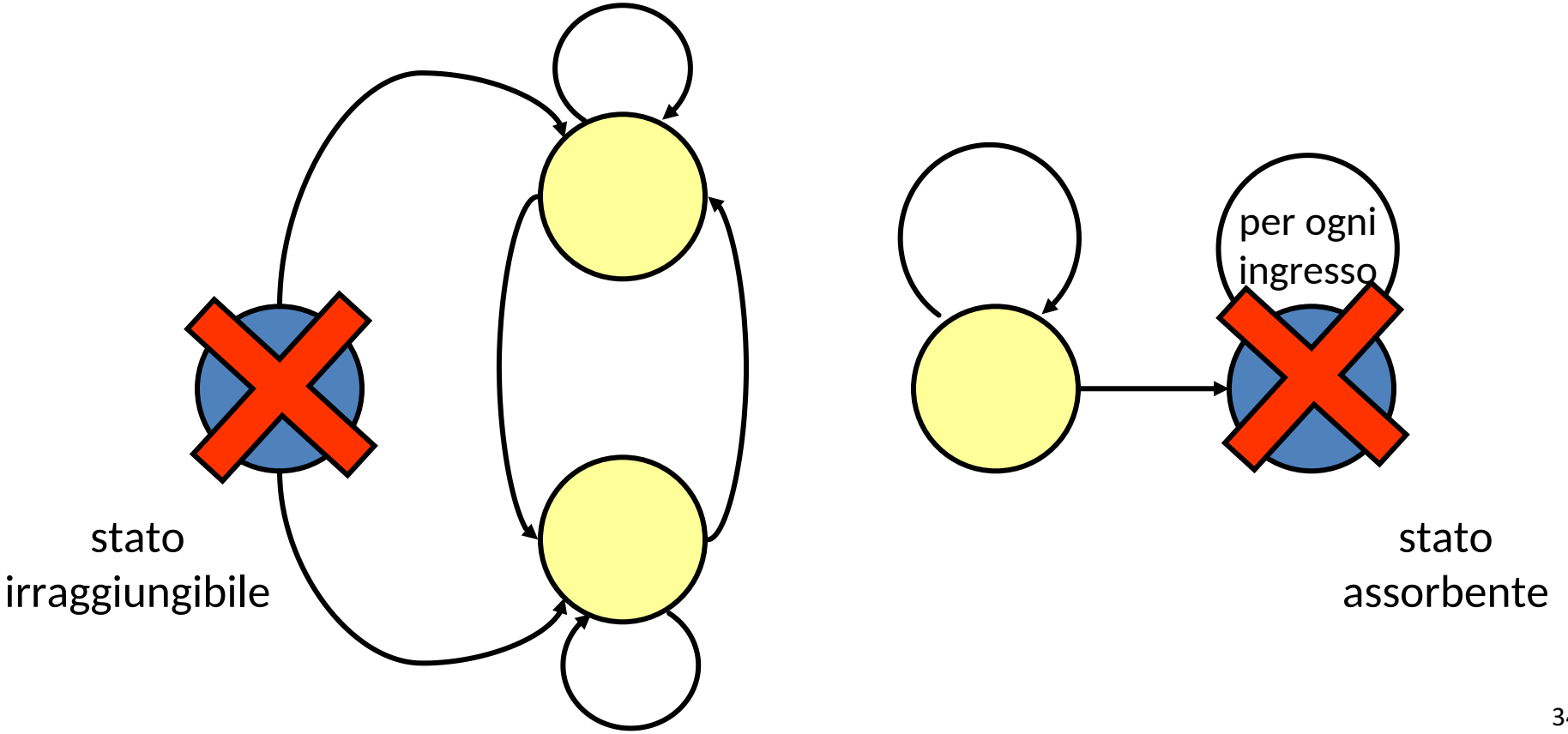


NO: L'uscita vale 1 sia nello stato presente che in quello futuro, deve quindi rimanere a 1 anche durante la breve transizione per evitare *glitch* sull'uscita stessa

Grafi strettamente connessi

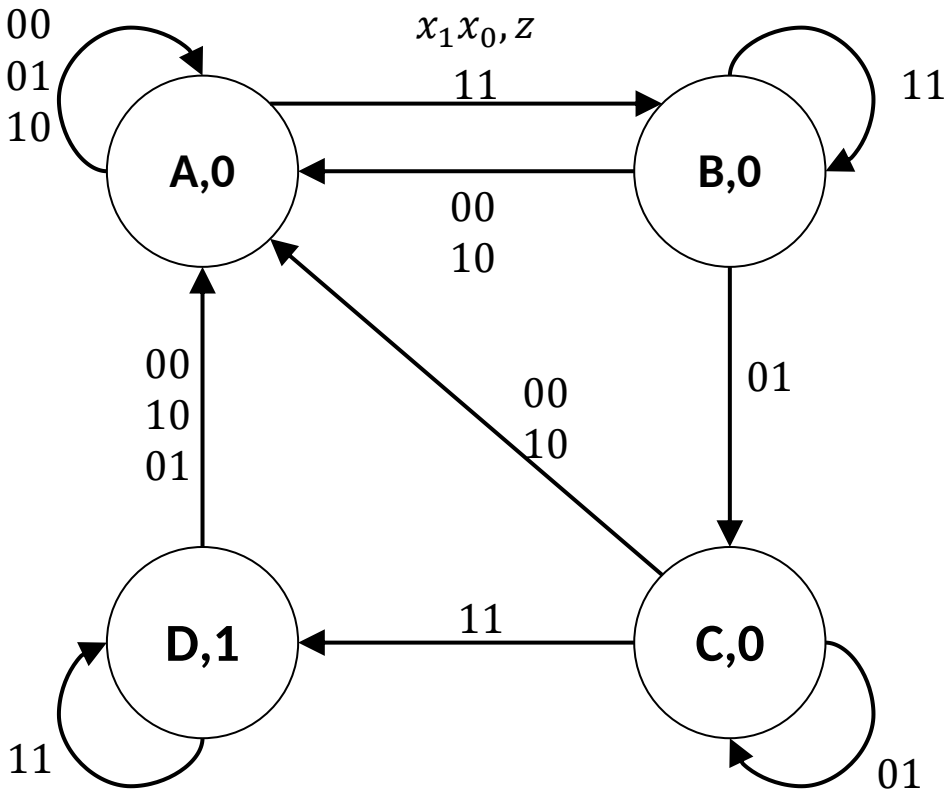
Per progettare macchine che operano con continuità

- occorre che esista sempre almeno un percorso per passare da un nodo arbitrariamente scelto a un altro (**grafo strettamente connesso**)
- occorre evitare **stati (o parti di grafo) irraggiungibili** (dotati di sole frecce uscenti) o **stati (o parti di grafo) assorbenti** (dotati di sole frecce entranti)

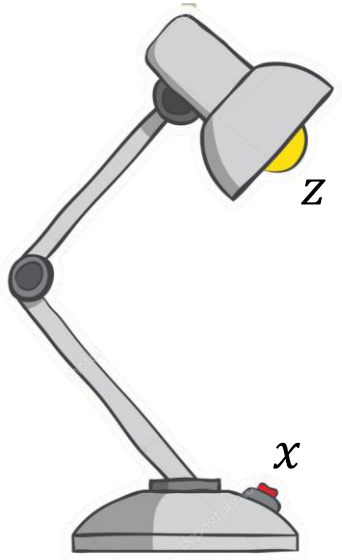


Grafo della cassaforte (Moore)

- Nel caso di automa di Moore, l'uscita dipende solo dallo stato presente.
- **La si indica quindi nello stato** e non nelle transizioni.
- Non è il caso di questo esempio, ma l'automa di Moore in generale può richiedere più stati rispetto a Mealy

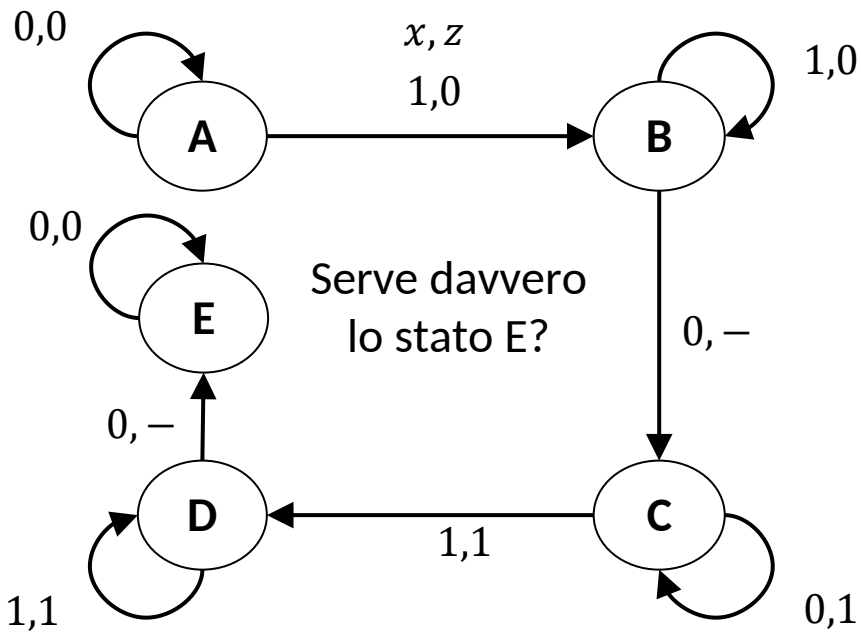


Simbolo	Riassunto
A	**«Non ho visto simboli della sequenza»**
B	**«Ho visto il primo $(1,1)$ della sequenza»**
C	**«La sequenza degli ingressi precedenti è stata $(1,1) - (0,1)$»**
D	**«Sequenza riconosciuta»**

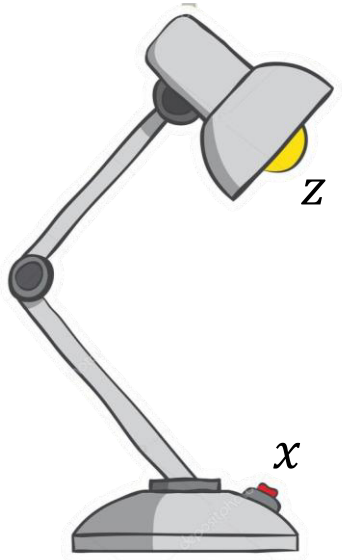


Lampada da tavolo

- Anche il comportamento di una lampada da tavolo è sequenziale asincrono: ogni volta che **premo e rilascio** il pulsante x la luce z si accende **se era spenta** e si spegne **se era accesa**
- Supponiamo che **all'accensione la lampada sia spenta**

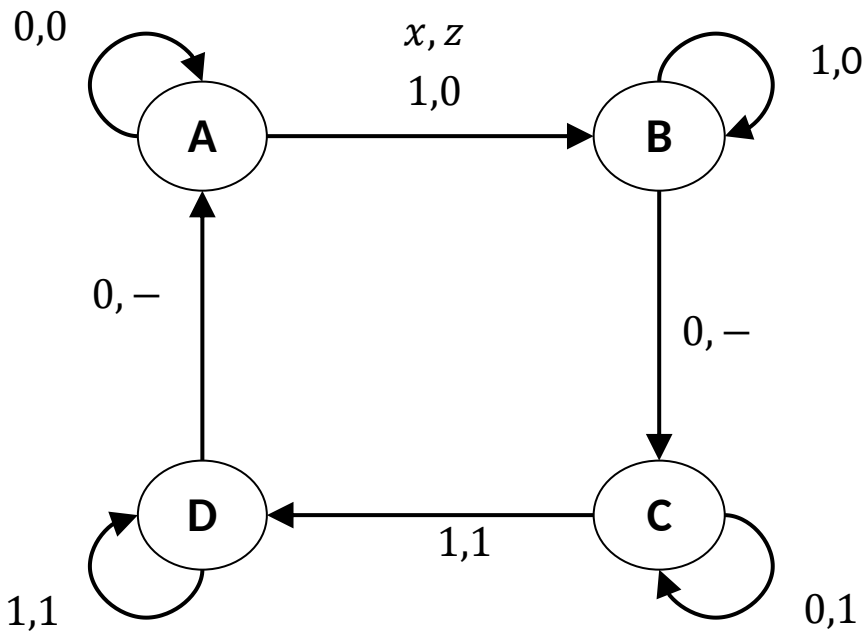


Simbolo	Riassunto
A	**«Il pulsante è stato premuto zero volte»**
B	**«Sto passando da 0 a una pressione»**
C	**«Il pulsante è stato premuto una volta»**
D	**«Sto passando da una a due pressioni»**
E	**«Il pulsante è stato premuto due volte»**



Lampada da tavolo

- Il fatto che il pulsante sia stato premuto 0, 2, 4 o un qualunque numero pari di volte **non fa differenza per la rete**.
- Posso quindi riusare gli stati A,B dandogli un significato più generale.
- Stesso ragionamento per gli stati C,D e un numero dispari di pressioni

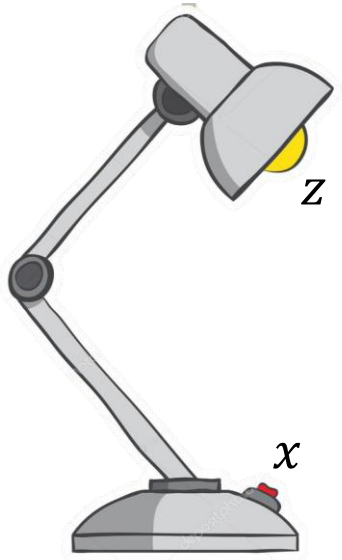


Simbolo	Riassunto
A	**«Il pulsante è stato premuto un numero pari di volte»**
B	**«Sto passando da numero di pressioni pari a dispari»**
C	**«Il pulsante è stato premuto un numero dispari di volte»**
D	**«Sto passando da numero di pressioni dispari a pari»**

Stato iniziale e reset

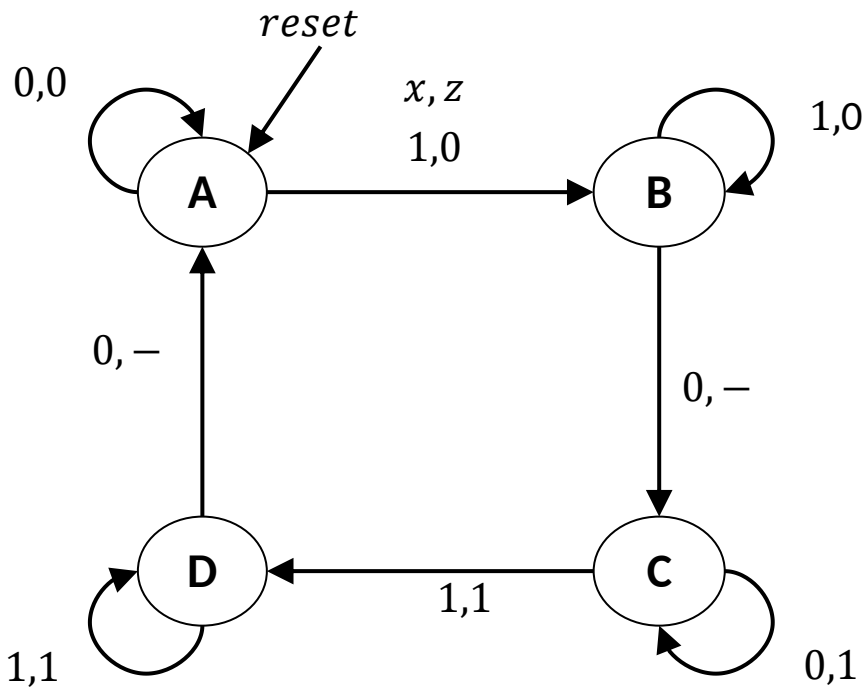
- Il comportamento di una rete sequenziale, essendo basato sulle variabili di stato che riassumono la storia passata del circuito, dipende dalla sequenza degli ingressi ma anche dallo **stato iniziale**.
- Nelle macchine elettroniche, il valore in cui si verranno a trovare i segnali in retroazione all'accensione è impredicibile (condensatori, velocità dei componenti, rumore, ...), e quindi non potrei avere certezza sullo stato iniziale.
- Perché è importante? Per esempio, la cassaforte potrebbe accendersi casualmente in stato D («sequenza riconosciuta»)...
- Tipicamente, quindi, è presente un ingresso aggiuntivo per la rete, detto ingresso di **reset**, che segnala alla rete quando la macchina si sta accendendo
- E' un segnale che assume valore «1» solo nei primi instanti di funzionamento della macchina e poi vale sempre «0».
- Può essere usato per forzare la RSA a iniziare la sua evoluzione da uno stato noto, in modo che il suo comportamento sia deterministico.

Esempio con Digital: 6_RSA_without_reset.dig, 6_RSA_with_reset.dig



Lampada da tavolo

- In precedenza abbiamo supposto che **all'accensione la lampada fosse spenta**
- Per garantire che questo accada, indichiamo nel grafo che l'ingresso aggiuntivo di reset porta la RSA in stato A



Simbolo	Riassunto
A	**«Il pulsante è stato premuto un numero pari di volte»**
B	**«Sto passando da numero di pressioni pari a dispari»**
C	**«Il pulsante è stato premuto un numero dispari di volte»**
D	**«Sto passando da numero di pressioni dispari a pari»**

Descrizione con tabella di flusso

- **Tabella di flusso:** modo alternativo di descrivere il *comportamento* di una RSA
- Nel caso di automa di Mealy, in ogni casella vengono riportati i valori delle funzioni G,F (ovvero dello stato futuro e dell'uscita) per il rispettivo ingresso e stato presente

insieme degli stati S

ingresso attuale

Insieme delle configurazioni d'ingresso

	i_1	i_2	....	i_i	....	i_N
s_1			....		....	
s_2			....		....	
⋮	⋮	⋮	⋮	⋮	⋮	⋮
s_j			....	s^*,u	....	
⋮	⋮	⋮	⋮	⋮	⋮	⋮
s_K				....	....	

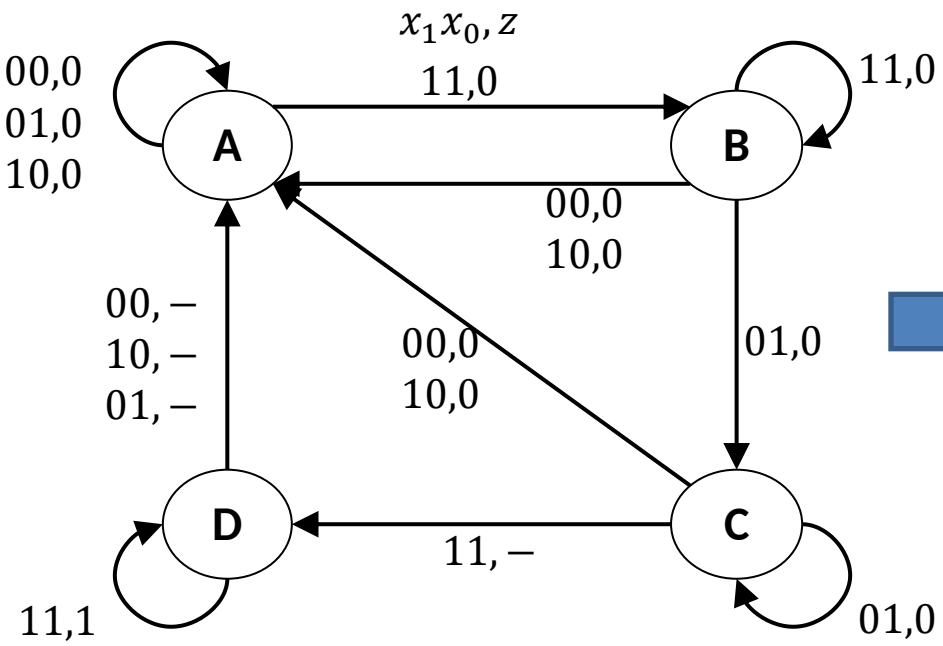
stato presente

stato futuro, uscita attuale

NB: la tabella, così come il grafo, **non specifica** l'istante in cui lo stato futuro diviene stato presente

Tabella di flusso (Mealy)

- Traduciamo il grafo in tabella di flusso per il caso della cassaforte
- Celle cerchiate indicano **condizioni di stabilità**
- Controlli formali di correttezza
  1. **Ogni riga deve avere almeno una condizione di stabilità** (stato futuro=stato presente)
  2. Le condizioni di instabilità **devono indicare uno stato futuro stabile (assenza di transizioni multiple)**



$x_1\ x_0$

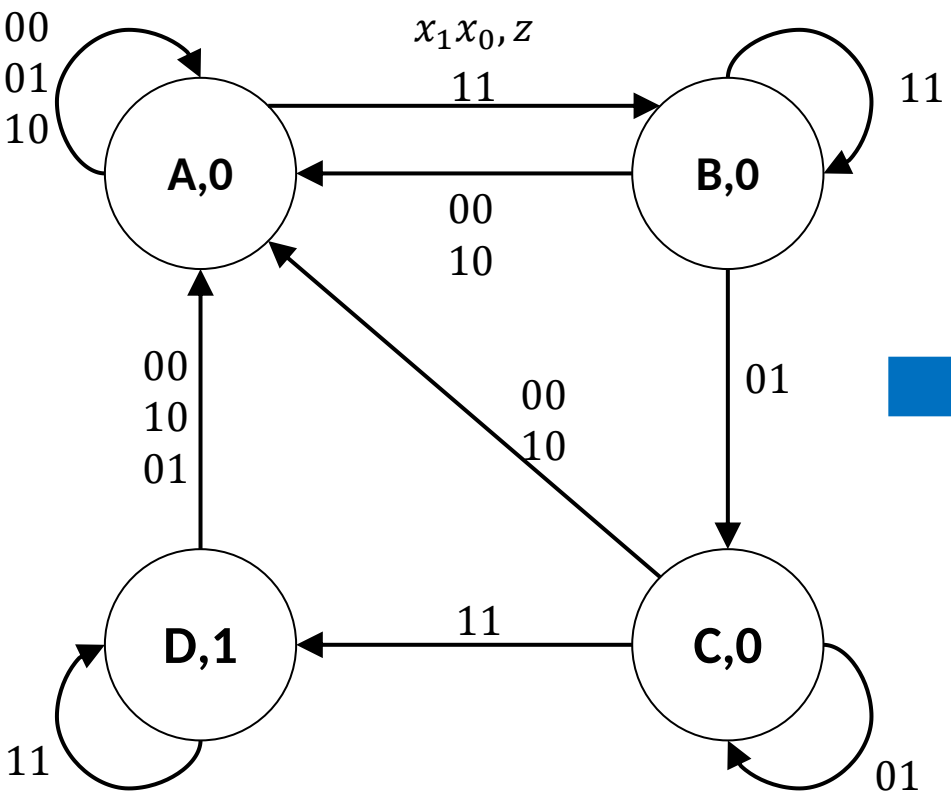
stato presente

	00	01	11	10
A	A,0	A,0	B,0	A,0
B	A,0	C,0	B,0	A,0
C	A,0	C,0	D,-	A,0
D	A,-	A,-	D,1	A,-

stato futuro, z

Tabella di flusso (Moore)

- Traduciamo il grafo in tabella di flusso per il caso della cassaforte con automa di Moore
- Poiché l'uscita dipende solo dallo stato presente, **è costante su ogni riga**
- Aggiungiamo quindi una colonna che riporta il valore dell'uscita per ogni stato



stato presente

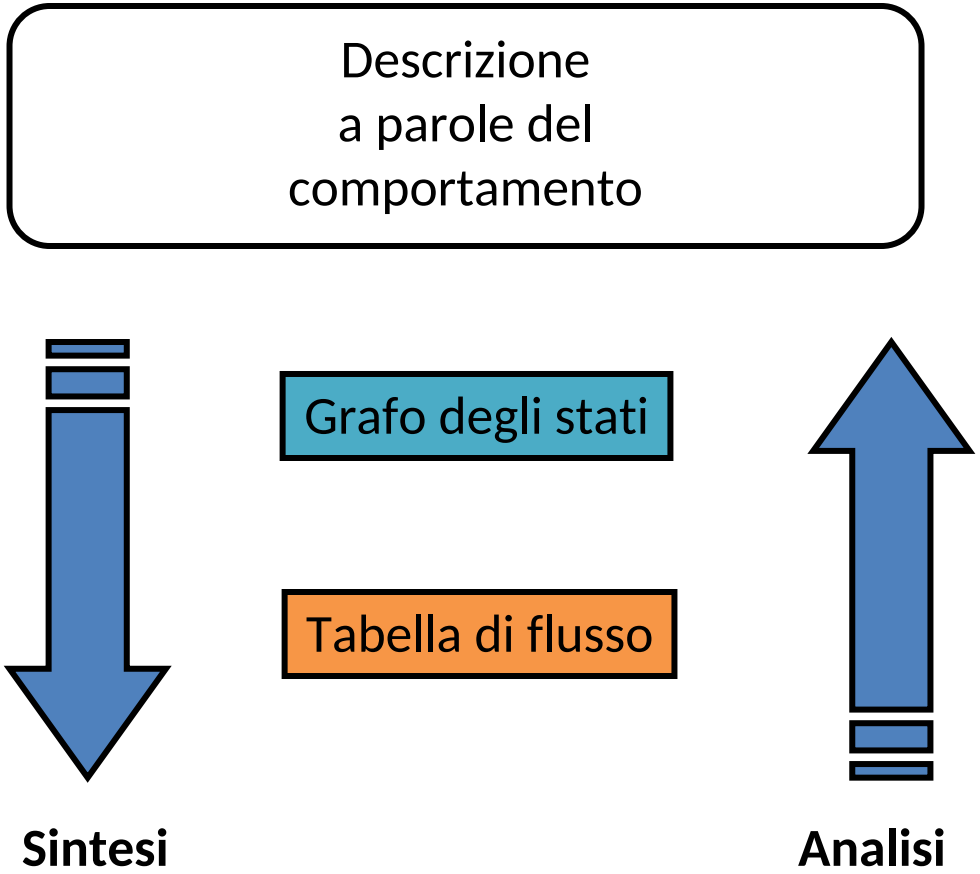
$x_1\ x_0$

	00	01	11	10	z
A	A	A	B	A	0
B	A	C	B	A	0
C	A	C	D	A	0
D	A	A	D	A	1

stato futuro

Analisi e Sintesi

- Grafo degli stati e tabella di flusso sono due strumenti diversi per descrivere una stessa macchina
- Il grafo degli stati è utile generalmente all'inizio del processo di **sintesi** (quando non è noto il numero di stati) e alla fine del processo di **analisi**
- La tabella di flusso (rappresentazione più ordinata) viene impiegata generalmente nel secondo passo della sintesi e nel penultimo passo dell'analisi



Sintesi di una RSA

Il procedimento di sintesi di una rete sequenziale asincrona è formato da 5 passi e consente di dedurne lo schema logico dal comportamento:

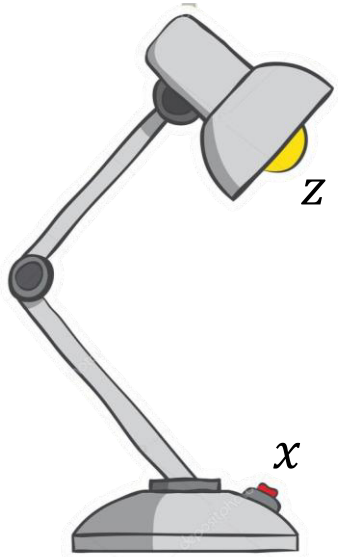
1: individuazione del grafo degli stati

2: definizione della tabella di flusso

3: codifica degli stati e definizione della tabella delle transizioni

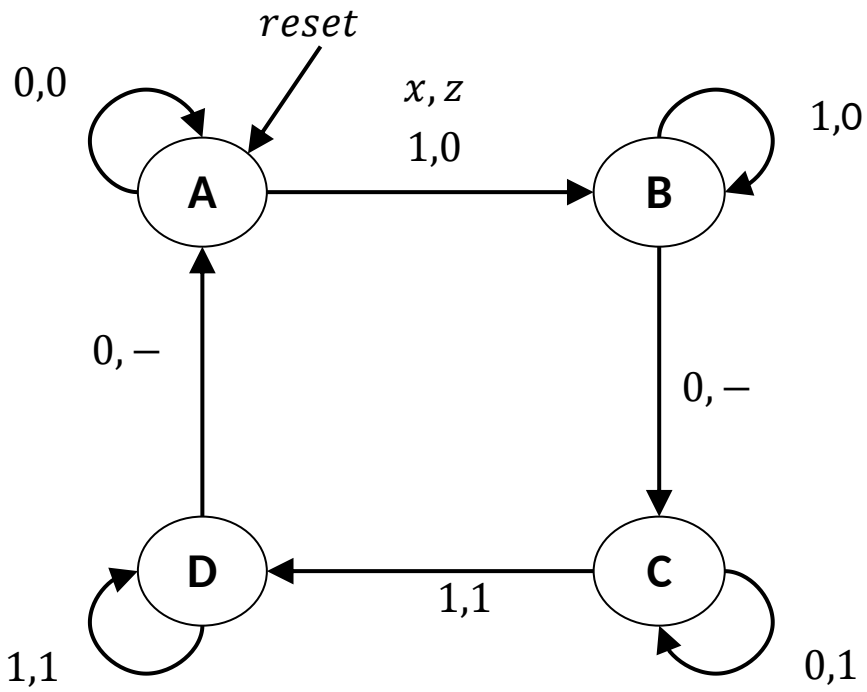
4: sintesi della reti combinatorie di uscita e stato futuro

5: schema logico che includa anche ingresso di reset, se presente



Esempio :Lampada da tavolo

- Abbiamo già risolto il passo 1, ovvero ricavato il grafo di una lampada da tavolo

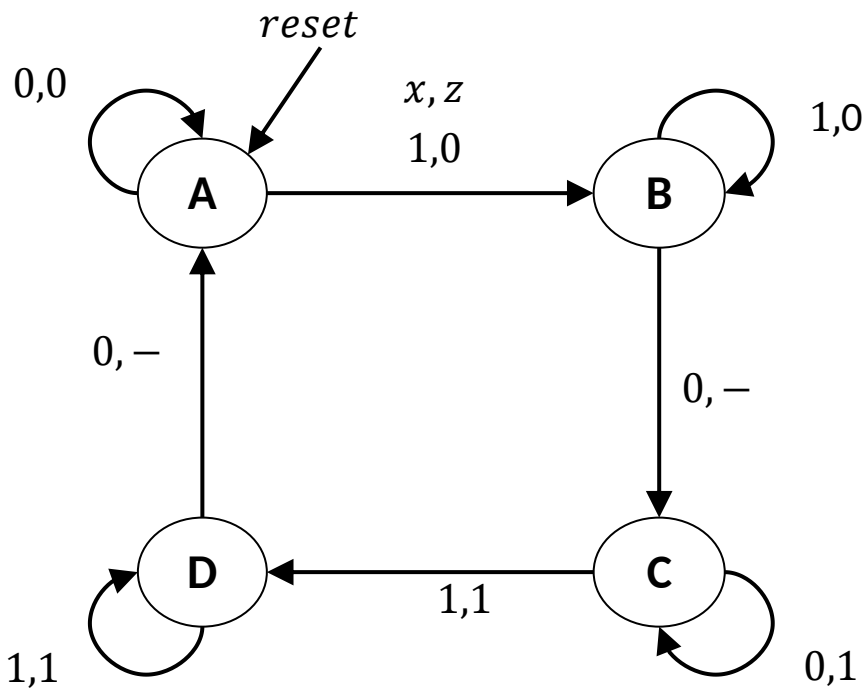


Simbolo	Riassunto
A	**«Il pulsante è stato premuto un numero pari di volte»**
B	**«Sto passando da numero di pressioni pari a dispari»**
C	**«Il pulsante è stato premuto un numero dispari di volte»**
D	**«Sto passando da numero di pressioni dispari a pari»**

Passo 2: Tabella di Flusso

Controlli formali

1. In ogni riga ci deve essere almeno una condizione di stabilità.
2. Le situazioni di instabilità devono indicare uno stato futuro stabile nella colonna (assenza di transizioni multiple).



stato presente \ x	0	1
A	A,0	B,0
B	C,-	B,0
C	C,1	D,1
D	A,-	D,1

stato futuro, z

Passo 3: tabella delle transizioni

- In ogni rete sequenziale, lo stato è rappresentato da una configurazione binaria dei bit di stato. Devo quindi scegliere **una codifica degli stati**
- Data la codifica, posso tradurre la tabella di flusso in **tabella delle transizioni**, sostituendo ad ogni stato presente e futuro la sua codifica binaria
- **Non tutte le codifiche producono un funzionamento corretto della rete**, come vedremo.
- **NOTA BENE**: la tabella delle transizioni **ha sempre un numero di righe pari a 2^k**, se i bit di stato necessari alla codifica sono k

Tabella di Flusso + Codifica degli stati = Tabella delle transizioni

x

stato presente	0	1
A	A,0	B,0
B	C,-	B,0
C	C,1	D,1
D	A,-	D,1

stato futuro, z

y_0

y_1	0	1
0	A	B
1	D	C

x

y_1y_0	0	1
A=00	00,0	01,0
B=01	11,-	01,0
C=11	11,1	10,1
D=10	00,-	10,1

Y_1Y_0, z

Passo 4: Espressioni combinatorie

La tabella delle transizioni è in realtà **una composizione di TdV combinatorie, ovvero di mappe di Karnaugh**, con cui è possibile eseguire la sintesi combinatoria delle funzioni che implementano il comportamento richiesto con le metodologie già studiate

y_1y_0 \ x	0	1
00	00,0	01,0
01	11,-	01,0
11	11,1	10,1
10	00,-	10,1

Y_1Y_0, z

x \ y_1y_0	00	01	11	10
0	0	-	1	-
1	0	0	1	1

$z = y_1$

x \ y_1y_0	00	01	11	10
0	0	1	1	0
1	0	0	1	1

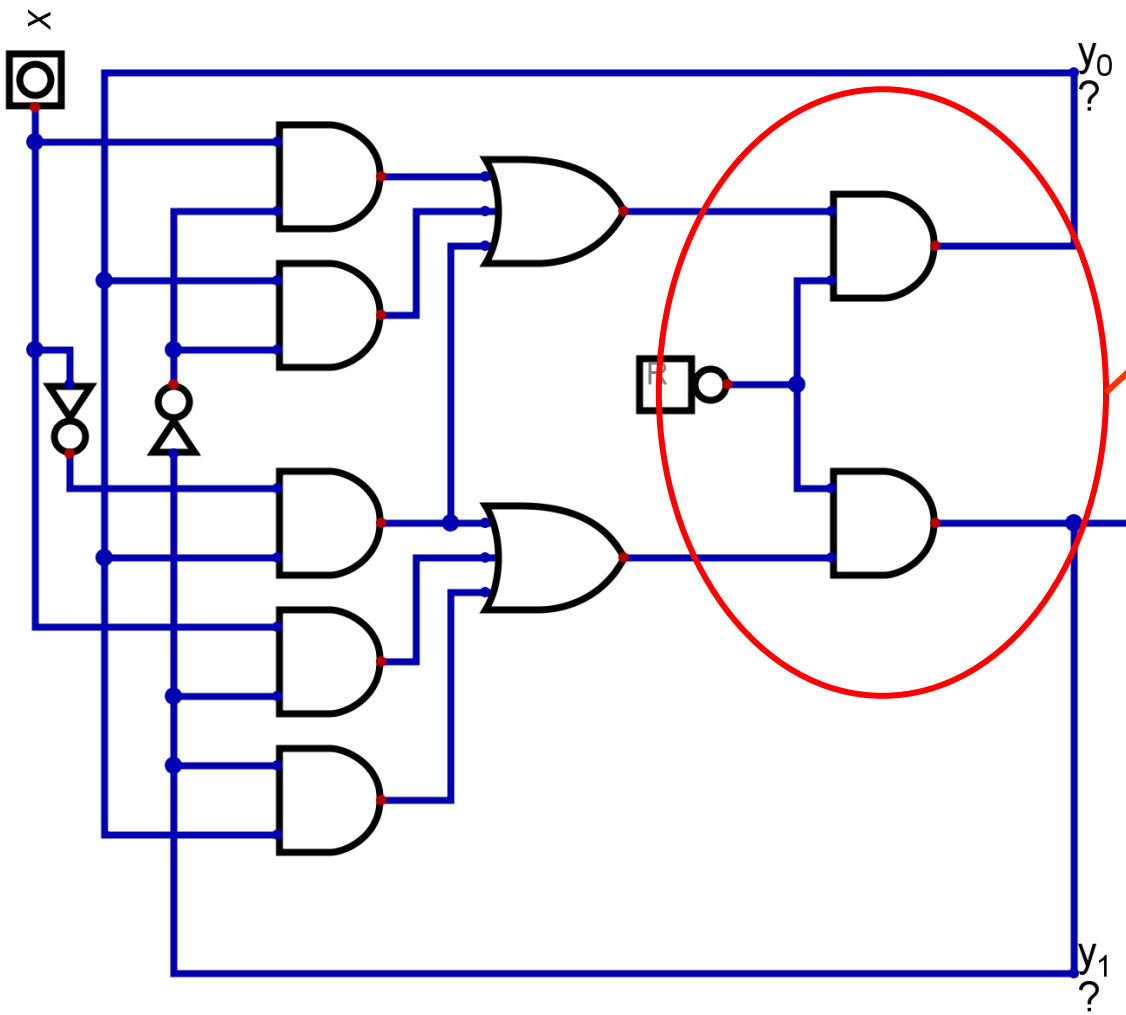
$Y_1 = xy_1 + x'y_0 + y_1y_0$

x \ y_1y_0	00	01	11	10
0	0	1	1	0
1	1	1	0	0

$Y_0 = xy_1' + x'y_0 + y_1'y_0$

Per evitare alee statiche (vedi dopo), eseguo una sintesi **non minima** di Y_0 e Y_1

Passo 5: schema logico e reset



$$Y_0 = xy_1' + x'y_0 + y_1'y_0$$

Rete di reset per garantire che lo stato presente sia A(=00) all'accensione

$$z = y_1$$

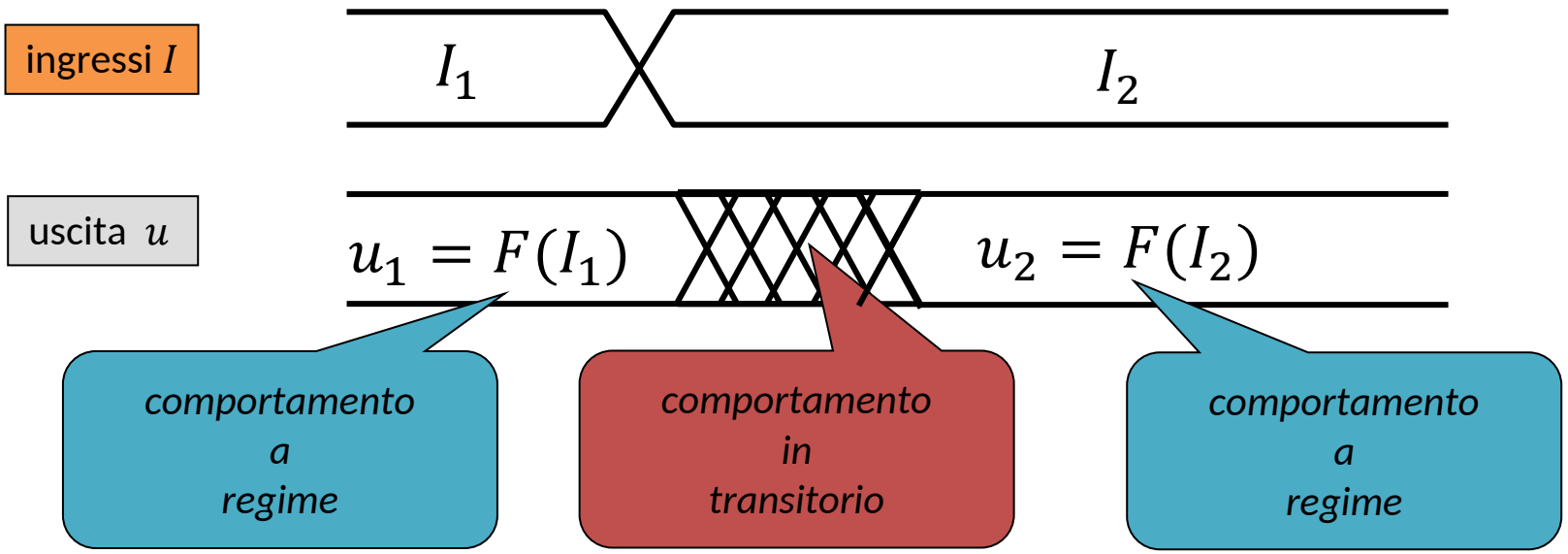
$$Y_1 = xy_1 + x'y_0 + y_1y_0$$

Async

ALEE DI RETI COMBINATORIE

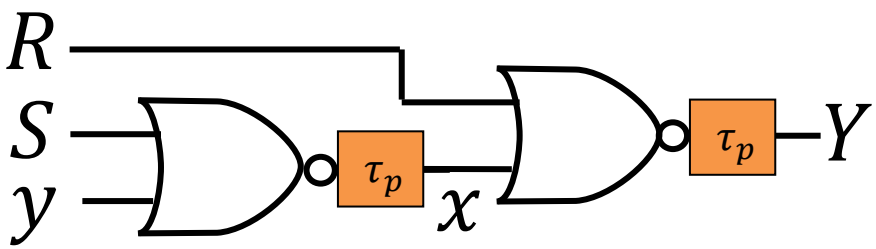
Comportamento *a regime* e *in transitorio*

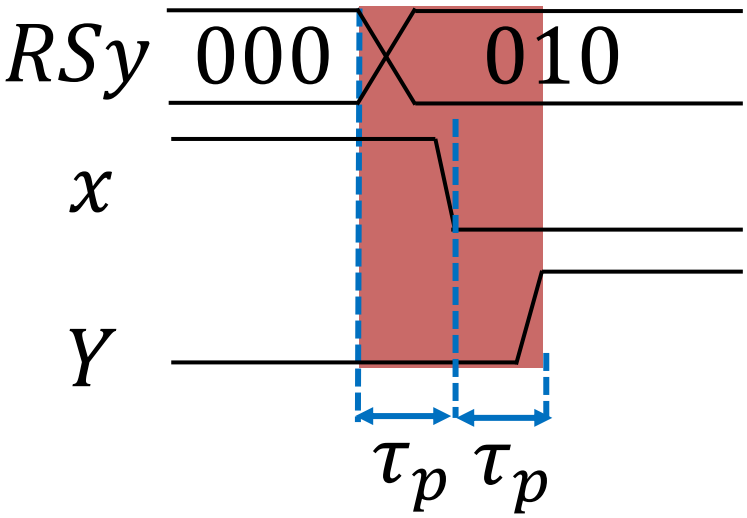
- Per capire i vincoli di corretto impiego di una RSA dobbiamo approfondire come risponde ai cambiamenti degli ingressi una rete combinatoria.
- Il mantenimento della vecchia uscita per un tempo pari al ritardo di propagazione T_p è una semplificazione di un fenomeno più complesso.
- Per studiarlo, parleremo di **comportamento in transitorio** per descrivere quella fase, successiva ad ogni cambiamento dei segnali d'ingresso di una rete combinatoria, caratterizzata dal fatto che l'uscita non ha ancora presentato il valore previsto per la nuova configurazione degli ingressi
- Al termine del transitorio vi è il **comportamento a regime**



Ritardo puro

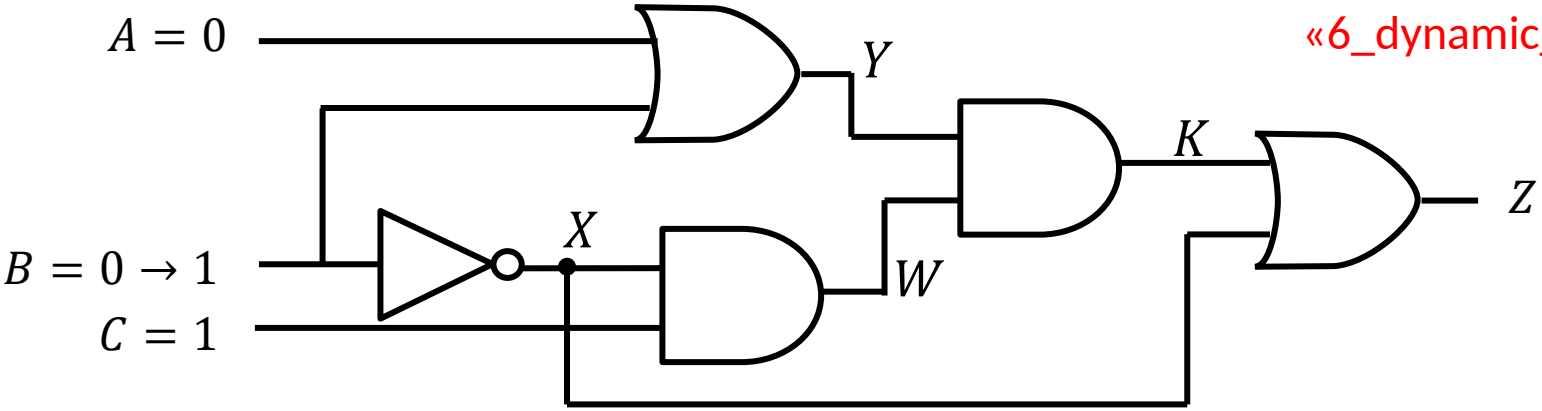
- Il caso più semplice di comportamento in transitorio è quello del ritardo che abbiamo già visto
- Al variare di un ingresso che provoca la variazione dell'uscita di una rete combinatoria, **la rete mantiene il precedente valore dell'uscita per il tempo di propagazione**, prima di sostituirla col nuovo valore.
- Questo transitorio **è ineliminabile ma non dannoso.**

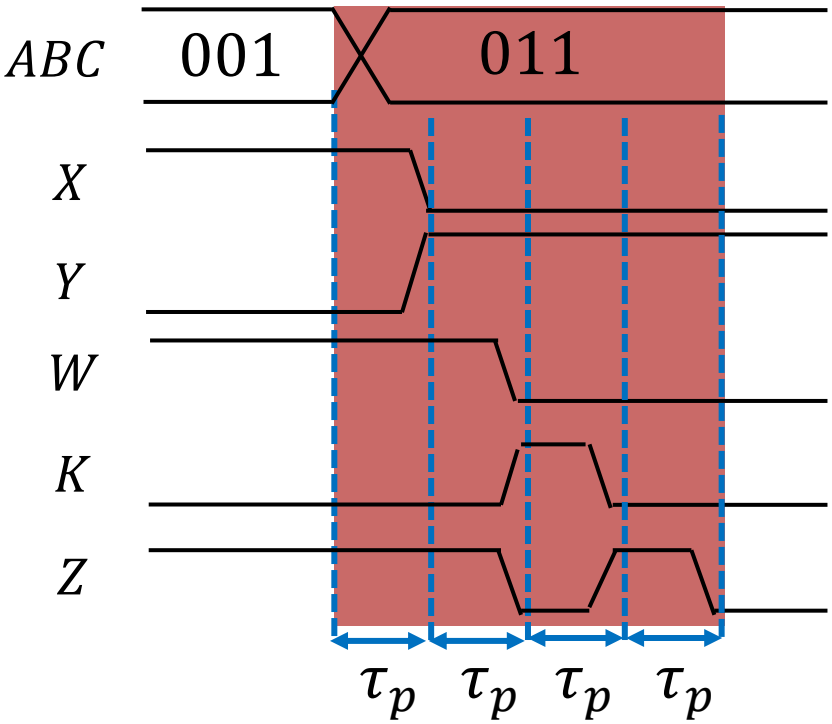




Alea Dinamica

Simulare con Digital
«6_dynamic_hazard.dig»



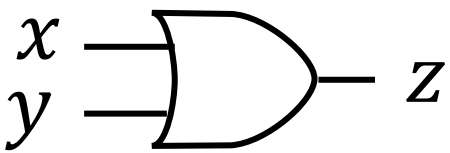


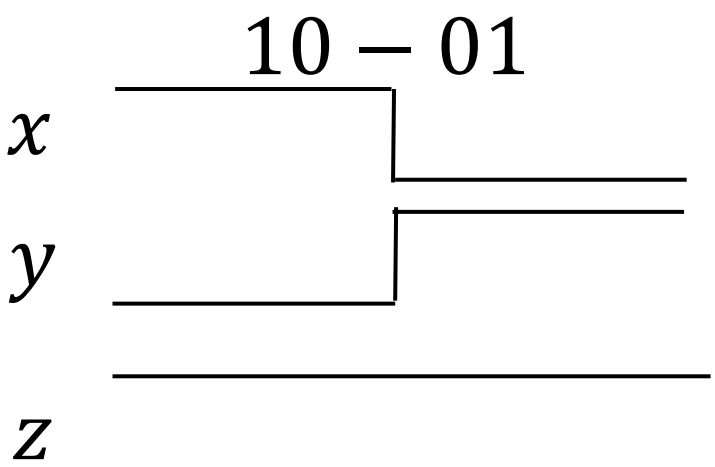
Alea dinamica: l'uscita, in regime di transitorio, **varia più volte** prima di assestarsi sul nuovo valore.
Il malfunzionamento è provocato dai diversi ritardi dei percorsi che agiscono su Z.

Una rete combinatoria descritta da espressioni SP o PS non presenta mai alee dinamiche

Alea Statica

- **Alea statica**: l'uscita, che dovrebbe rimanere costante, subisce in regime di transitorio **una temporanea variazione**
- Esempio: i due ingressi di un gate OR **cambiano valore «contemporaneamente»**
- Nel caso ideale (impossibile), l'uscita rimane costantemente uguale a «1»
- N.B: **non è il motivo principale** per cui può verificarsi alea statica, vedi slide sul MUX.





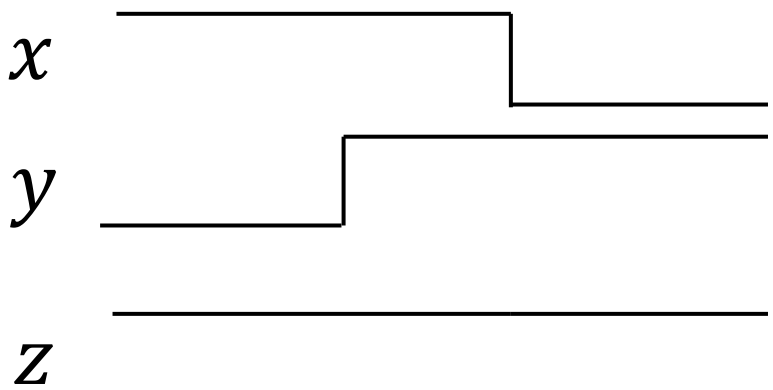
Caso ideale
(impossibile)

Alea statica

- In realtà **è impossibile che due segnali siano perfettamente sincronizzati** (così come era impossibile avere allineamento meccanico perfetto nell'esempio della posizione del carrello visto durante l'introduzione).
- Uno dei due ingressi cambierà prima dell'altro. Non sappiamo quale, è un fenomeno **aleatorio**

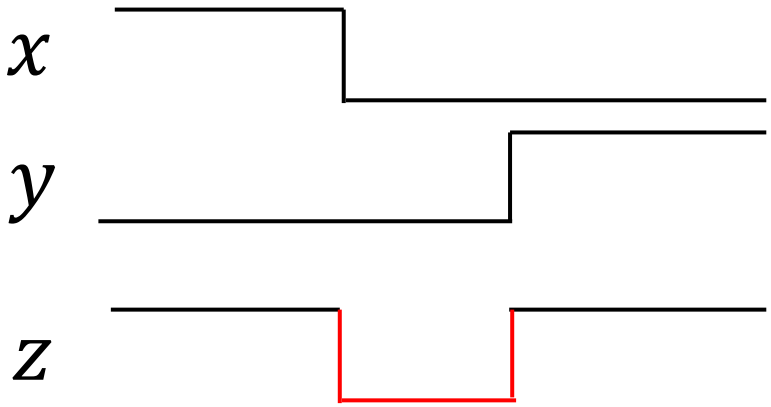
Per evitare alea statica sull'uscita di una rete combinatoria, è necessario (ma non sufficiente, vedi prossima slide) **variare un solo ingresso alla volta**

$$10 - 11 - 01$$



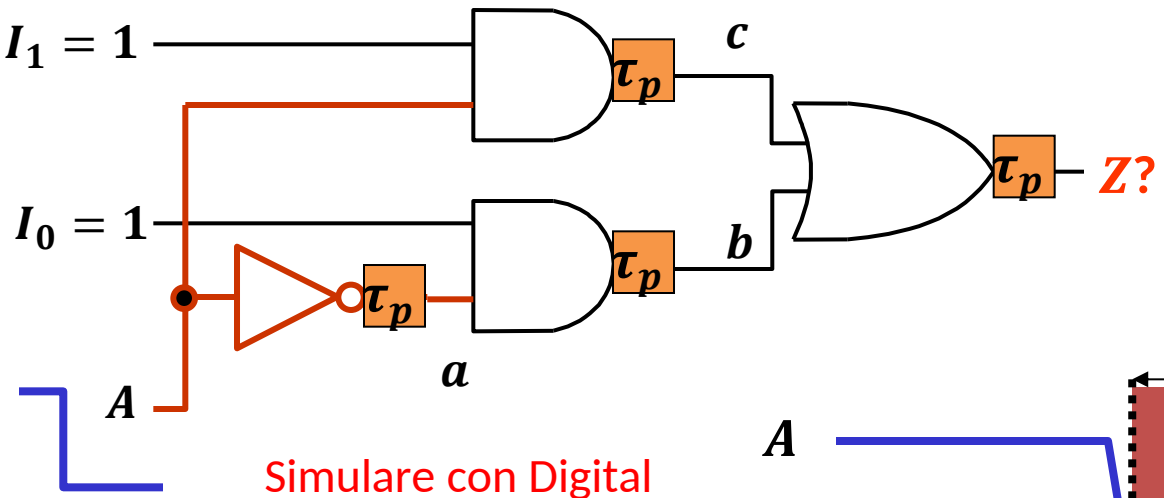
Caso reale 1
OK

$$10 - 00 - 01$$



Caso reale 2
glitch sull'uscita

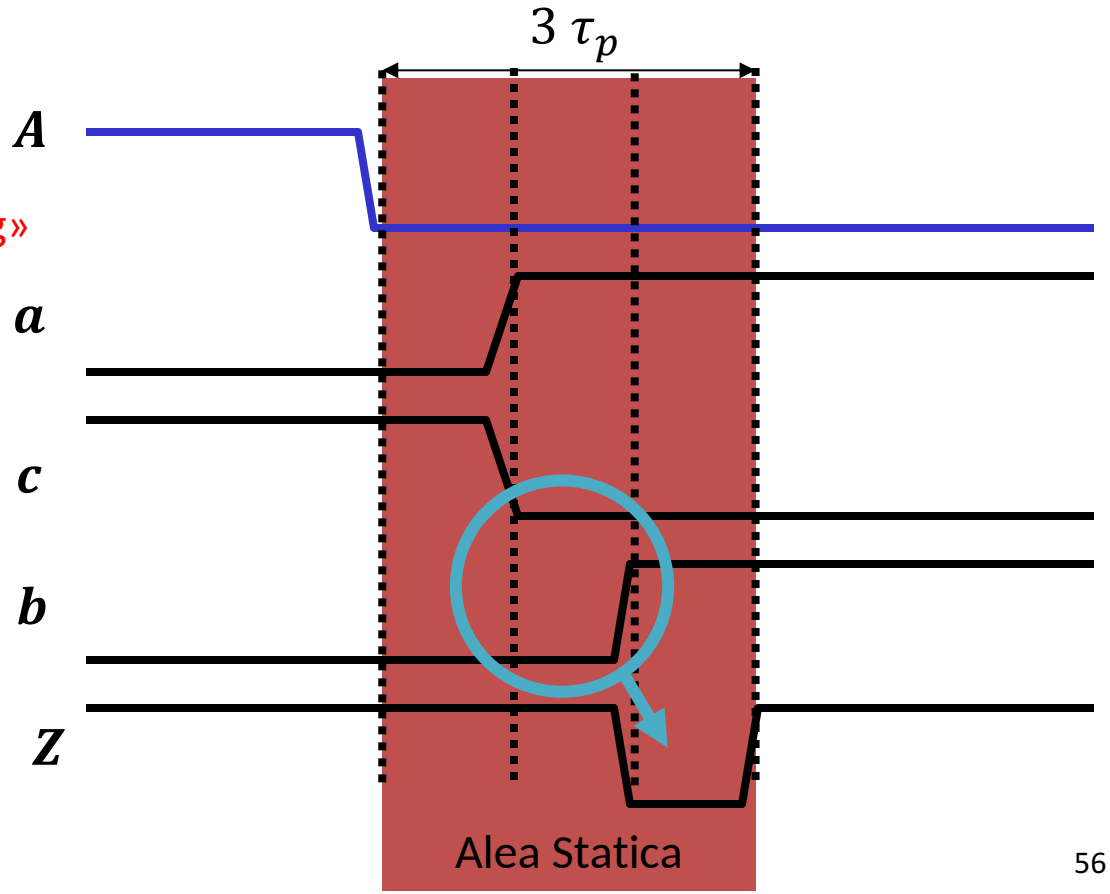
Alea statica nel MULTIPLEXER



Un MUX con vie d'ingresso entrambe ad «1» dovrebbe avere sempre uscita «1» indipendentemente dal valore del bit di indirizzo A

Simulare con Digital «6_2way_mux_with_hazard.dig»

Invece, come si vede nell'esempio quando A passa da 1 a 0, anche sotto l'ipotesi che cambi un solo ingresso alla volta, il **disallineamento dei loro complementi** causato dal ritardo dei NOT può provocare alea statica.

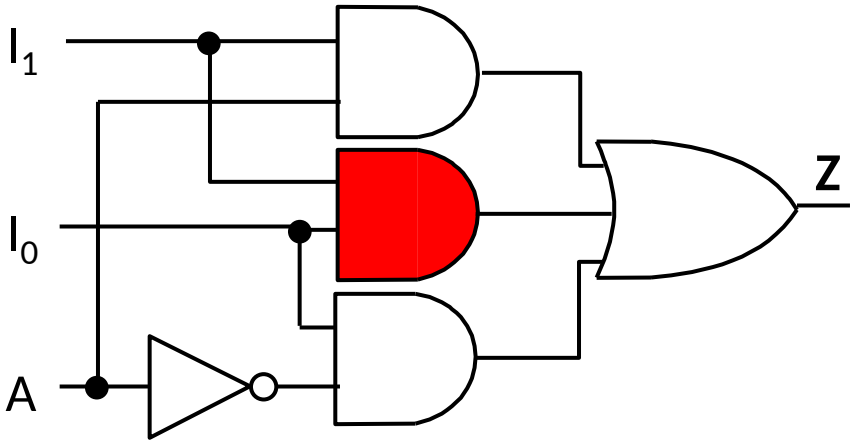


La copertura ridondante

Per evitare a priori l'insorgere di alee statiche, quando si effettua la sintesi con le mappe di Karnaugh si deve scegliere **un'espressione ridondante** (quindi non minima) che **racchiuda in uno stesso raggruppamento rettangolare ogni coppia di «1» o «0» adiacenti**.

Esempio con il MUX a 2 vie:

A \ $I_1 I_0$	00	01	11	10
0	0	1	1	0
1	0	0	1	1



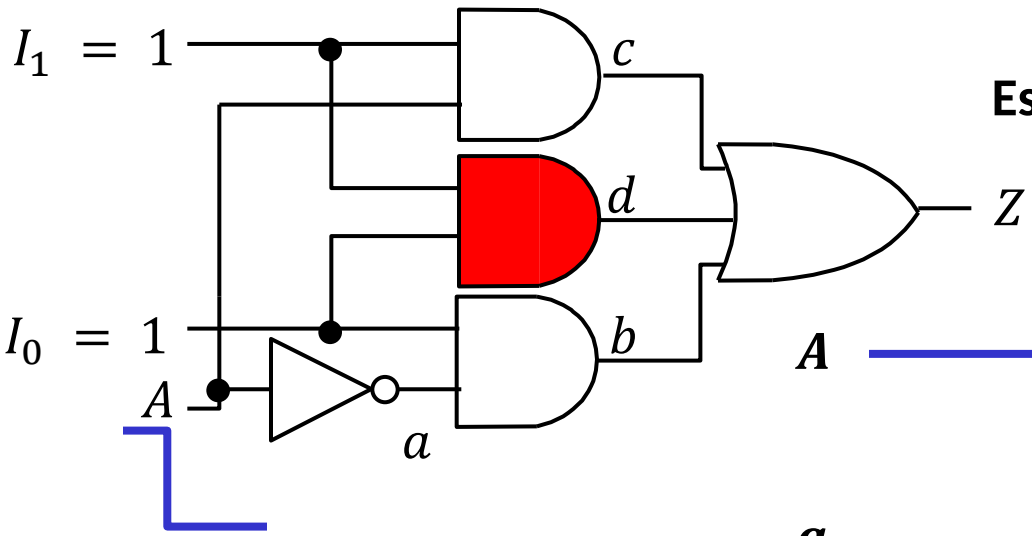
SP minima:

$$Z = A'I_0 + AI_1$$

SP priva di alee:

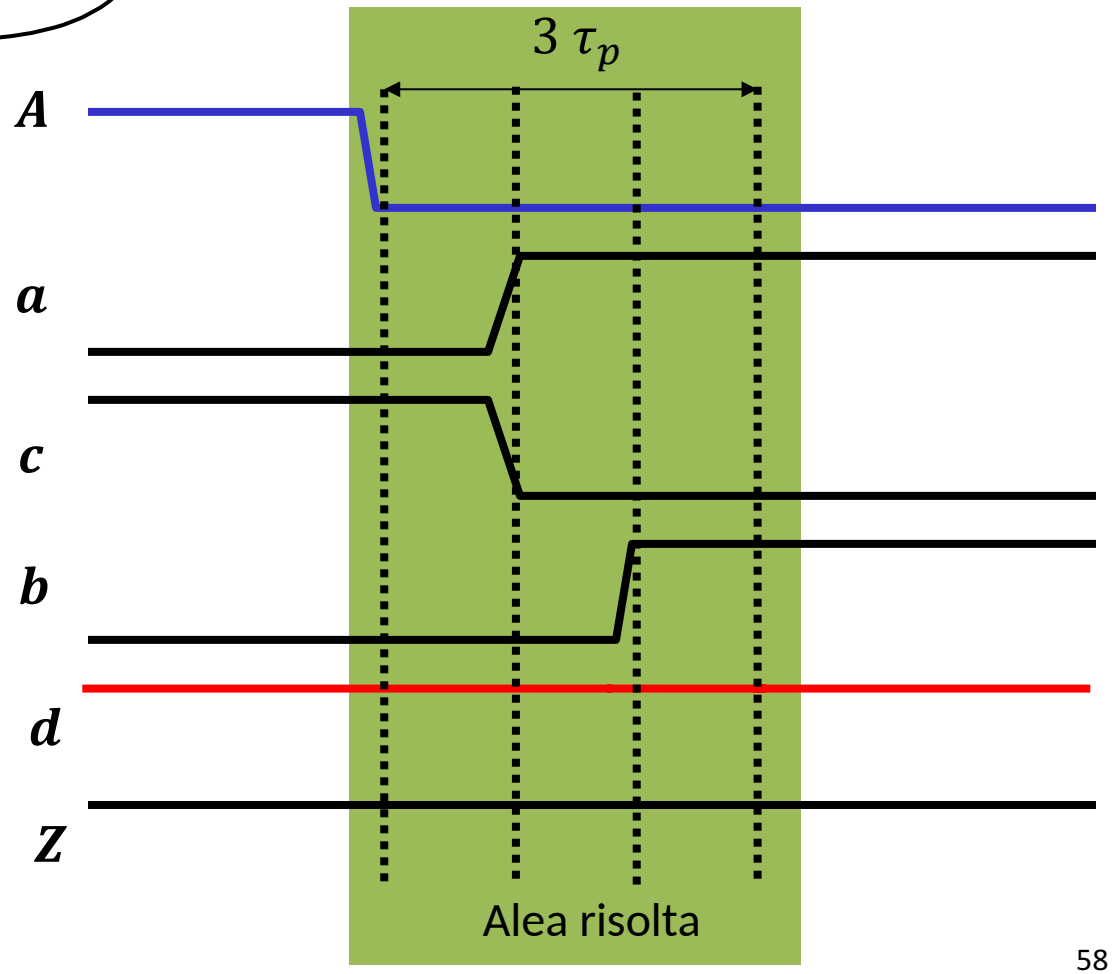
$$Z = A'I_0 + I_1 I_0 + AI_1$$

SELETTORE: rete non minima



Espressione non minima (ridondante):

$$Z = A'I_0 + I_1 I_0 + A I_1$$



L'AND in più mantiene il corretto valore di Z per tutto il transitorio.

Simulare con Digital «6_2way_mux_without_hazard.dig»

Eliminazione a priori delle alee statiche

- Quello che abbiamo appena visto con il MUX è l'applicazione di una regola di carattere generale.
- Una rete combinatoria in cui i segnali d'ingresso cambiano di valore uno solo alla volta **non presenta alea statica** se è descritta da un'espressione SP o PS formata da **tutti i termini primi**

- Per realizzare la sintesi di un'espressione con tutti i termini primi, posso applicare la procedura della sintesi minima mediante mappe di Karnaugh assicurandomi che valga la seguente condizione:

Per eliminare a priori le alee statiche in una rete combinatoria in cui i segnali d'ingresso cambiano di valore uno solo alla volta è necessario e sufficiente scegliere una copertura in cui **ogni coppia di 1 (o di 0) contenuta in celle adiacenti sia racchiusa in almeno un RR.**

Altri esempi

$c \backslash ab$	00	01	11	10
0	0	1	0	0
1	0	1	1	1

PS priva di alee:

$$Z = (a + b)(a' + c)(b + c)$$

$cd \backslash ab$	00	01	11	10
00	0	0	1	1
01	1	1	1	1
11	1	1	1	0
10	0	0	1	1

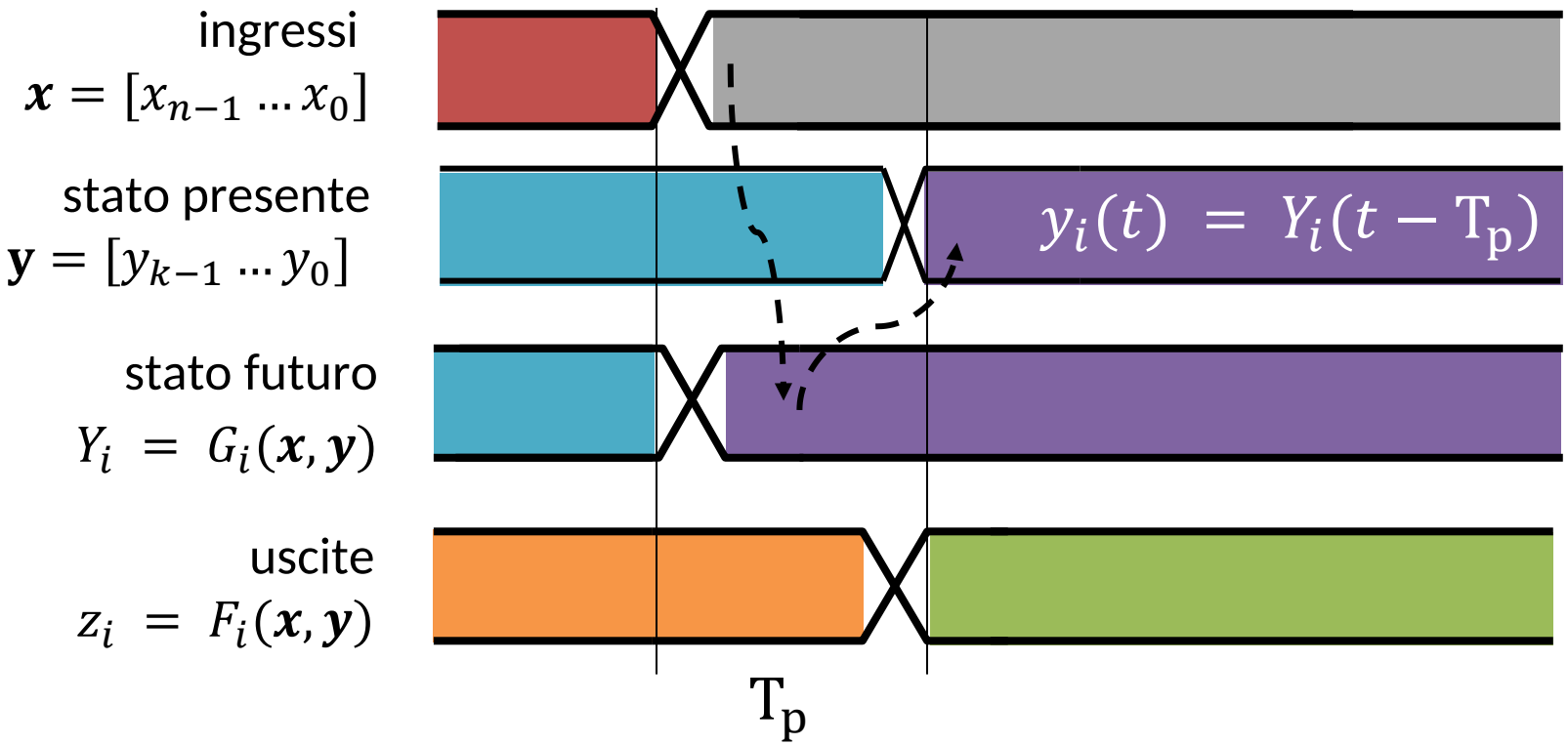
SP priva di alee:

$$Z = a'd + ab + ad' + c'd + bd + ac'$$

VINCOLI PER IL CORRETTO FUNZIONAMENTO DI UNA RSA

Funzionamento di una RSA

- Le reti sequenziali asincrone funzionano ad **inseguimento degli ingressi** come quelle combinatorie: l'uscita e lo stato interno si modificano **soltanto quando cambiano i segnali di ingresso**
- A fronte di un cambiamento degli ingressi, la rete, dopo un brevissimo lasso di tempo T_p in cui lo stato futuro è diverso da quello presente, **raggiunge <u>un nuovo stato stabile</u>**.
- Le RSA tengono conto **esplicitamente solo della sequenza** dei segnali di ingresso. La **durata degli ingressi** non ha nessuna influenza sul comportamento della rete



RSA: regole di corretto impiego

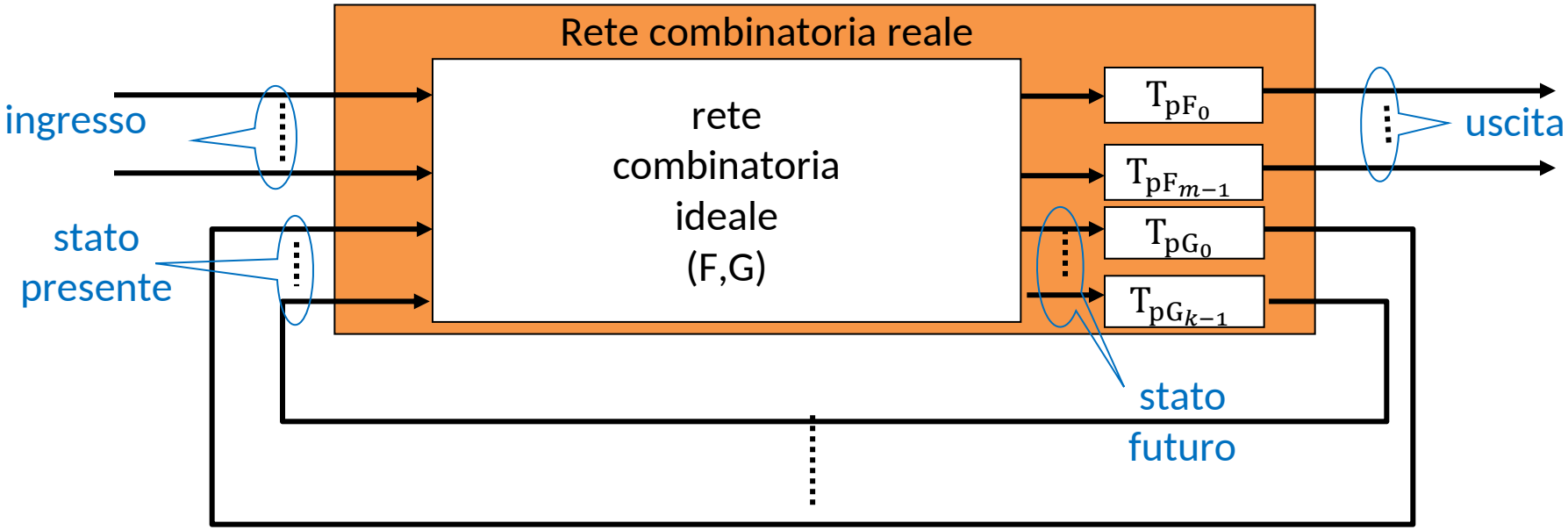
Obiettivo: ottenere un funzionamento corretto (passaggio da una condizione di stabilità all'altra) per qualsiasi valore dei ritardi presente sugli anelli di retroazione.

- Il passaggio da una condizione di stabilità alla successiva è detto **funzionamento in modo fondamentale**
- Affinché una RSA operi correttamente è necessario garantire che questo accada per ogni possibile ingresso e stato presente.
- Perché questo sia garantito, è necessario che siano verificate una serie di condizioni in merito a:
    1. Durata degli ingressi
    2. Cambiamento dei bit di ingresso
    3. Alee statiche
    4. Codifica degli stati

1. Durata degli ingressi

I segnali di ingresso possono essere modificati solo dopo che la rete ha raggiunto la nuova stabilità

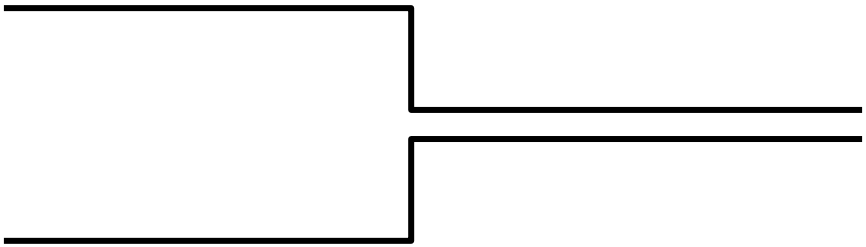
- Come per le reti combinatorie, quindi, **esiste un limite superiore alla velocità di funzionamento** di una RSA.
- Per calcolare il tempo minimo di durata di una configurazione binaria in ingresso, **devo considerare il ritardo massimo tra $\mathbf{T}_{pG_0}, \dots, T_{pG_{k-1}}$** che indichiamo con $\mathbf{T}_{pG}$
- In generale, una configurazione d'ingresso deve permanere per almeno $2\mathrm{T}_{pG}$: la prima volta per calcolare lo stato futuro a partire dal vecchio stato presente, e la seconda per renderlo stabile.
- Se sono presenti transizioni multiple e la più lunga prevede t transizioni (vedi dopo), il tempo sarà $(1+t)\mathrm{T}_{pG}$, ovvero devo moltiplicarlo per il numero di transizioni necessarie a rendere stabile lo stato futuro.



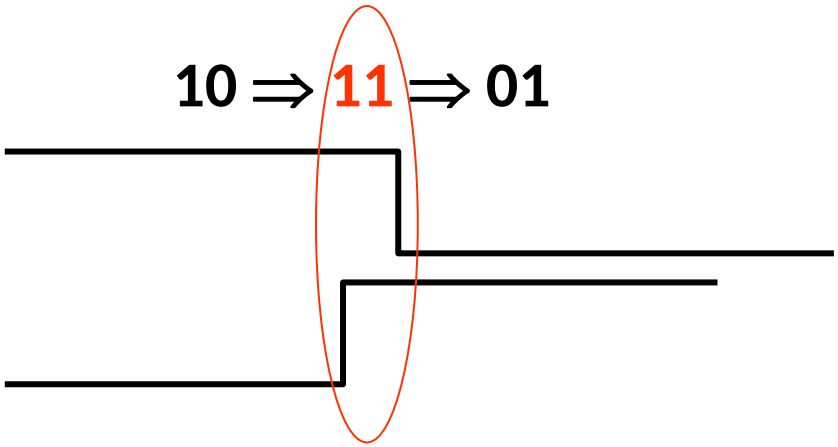
2. Cambiamento dei bit di ingresso

- Abbiamo visto che è impossibile variare contemporaneamente gli ingressi di una rete combinatoria. Se configurazioni in ingresso consecutive differiscono per più di un bit, **la rete vedrà in realtà configurazioni di ingresso «spurie»** oltre a quelle che volevo.
- Ma **le RSA sono reti combinatorie**, con alcuni ingressi in retroazione. Dunque, configurazioni consecutive degli ingressi devono essere **adiacenti.**
- **Questa è anche una condizione necessaria per prevenire alee statiche sugli stati** (vedi prossimo vincolo)

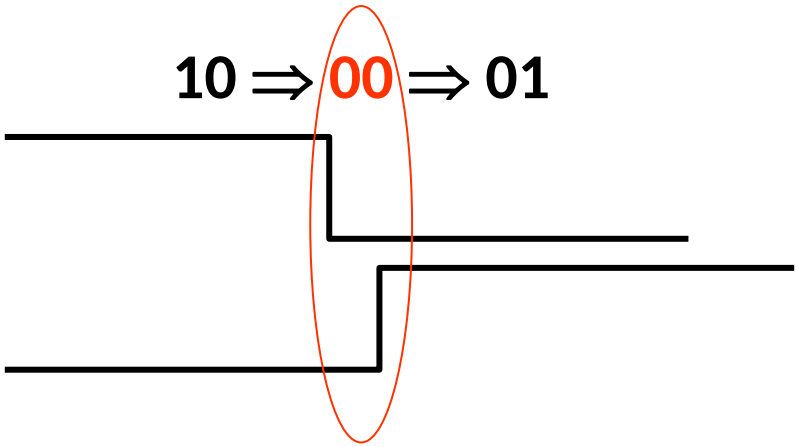
***Esempio di situazione d'ingresso pericolosa*:**
10 ⇒ 01



10 ⇒ 11 ⇒ 01



10 ⇒ 00 ⇒ 01



Il progetto e l'uso di una RSA devono prevedere che cambi solo un bit di ingresso alla volta.

Come si sposta la rete su una tabella

I vincoli sulla durata degli ingressi e sul cambiamento di un solo bit di ingresso determinano un preciso percorso con cui la rete può spostarsi su una tabella di flusso.

In particolare, a partire da una cella stabile, quando gli ingressi cambiano:

1. La rete si sposta nella colonna **adiacente** (vincolo 1) alla cella stabile che corrisponde ai nuovi ingressi rimanendo sulla stessa riga (=stato presente)
2. Poi si sposta nella riga (=stato futuro) indicata dalla cella, rimanendo nella stessa colonna (vincolo 2, ingressi stabili fino al raggiungimento di una nuova stabilità)
3. Ripete il punto 2 fino al raggiungimento di una cella stabile

Per esempio, dalla cella (B,11), quando gli ingressi assumono valore 10, la rete si sposta in stato D, quindi in stato C, e poi in stato A

$x_1\ x_0$

stato presente \ stato futuro, z	00	01	11	10
A	A,0	A,0	B,0	A,0
B	A,0	C,0	B,0	D,0
C	A,0	C,0	D,-	A,0
D	A,-	A,-	D,1	C,-

Esempio di possibili malfunzionamenti

Ipotizziamo di trovarci in stato A e di ricevere in ingresso la sequenza 00-11-10

X_1X_2

	00	01	11	10	Z
A	A	C	B	D	0
B	A	D	B	C	0
C	A	C	C	C	1
D	A	D	D	D	0

X_1	0	1	1
X_2	0	1	0
s.p.	A	B	C
Z	0	0	1

Caso ideale

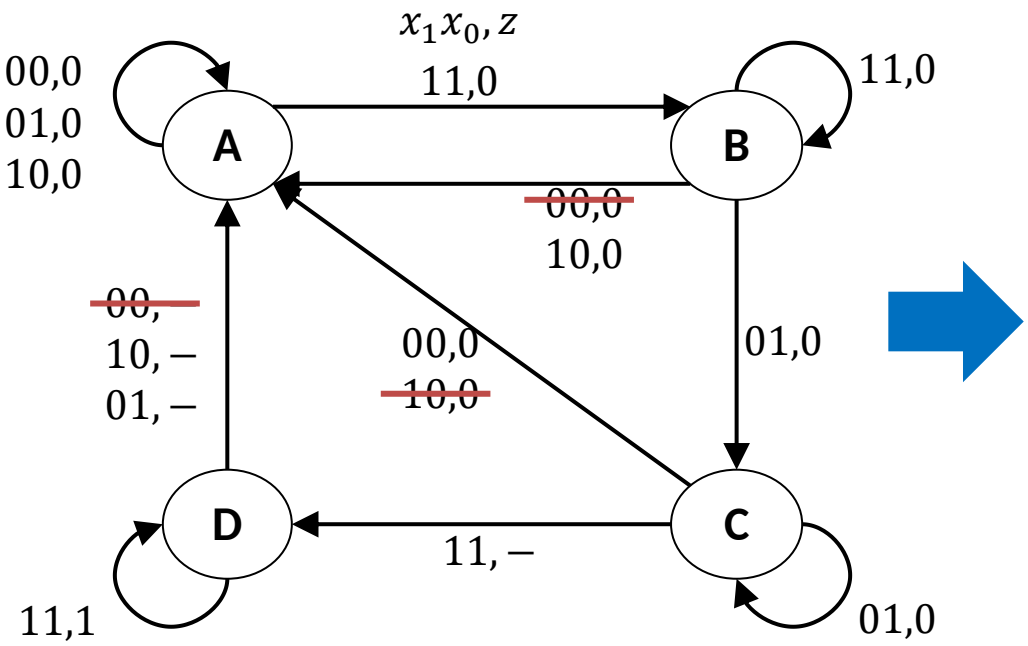
X_1	0	0	1	1
X_2	0	1	1	0
s.p.	A	C	C	C
Z	0	1	1	1

X_1	0	1	1	1
X_2	0	0	1	0
s.p.	A	D	D	D
Z	0	0	0	0

Casi reali: la rete raggiunge comunque lo stato C ma l'uscita assume troppo presto il valore "1", oppure la rete finisce in stato D invece che C, alterando completamente il suo comportamento presente e futuro.

Grafo e TdF per la cassaforte

- La regola di corretto impiego che prevede che configurazioni d'ingresso consecutive differiscano al più di un bit implica che non tutti gli ingressi sono possibili in ogni stato.
- In particolare, **sono possibili solo configurazioni adiacenti a quelle per cui lo stato è stabile**.
- Per esempio, il grafo e la tabella di flusso della cassaforte si semplificano come segue



$x_1\ x_0$

stato presente \ stato futuro, z	00	01	11	10
A	A,0	A,0	B,0	A,0
B	-,-	C,0	B,0	A,0
C	A,0	C,0	D,-	-,-
D	-,-	A,-	D,1	A,-

3. Alee Statiche

- Usando espressioni minime per realizzare le reti di calcolo dello stato futuro, possiamo avere reti affette da alea statica, come abbiamo visto.
- Mentre l'alea può non essere un problema per reti non all'interno di anelli combinatori in retroazione, è potenzialmente molto dannosa per le RSA, in quanto **può introdurre stati «spuri» all'interno della sequenza**, impedendo il corretto funzionamento della rete.
- E' quindi necessario eliminare a priori le alee statiche quando si sintetizzano le reti che calcolano lo stato futuro tramite la tecnica della copertura ridondante che abbiamo visto, ovvero

Nella sintesi delle reti che calcolano lo stato futuro è necessario che ogni coppia di celle adiacenti da coprire sia racchiusa in un RR (anche gli eventuali RR ridondanti devono avere dimensione massima).

Esempio di possibili malfunzionamenti (1)

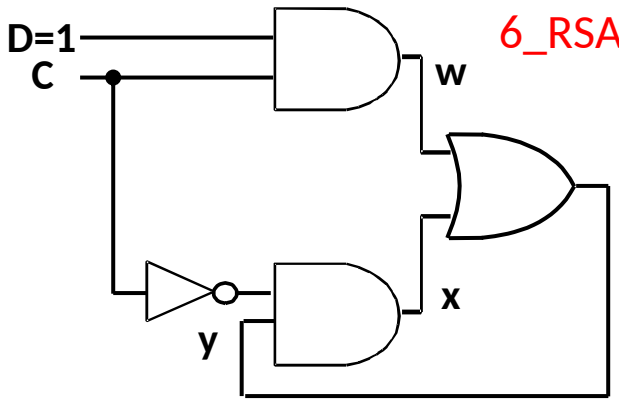
- Data la tabella delle transizioni riportata, la sintesi della variabile di stato futuro Y si può effettuare con l'espressione di costo minimo…

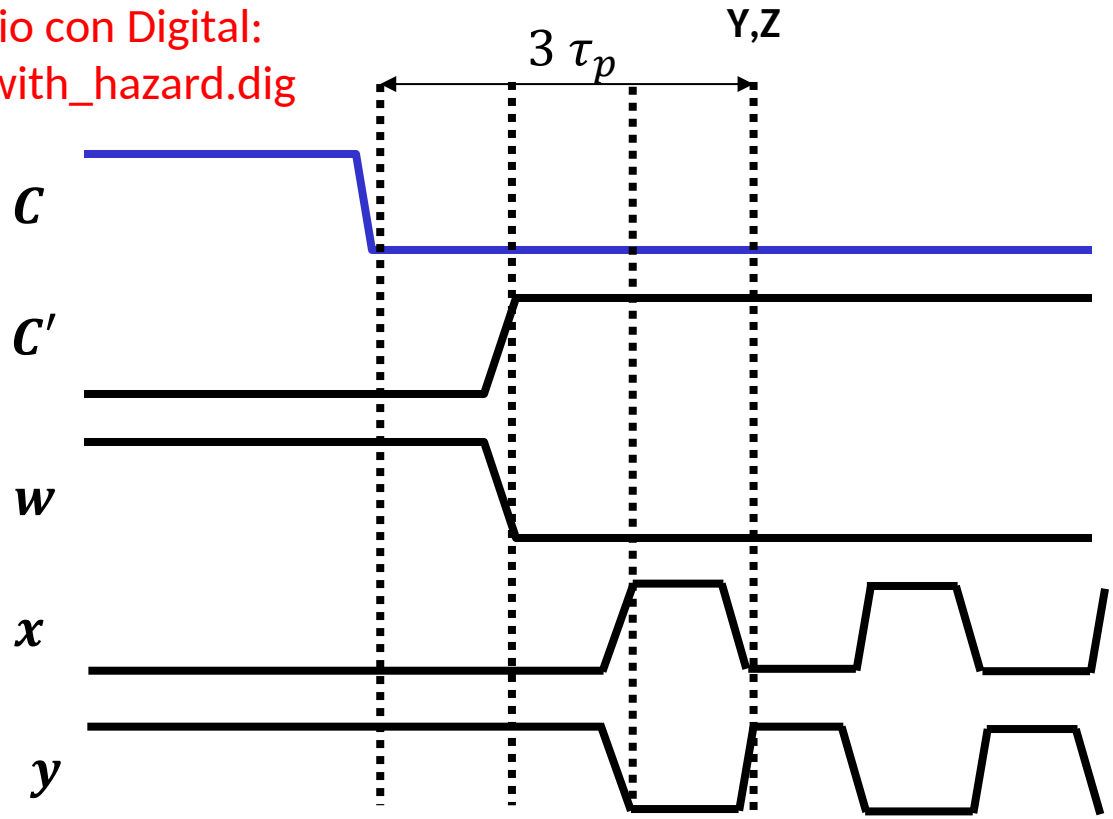
CD

y	00	01	11	10
A=0	0,0	0,0	1,-	0,0
B=1	1,1	1,1	1,1	0,-

Y,Z

Esempio con Digital:
6_RSA_with_hazard.dig





CD

y	00	01	11	10
A=0	0	0	1	0
B=1	1	1	1	0

Y

$Y = C\,D + C'\,y$

N.B. non assumendo ritardi identici e deterministici tra i gate, l'uscita si stabilizzerebbe a uno dei due valori

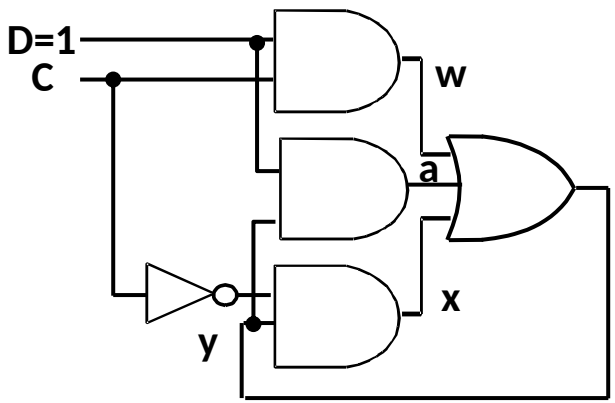
Esempio di possibili malfunzionamenti (2)

...o con eliminazione a priori dell'alea, che invece garantisce di rimanere in stato B, come da specifica.

CD

y	00	01	11	10
A=0	0,0	0,0	1,-	0,0
B=1	1,1	1,1	1,1	0,-

Y,Z

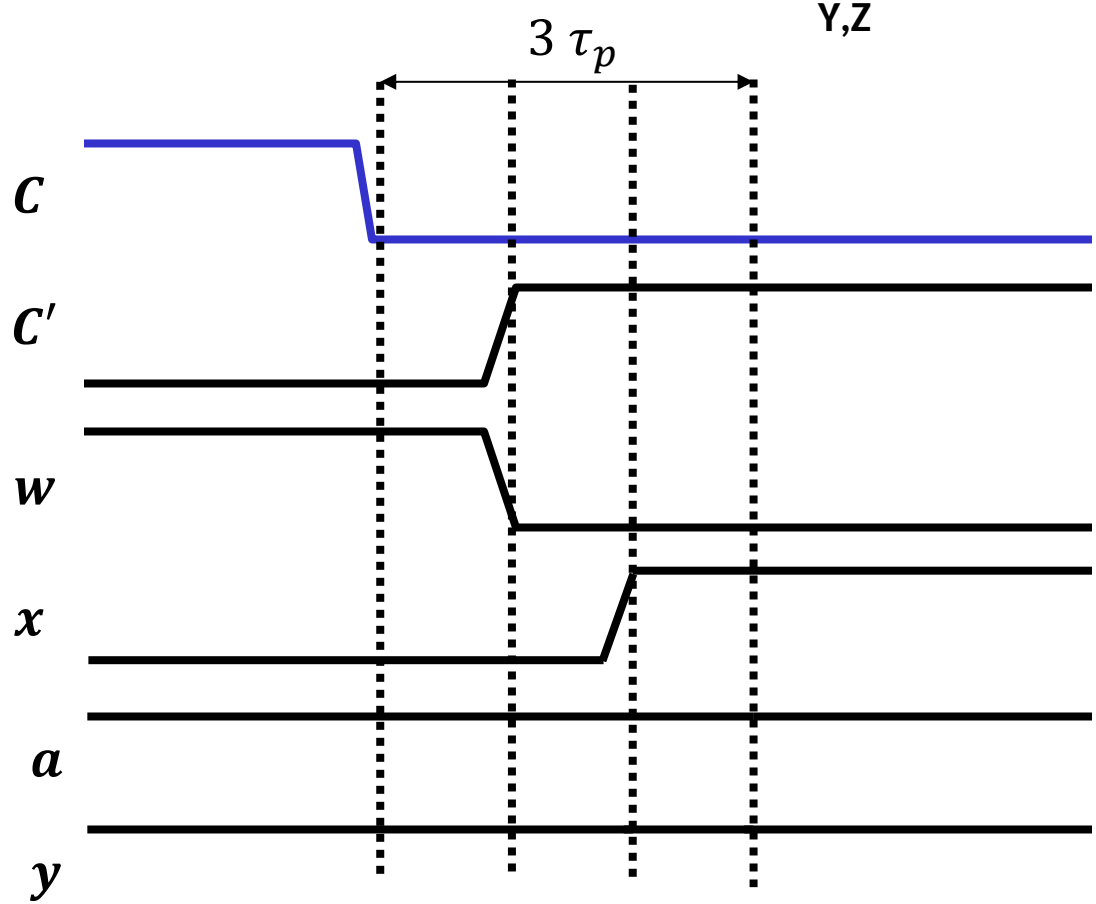


CD

y	00	01	11	10
A=0	0	0	1	0
B=1	1	1	1	0

Y

$Y = C\,D + C'\,y + D\,y$



4. Codifica degli stati

- Essendo i bit di stato presente a tutti gli effetti ingressi delle reti combinatorie che formano una RSA, una situazione analoga alla necessità di avere configurazioni consecutive adiacenti si verifica anche per quanto riguarda la **codifica degli stati interni.**
- In particolare, essendo il valore di questi ingressi lo stato futuro calcolato a partire dall'attuale stato presente, deve valere la condizione:

Le configurazioni binarie associate ad ogni coppia (stato presente, stato futuro) devono essere adiacenti

- Se applicato a tutte le coppie stato presente-stato futuro, questo principio richiederebbe un codice molto ridondante, che utilizzerebbe molti più bit del necessario
- Fortunatamente si tratta di un vincolo spesso eccessivo: si può garantire il corretto funzionamento della rete anche in presenza di variazione contemporanea di più di un bit di stato interno per alcune configurazioni d'ingresso («corse non critiche», v. prossime slide)

Codifica degli stati

- Per passare dalla tabella di flusso alla tabella delle transizioni, è necessario scegliere una codifica degli stati.
- Per il vincolo 4, non tutte le codifiche sono valide.
- Esempio di codifica non valida

	00	01	11	10
A	A	B	A	A
B	A	B	D	B
C	A	C	C	C
D	A	C	D	D

	00	01	11	10
A=00	00	01	00	00
B=01	00	01	10	01
C=11	00	11	11	11
D=10	00	11	10	10

- Notare che nelle celle evidenziate non è rispettato il vincolo di adiacenza tra stato presente e futuro
- Come possiamo determinare se questa codifica è valida o no?

Corse critiche e corse non critiche

- Segnali in retroazione per cui è stata prevista una modifica **contemporanea** di valore si trovano in una situazione di **corsa.** Nel circuito i cambiamenti si verificheranno a istanti diversi e con un ordine dettato dai ritardi interni.
- Una **corsa** è **critica** se si possono **raggiungere stabilità diverse.**
- Una **corsa non critica** genera una **transizione multipla**, ovvero la rete passa da uno o più stati intermedi prima di raggiungere quello stabile.
- Le transizioni multiple non sono un problema a patto che
  - L'uscita non presenti andamenti diversi dal comportamento voluto durante la transizione
  - Gli ingressi rimangano stabili fino al raggiungimento dello stato stabile

Stato presente $\mathbf{C = 11}$
ingressi $\mathbf{01 \rightarrow 00}$

Transizione ideale:
$C = 11 \rightarrow A = 00$
Transizioni reali:
$11 \Rightarrow 01 \Rightarrow \boxed{00}$
$11 \Rightarrow 10 \Rightarrow \boxed{00}$
corsa non critica

Transizione multipla

	00	01	11	10
A=00	00	01	00	00
B=01	00	01	10	01
C=11	00	11	11	11
D=10	00	11	10	10

Stato presente $\mathbf{B = 01}$
ingressi $\mathbf{01 \rightarrow 11}$

Transizione ideale:
$B = 01 \rightarrow D = 10$
Transizioni reali:
$01 \Rightarrow \boxed{00}$
$01 \Rightarrow \boxed{11}$
corsa critica

Codifica errata

Prevenzione a priori delle corse critiche

- Come si nota nell'esempio precedente, la presenza di corse critiche si ha nei casi in cui **una colonna** della tabella di flusso **presenti più di uno stato stabile**
- **In caso di colonne con una sola stabilità**, infatti, se tutti gli stati non stabili riconducono allo stato stabile, **si avranno sempre corse non critiche**

- Si possono eliminare a priori le situazioni di corse critiche seguendo le seguenti regole:

1. Nelle colonne con **una sola stabilità** si inserisce il simbolo dello stato stabile al posto di eventuali condizioni d'indifferenza.
2. Per le sole colonne **con più stabilità** si traccia il **grafo delle adiacenze**: ad ogni stato è associato un nodo e ad ogni coppia stato presente - stato futuro un ramo orientato che connette i due nodi corrispondenti.
3. Si sovrappone il grafo ad una mappa con codici di Gray su righe e colonne (come nelle mappe di Karnaugh) per il minimo numero di variabili di stato necessarie e si verifica se è possibile **assegnare configurazioni adiacenti ad ogni coppia di stati coinvolta in una transizione**.
4. Se è impossibile soddisfare tutti i vincoli di adiacenza, si cerca di ridurli ricorrendo a **transizioni multiple**.
5. Se non ci si riesce, si **incrementa il numero delle variabili di stato** e si ritorna al punto 3.

Codifica per la lampada da tavolo

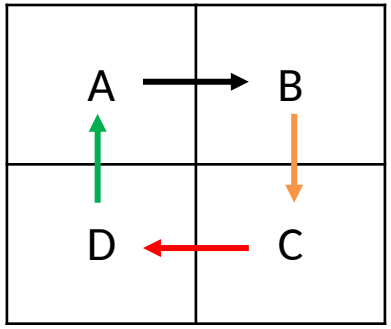
1. Non ci sono colonne con una sola stabilità, quindi devo considerare tutte le colonne al passo 2

x

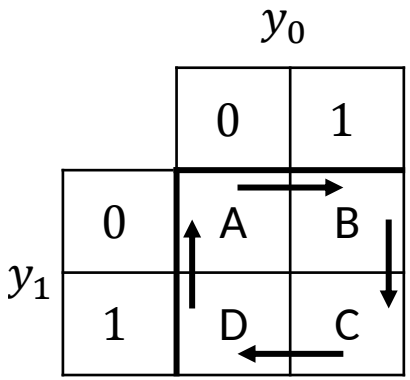
stato presente	0	1
A	A,0	B,0
B	C,-	B,0
C	C,1	D,1
D	A,-	D,1

stato futuro, z

2. Grafo delle adiacenze: per ogni stato presente traccio una freccia verso i suoi stati futuri



3. Sovrappongo mappa con 2 variabili di stato (2^2 stati -> 2 bit). Ho configurazioni adiacenti per ogni transizione, quindi ho trovato una codifica valida.



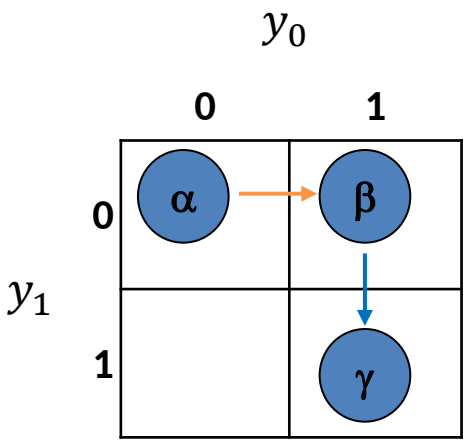
Esempio con corse non critiche (1)

1. La colonna 00 e 10 hanno solo una stabilità, quindi le ignoro nel grafo delle adiacenze (ma devo ricordarmi di sostituire le indifferenze con valori appropriati, vedi dopo)

s.p. \ x_1x_2	00	01	11	10
α	α,0	α,0	β,-	γ,0
β	-,-	γ,-	β,1	γ,-
γ	α,0	γ,0	γ,0	γ,0

stato futuro, Z

2. Considerando solo le transizioni **verso le colonne 01 e 11**, rimane che da α devo raggiungere β e da β devo raggiungere γ.
3. E' possibile soddisfare queste adiacenze con 2 bit di stato



Esempio con corse non critiche (2)

x_1x_2

s.p.	00	01	11	10
α	α,0	α,0	β,-	γ,0
β	-,-	γ,-	β,1	γ,-
γ	α,0	γ,0	γ,0	γ,0

stato futuro, Z

y_0

y_1	0	1
0	α	β
1		γ

x_1x_2

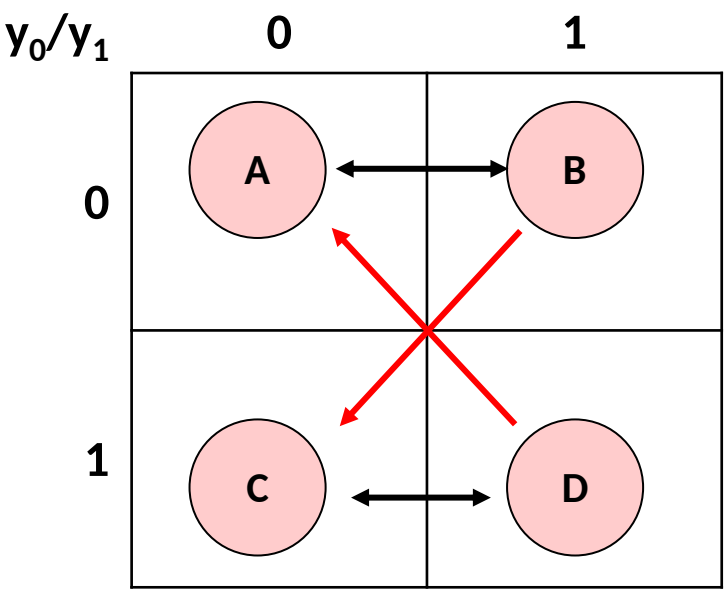
y_1y_0	00	01	11	10
00	00,0	00,0	01,-	11,0
01	00,0	11,-	01,1	11,0
11	00,0	11,0	11,0	11,0
10	00,0	--,-	--,-	11,0

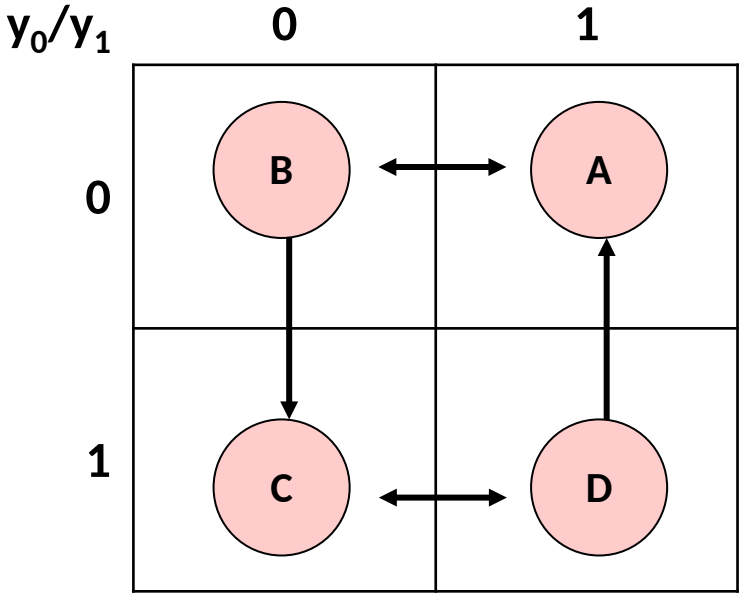
x_1x_2

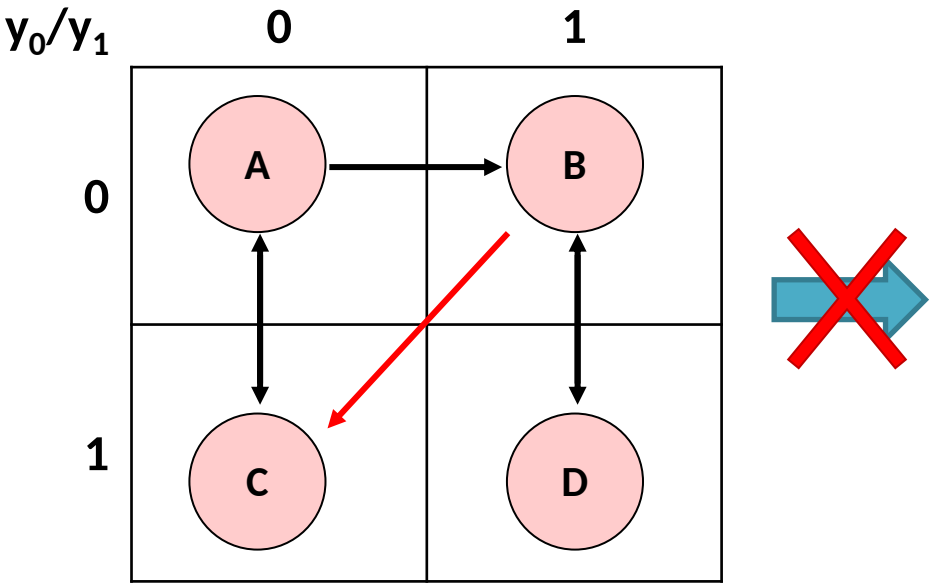
y_1y_0	00	01	11	10
00	00,0	00,0	01,-	11,0
01	--,-	11,-	01,1	11,-
11	00,0	11,0	11,0	11,0
10	--,-	--,-	--,-	--,-

Ottengo la tabella delle transizioni finali applicando la codifica trovata alla tabella di flusso, ma anche ricordandomi di **sostituire le indifferenze nelle colonne con un solo stato stabile** con la codifica di quello stato e il valore dell'uscita assunto in quello stato.

Passo 4: transizioni multiple



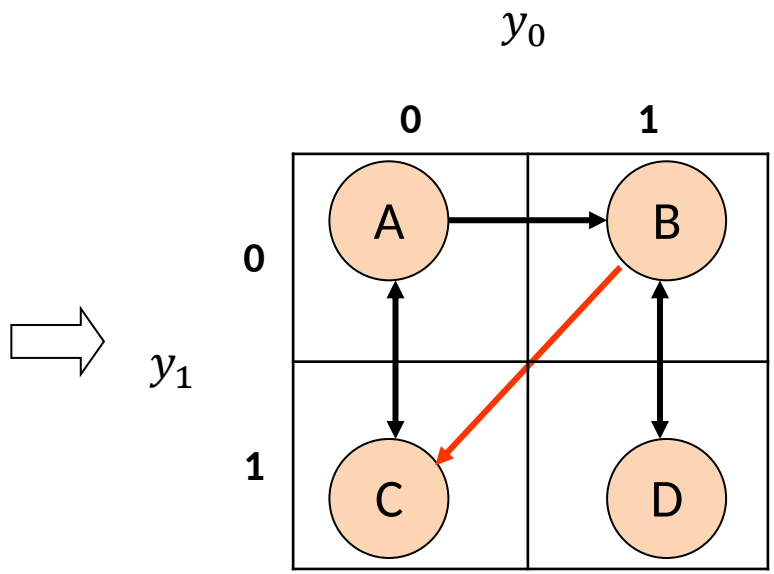




- Tramite il grafo delle adiacenze, si cerca una codifica che permetta solo transizioni tra stati adiacenti
- A volte è sufficiente riordinare il grafo per ottenerla, come nell'esempio riportato sopra.
- Tuttavia, non sempre esiste una codifica con queste caratteristiche: in tale caso, occorre procedere con il passo successivo (utilizzo di **transizioni multiple**)

Passo 4: utilizzo di transizioni multiple (1)

	00	01	11	10
A	A	B	C	A
B	D	B	C	-
C	A	C	C	A
D	D	B	D	D



	00	01	11	10
A=00	00	01	10	00
B=01	11	01	**10**	-
C=10	00	10	10	00
D=11	11	01	11	11

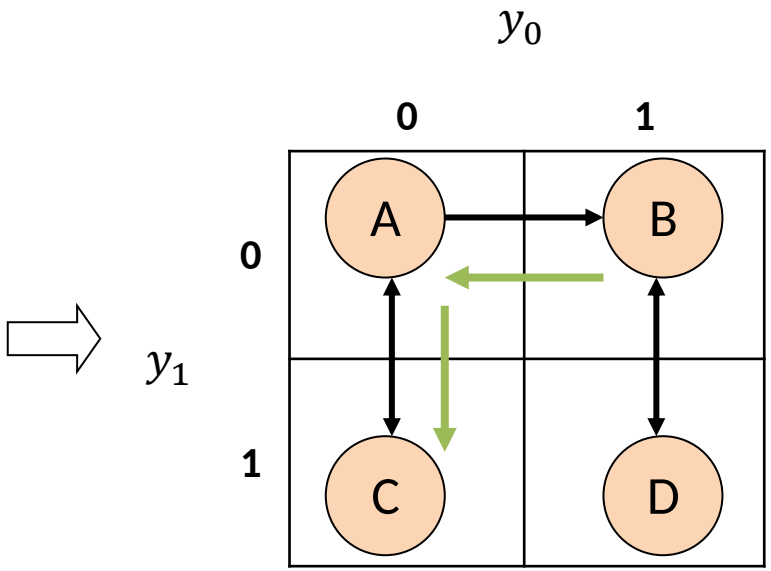
In questo caso, non esiste un ordinamento degli stati che eviti la presenza di corse critiche

Corsa
01->11
01->00->10

Non raggiunge lo stesso stato stabile -> critica

Passo 4: utilizzo di transizioni multiple (1)

	00	01	11	10
A	A	B	C	A
B	D	B	A	-
C	A	C	C	A
D	D	B	D	D



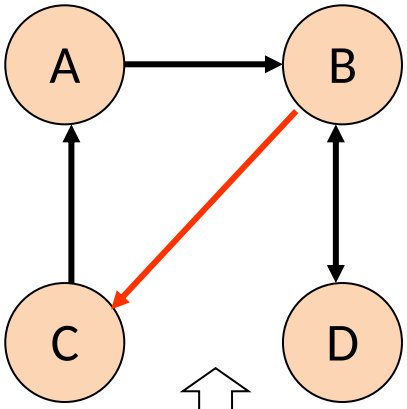
Posso però **realizzare la transizione $B \rightarrow C$ per ingresso 11 passando da A**, che già ha come stato futuro C. A questo punto è possibile codificare gli stati con due bit senza corse critiche.

	00	01	11	10
A=00	00	01	10	00
B=01	11	01	00	-
C=10	00	10	10	00
D=11	11	01	11	11

Transizione multipla

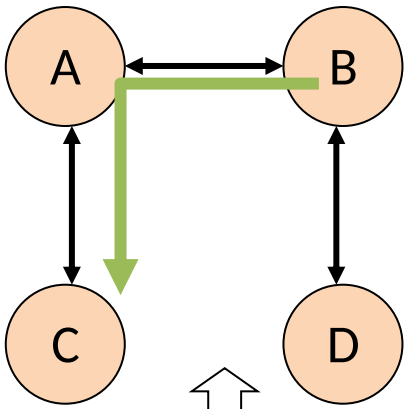
01->00->10

Passo 4: utilizzo di transizioni multiple (2)



	00	01	11
A	A	B	-
B	D	B	C
C	A	C	C
D	D	B	D

N.B.: La colonna corrispondente ad ingressi 10 non è riportata per motivi di leggibilità



	00	01	11
A	A	B	C
B	D	B	A
C	A	C	C
D	D	B	D

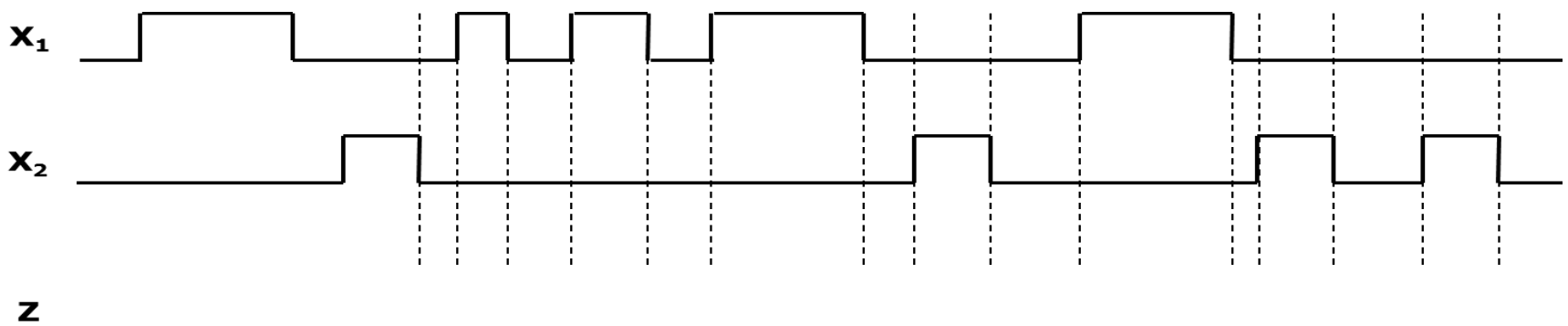
- Il passo 4) può essere applicato **anche in presenza di stati con stato futuro indifferente** (stato A nell'esempio)
- In questo caso si sfrutta l'impossibilità dell'ingresso 11 per lo stato A per inserire la transizione multipla al posto dell'indifferenza

ESEMPI DI SINTESI

Esempio 1

Una rete sequenziale asincrona ha due ingressi $x_1 x_2$ ed una uscita z. I segnali di ingresso **non cambiano mai di valore contemporaneamente**, e **non presentano mai contemporaneamente il valore 1**.

Finché gli ingressi assumono valore 1 in modo alternato, ovvero ad una configurazione in cui x_1 = 1 segue una configurazione in cui x_2=1, l'uscita z deve mantenere il valore 0. z deve assumere valore 1 non appena viene violata tale alternanza. L'uscita z deve in questo caso tornare ad assumere il valore 0 **quando è terminato il valore 1 del segnale che ripristina l'alternanza corretta** tra i due segnali d'ingresso (v. forme d'onda seguenti). Sintetizzare la rete secondo **il modello di Moore** usando reti di costo minimo **in forma SP** per realizzare le funzioni combinatorie necessarie.

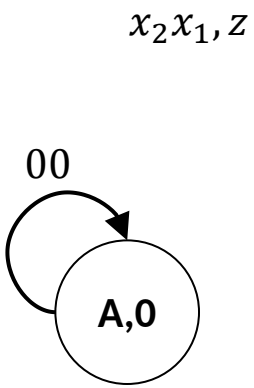


N.B.: Non posso sapere il valore dell'uscita all'inizio della sequenza perché non so se in precedenza l'alternanza era stata rispetta (rete sequenziale)

...

N.B.: L'uscita di una RSA può cambiare solo quando cambiano gli ingressi, quindi ha senso chiedersi cosa succede solo negli istanti tratteggiati

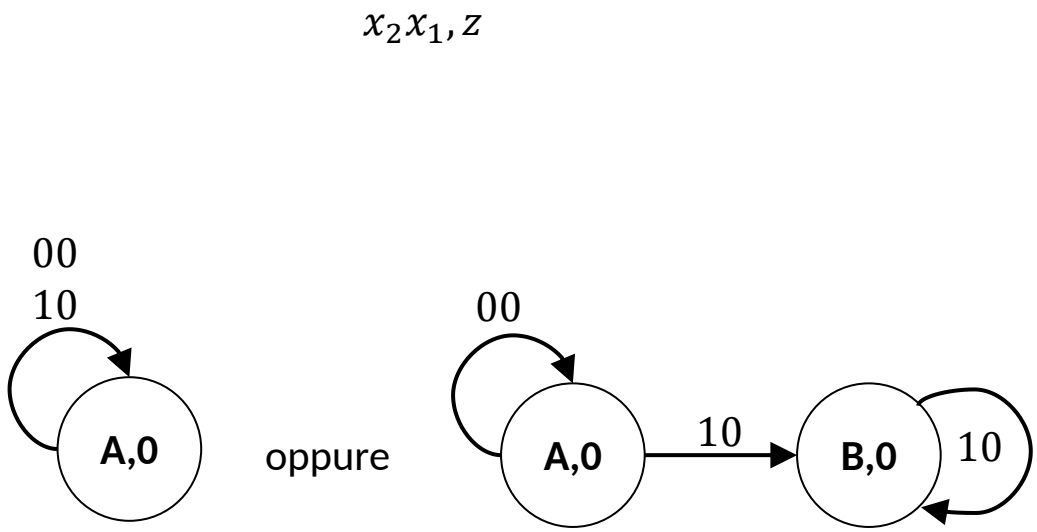
Grafo degli stati (1)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»

Il punto di partenza per fare il grafo è arbitrario, purché poi ci si ricordi di farlo tutto. Nel nostro esempio, inizio assumendo di aver visto l'alternanza richiesta in passato (**devo fare assunzioni sul passato** perché sto progettando una rete sequenziale), che l'ultimo segnale andato a 1 sia x_1, e di avere ingresso attuale 00. Ovvero le ultime due configurazioni degli ingressi sono stati 01-00 (Stato A).

Grafo degli stati (2)

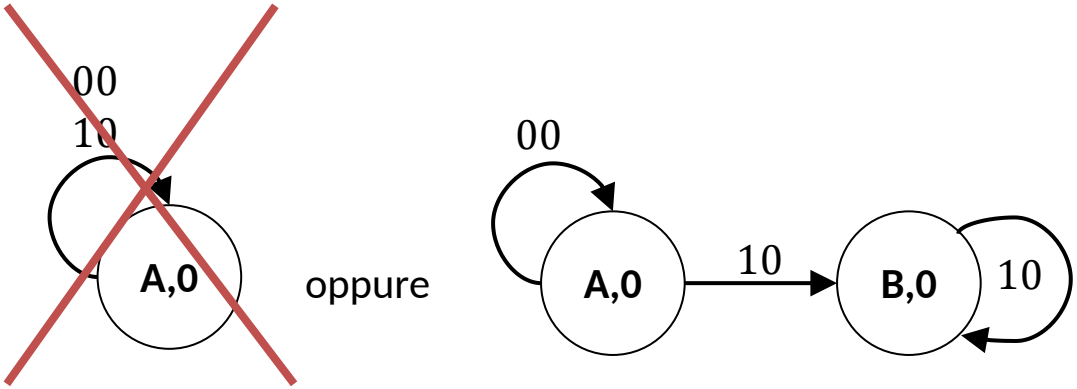


In stato A stabile per 00 sono possibili due configurazioni degli ingressi, ovvero 10 e 01 (11 è impossibile da specifica e anche per corretto funzionamento di una RSA). Quello che rispetta l'alternanza è 10. Per considerarlo, ho due alternative: restare in stato A (cioè rendere A stabile anche per 10) o andare in un nuovo stato B, stabile per 10. Quale è quella corretta?

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»

Grafo degli stati (3)

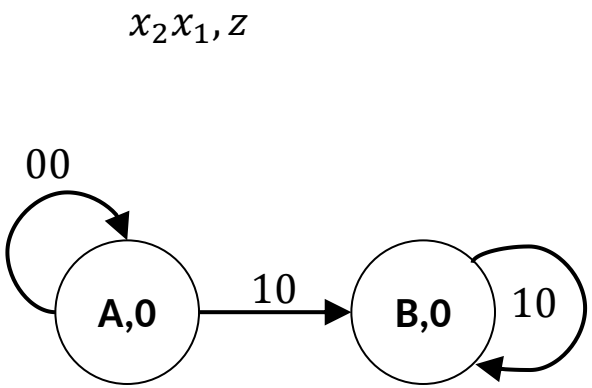
$x_2 x_1, z$



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»

Se rimanessi in stato A, gli ingressi potrebbero continuamente assumere valori 00 seguiti da 10 (entrambe configurazioni stabili per A). Questo violerebbe l'alternanza, **ma l'uscita rimarrebbe a 0**. In altre parole, non ho modo di riconoscere l'alternanza richiesta con questa scelta, quindi è sbagliata.

Grafo degli stati (4)



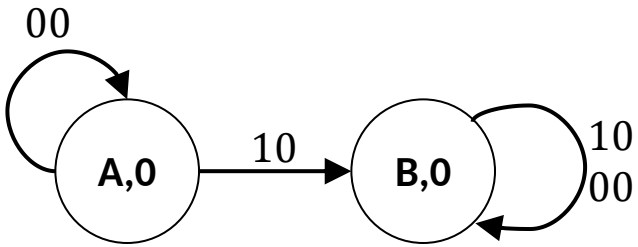
Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»

Lo stato B mi serve quindi per ricordare di aver visto x_2 come ingresso con ultimo valore ad 1. In altre parole, mi serve per ricordare che le prossime configurazioni degli ingressi che rispettano la sequenza sono 00 seguita da 01.
Da B è possibile che l'ingresso vari solo assumendo valore 00 (regole di corretto impiego), e di nuovo devo scegliere se restare in B, tornare in A o spostarmi in un nuovo stato C.

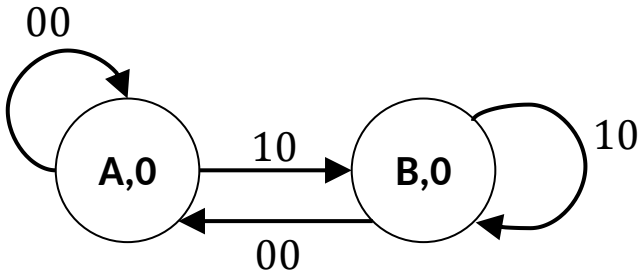
Grafo degli stati (5)

$x_2 x_1, z$

NO



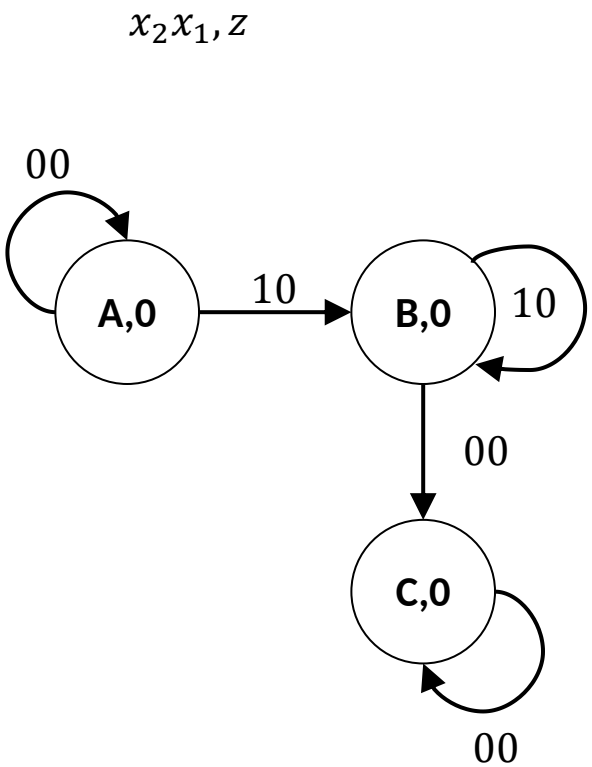
NO



Se restassi in stato B, rendendolo stabile per 00, avrei di nuovo il problema che gli ingressi potrebbero assumere valori 00-10-00-10-… all'infinito, violando l'alternanza senza che la rete possa portare l'uscita a 1.
Similmente, ritornando in A, potrei generare la stessa sequenza di ingressi, spostandomi continuamente da A a B.

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»

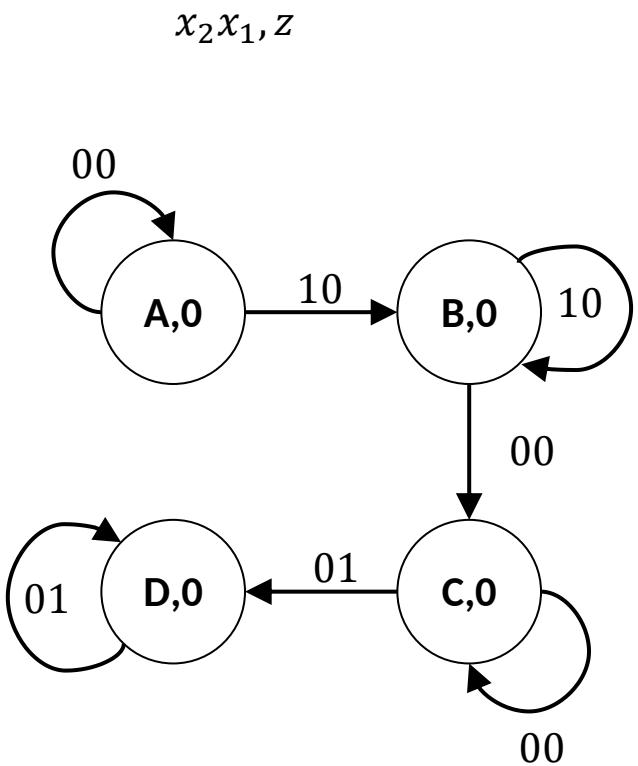
Grafo degli stati (6)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»

La soluzione corretta prevede di spostarsi in C, stabile per 00. C è il **simmetrico** di A, ricorda che gli ultimi due ingressi sono stati 10-00, e quindi per rispettare l'alternanza devo vedere 01.
Da C posso uscire con ingressi 01 (sequenza corretta) o ingressi 10. Continuiamo a progettare il riconoscimento dell'alternanza corretta.

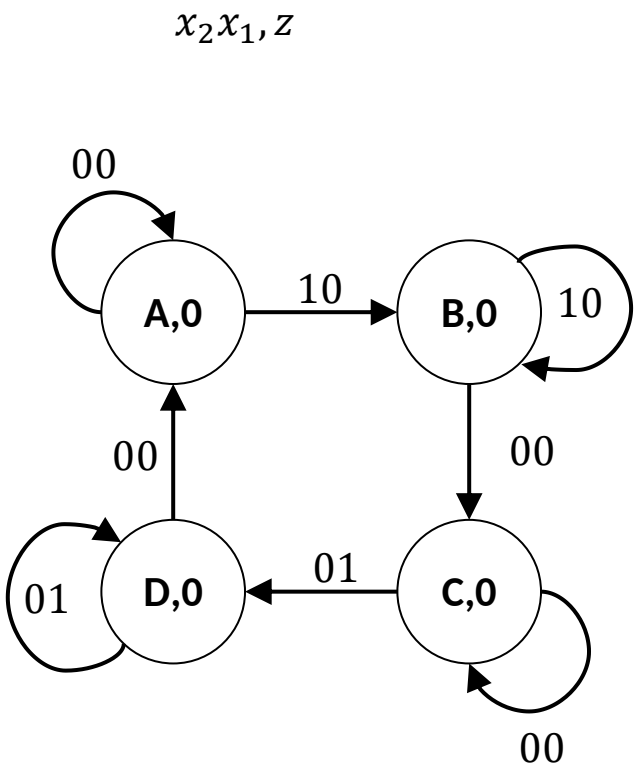
Grafo degli stati (7)



Da C, per ingressi 01, non posso andare in A (che ricorda la sequenza 01-00), né in B (che ricorda di aver visto come ultimo valore a 1 l'ingresso x_2), né restare in C (perché altrimenti potrei violare l'alternanza generando ingressi 00-01-00-01-... senza portare l'uscita ad 1). La rete si porterà quindi in stato D (simmetrico di B) che ricorda che x_1 è l'ultimo ingresso visto con valore 1.

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»

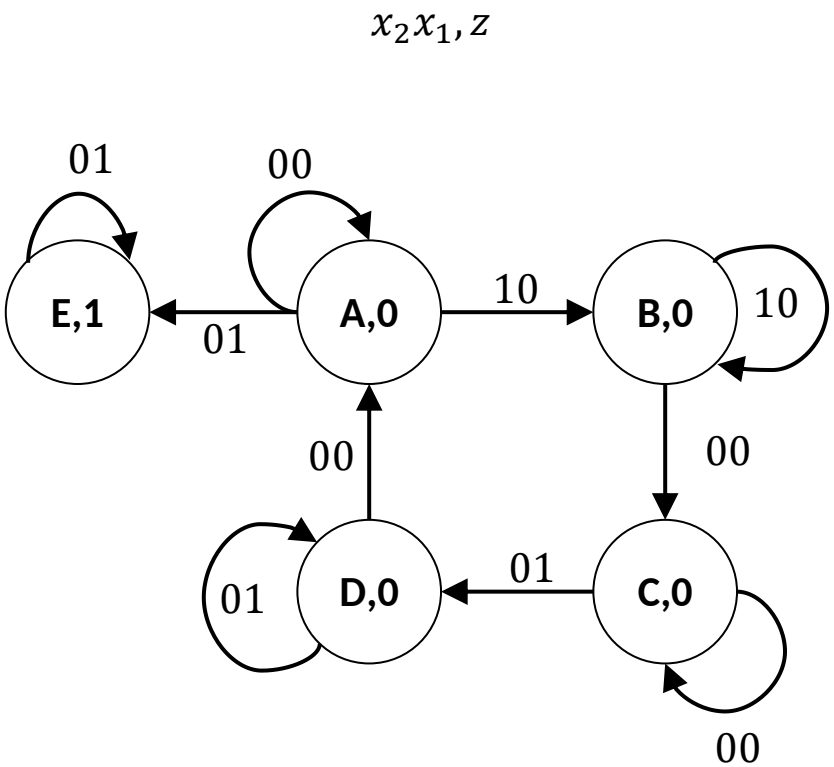
Grafo degli stati (8)



L'unica configurazione degli ingressi possibile per lasciare lo stato D è 00 (regole di corretto impiego).
Se in D osservo ingressi 00, poiché D ricorda di aver visto x_1 come ultimo ingresso a 1, significa che la sequenza degli ingressi è stata 01-00 è l'alternanza è stata rispettata, **ovvero ciò che ricorda A**. Mi sposto quindi in A, chiudendo il ciclo di stati che riconoscono il rispetto dell'alternanza.

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»

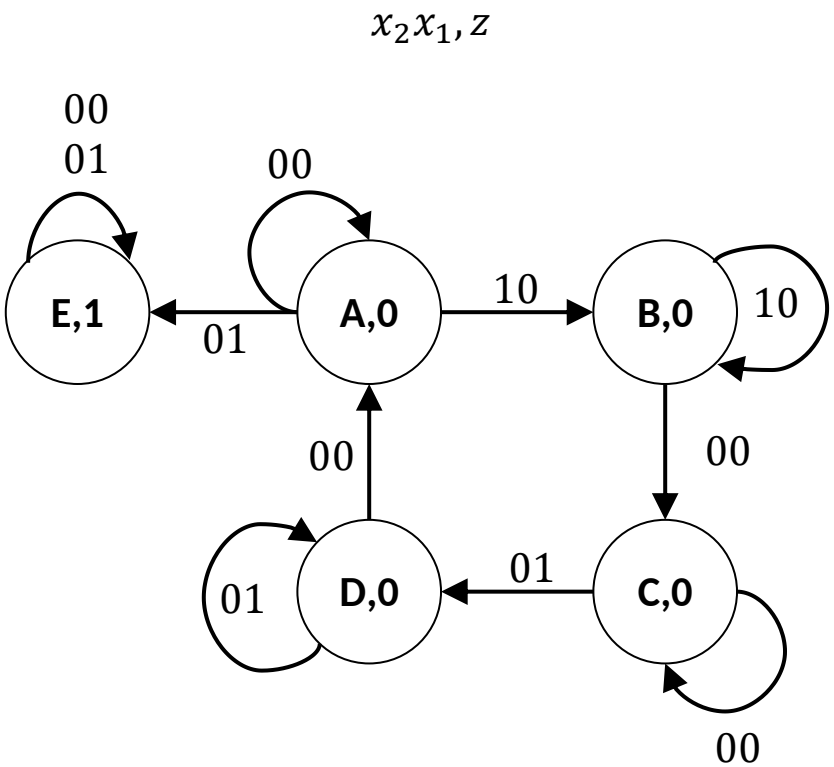
Grafo degli stati (9)



Ho già considerato tutte le configurazioni di ingressi **possibili** per B e D. Devo inserire ancora ingresso 10 per A e 01 per C, ovvero quelli che violano l'alternanza. In entrambi i casi, dovrò spostarmi in un nuovo stato, per cambiare l'uscita (modello di Moore, uscita funzione solo dello stato).

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»

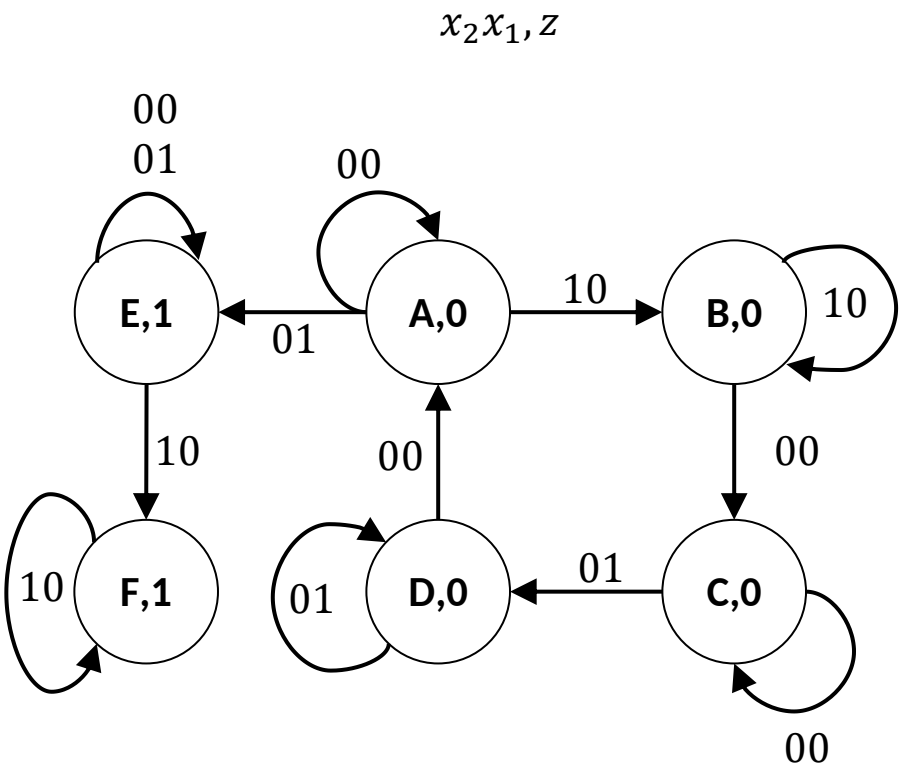
Grafo degli stati (10)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»

Stato E ricorda sicuramente che l'alternanza è stata violata. Inoltre, ricorda che è stata violata da configurazioni di ingressi consecutivi con x_1=1, ovvero che ho visto almeno due volte di seguito la sequenza di ingressi 00-01. **Vederla altre volte non cambia ciò che devo fare in futuro,** ovvero attendere 00-10 per ripristinare la corretta alternanza. Posso quindi rimanere in stato E, sia per ingressi 01, sia per 00.

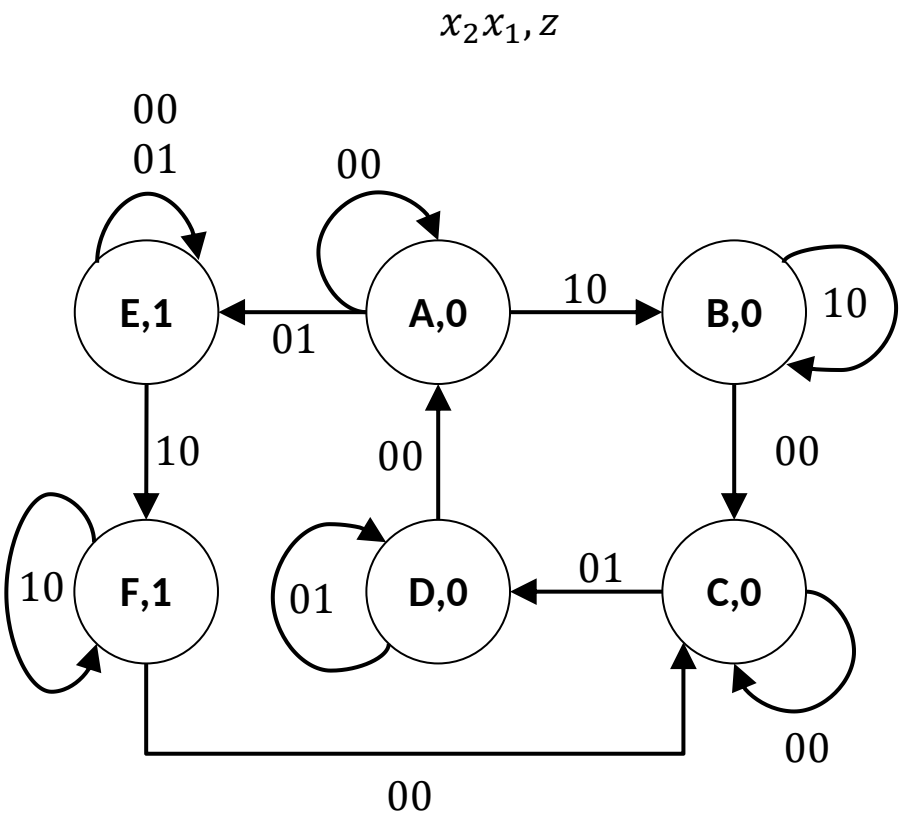
Grafo degli stati (11)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»
F	«Alternanza ripristinata, x2 ultimo ingresso a valore 1»

Da stato E, quindi, con ingresso 10 (unico che devo ancora considerare) si ripristina l'alternanza. Non posso però ancora tornare nel ciclo A-B-C-D perché da specifica devo mantenere uscita 1 fino a quando x_2 torna a 0. Non posso nemmeno rimanere in E, perché sarebbe stabile per ogni configurazione possibile degli ingressi (stato assorbente). Mi sposto quindi in F, che ricorda che stiamo per ripristinare l'alternanza voluta.

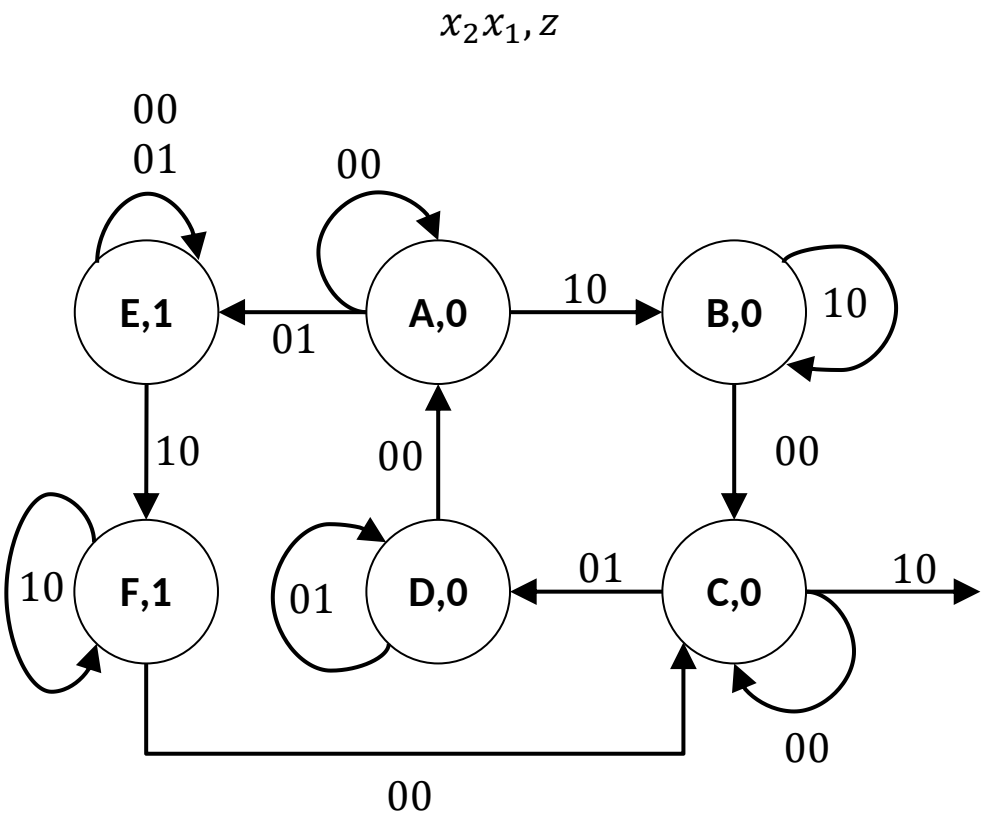
Grafo degli stati (12)



Da F, quando x_2 torna a 0, ovvero ho ingressi 00, posso tornare nel ciclo corretto. In quale stato? **Sicuramente in uno stabile per 00,** quindi o A o C. A ricorda che gli ultimi due ingressi sono stati 01-00, ma se provengo da F, gli ultimi due ingressi sono stati 10-00. La scelta corretta è quindi C.

Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»
F	«Alternanza ripristinata, x2 ultimo ingresso a valore 1»

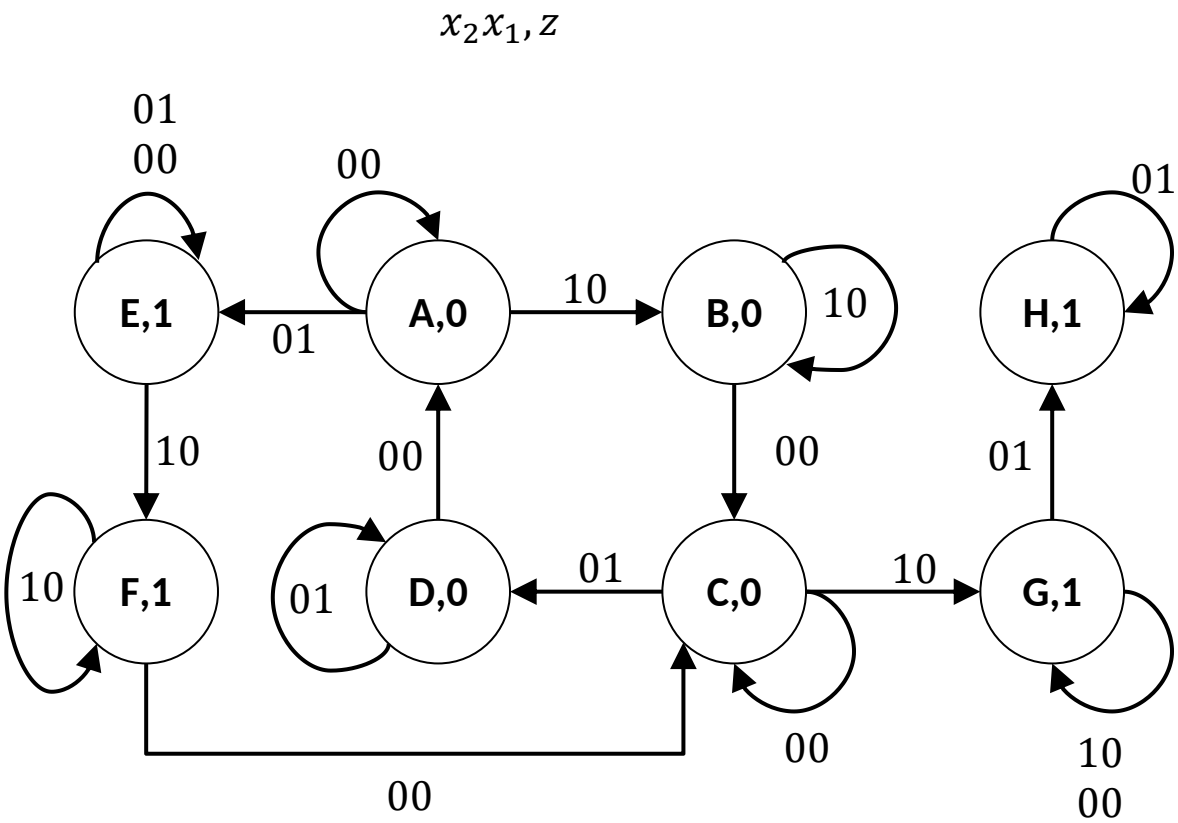
Grafo degli stati (13)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»
F	«Alternanza ripristinata, x2 ultimo ingresso a valore 1»

Rimane da considerare solo ingresso 10 per stato C. Rappresentando un'altra violazione dell'alternanza richiesta posso spostarmi solo in E o F (stati con uscita 1), o in un nuovo stato. Posso spostare la rete in E o F? No, perché E ed F attendono x_2=1 per tornare nel ciclo corretto, ma provenendo da C serve x_1=1 per ripristinare l'alternanza.

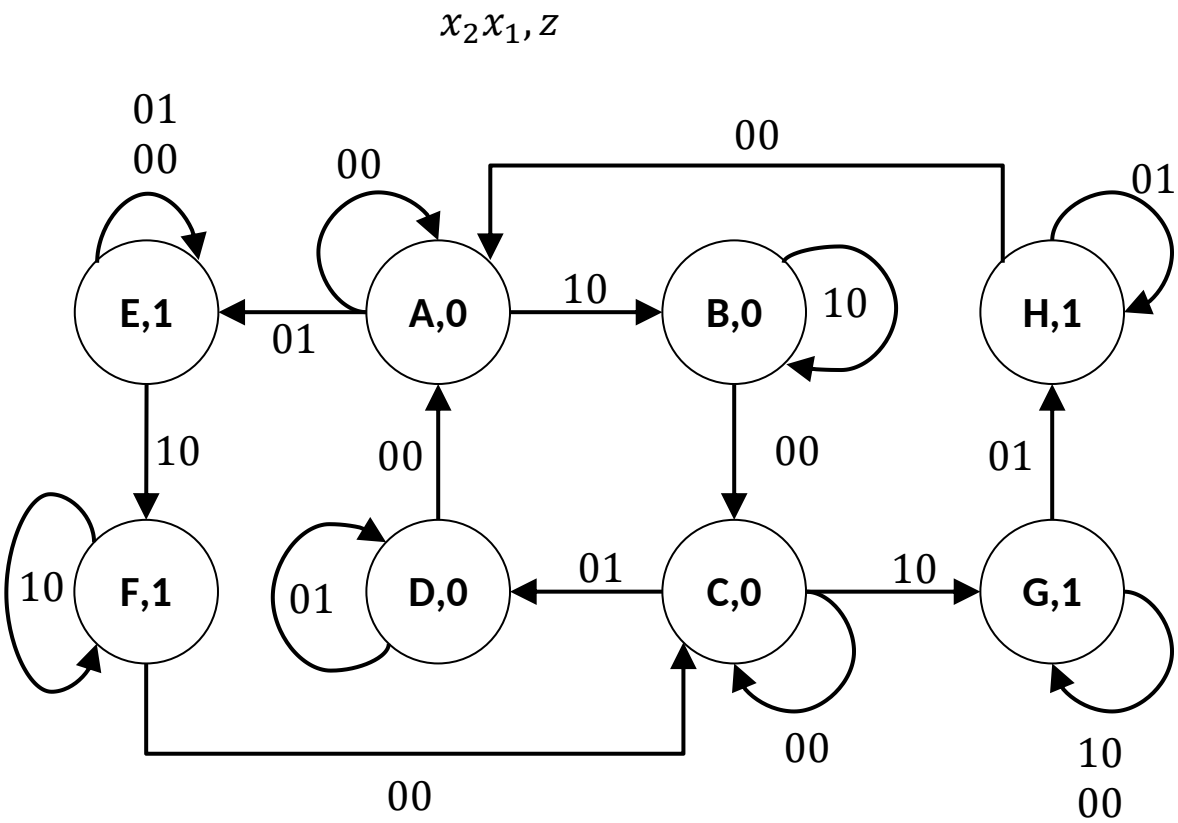
Grafo degli stati (14)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»
F	«Alternanza ripristinata, x2 ultimo ingresso a valore 1»
G	«Alternanza violata, attendo x1=1 per ripristinarla»
H	«Alternanza ripristinata, x1 ultimo ingresso a valore 1»

Mi sposto quindi in G, che con ragionamenti analoghi a E, può essere stabile non solo per 10, ma anche per 00 (vedere infinite volte 00-10-00-10-00-10-… non cambia il mio comportamento futuro). Similmente, avrò bisogno di H, quando vedo finalmente x_1=1, per attendere che x_1 torni a 0 prima di rientrare nel ciclo A-B-C-D.

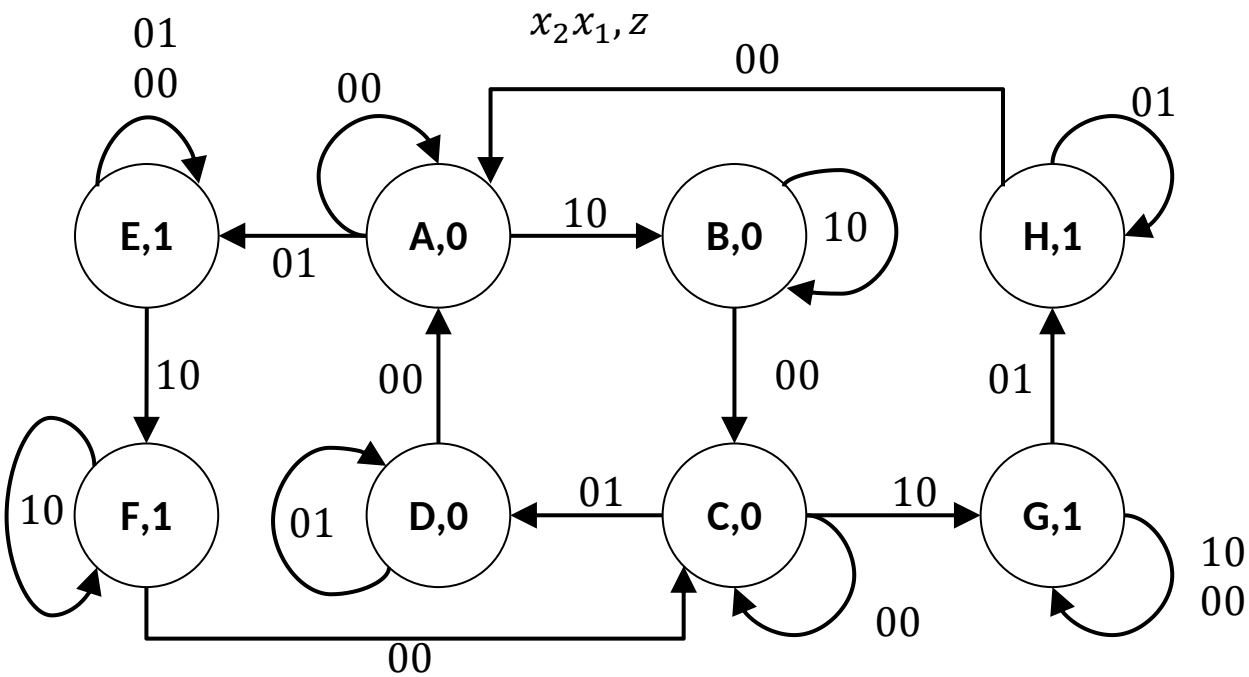
Grafo degli stati (15)



Stato	Riassunto
A	«Alternanza OK, ultimi due ingressi 01-00»
B	«Alternanza OK, x2 ultimo ingresso a valore 1»
C	«Alternanza OK, ultimi due ingressi 10-00»
D	«Alternanza OK, x1 ultimo ingresso a valore 1»
E	«Alternanza violata, attendo x2=1 per ripristinarla»
F	«Alternanza ripristinata, x2 ultimo ingresso a valore 1»
G	«Alternanza violata, attendo x1=1 per ripristinarla»
H	«Alternanza ripristinata, x1 ultimo ingresso a valore 1»

Infine, da H con ingresso 00 torno nel ciclo corretto, in particolare in stato A, essendo quello che memorizza il fatto che le ultime due configurazioni degli ingressi sono state 01 seguite da 00.

Verifica



Si consiglia di **verificare** se il grafo che abbiamo creato rispetta le specifiche e genera l'evoluzione dell'uscita prevista.

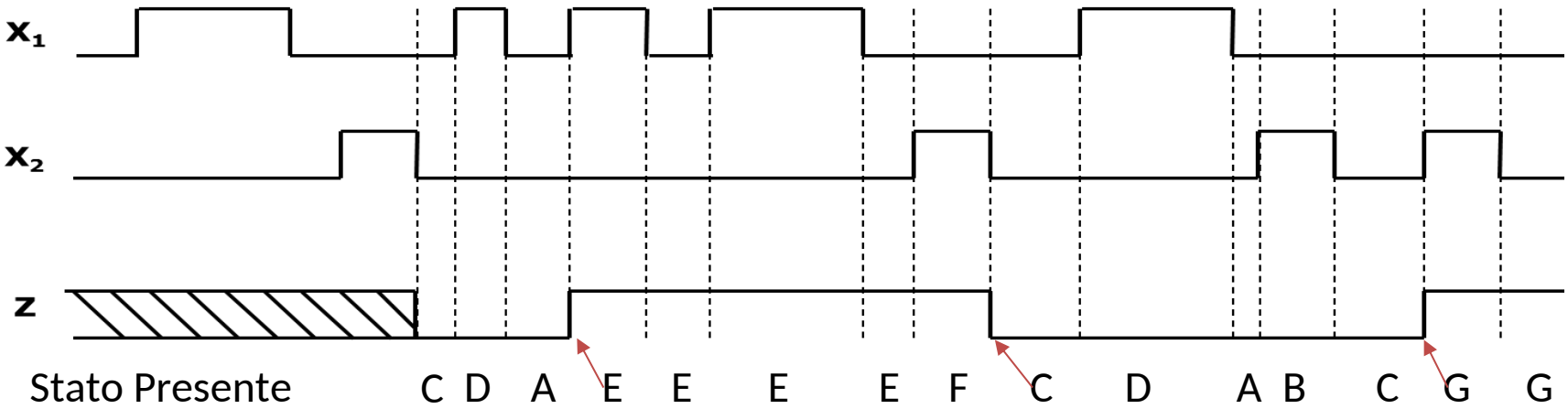
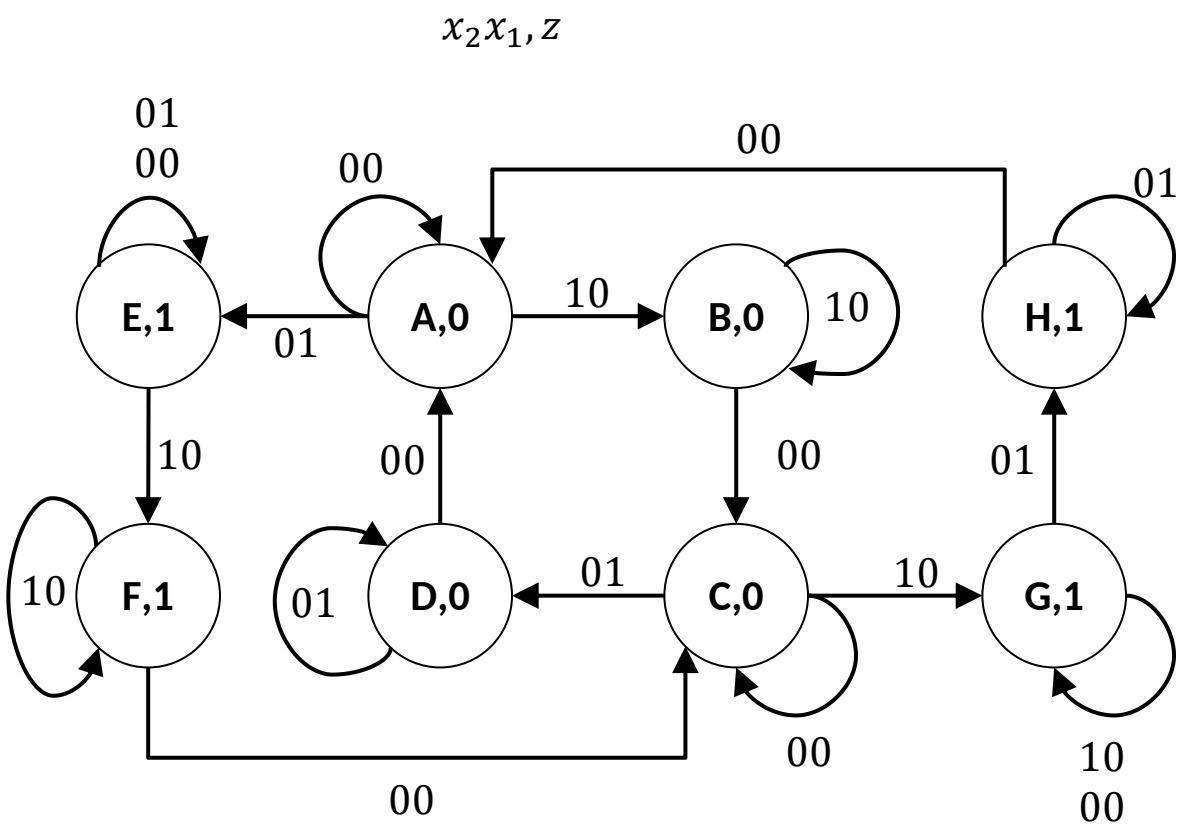


Tabella di Flusso



$x_2\ x_1$

stato presente	00	01	11	10	z
A	A	E	-	B	0
B	C	-	-	B	0
C	C	D	-	G	0
D	A	D	-	-	0
E	E	E	-	F	1
F	C	-	-	F	1
G	G	H	-	G	1
H	A	H	-	-	1

stato futuro

Adiacenze tra stati e codifica

- Non ci sono colonne con una sola stabilità, quindi non posso escluderne nessuna dall'analisi sulle adiacenze tra stati presenti e futuri
- Con l'ipotesi di codifica sottostante, ottenuta **ricalcando il grafo**, ho due corse critiche: H→A per ingresso 00 e F→C sempre **per ingresso 00**.
- Non ho indifferenze da sfruttare nella colonna 00.
- Ho però altre due transizioni che puntano già a A e C, ovvero D →A e B →C. Ma con questa disposizione non riesco a sfruttarle per effettuare transizioni multiple H →D →A e F →B →C, perché H non è adiacente a D e F non è adiacente a B.

$y_2 \backslash y_1 y_0$	00		01		11		10
0	**E**	←	**A**	→	**B**		**H**
	↓		↑		↓		↑
1	**F**		**D**	←	**C**	→	**G**

NO

$x_2\ x_1$

stato presente	00	01	11	10	z
A	A	E	-	B	0
B	C	-	-	B	0
C	C	D	-	G	0
D	A	D	-	-	0
E	E	E	-	F	1
F	C	-	-	F	1
G	G	H	-	G	1
H	A	H	-	-	1

stato futuro

Adiacenze tra stati e codifica

- A → B, A → E
- B → C
- C → D, D → A
- C → G
- E → F, F → C sfrutto B → C
- G → H
- H → A sfrutto D → A

$x_2\ x_1$

stato presente	00	01	11	10	z
A	A	E	-	B	0
B	C	-	-	B	0
C	C	D	-	G	0
D	A	D	-	-	0
E	E	E	-	F	1
F	C	-	-	F	1
G	G	H	-	G	1
H	A	H	-	-	1

stato futuro

Adiacenze tra stati e codifica

- Realizzando il ciclo A →B →C →D →A sulla prima riga, riesco ad avere B adiacente a F e D adiacente ad H.
- Riesco quindi a sfruttare queste adiacenze per realizzare le transizioni esposte a corsa critica F →C e H →A come transizioni multiple H →D →A e F →B →C

$y_2 \backslash y_1 y_0$	00		01		11		10	
0	**A**	→	**B**	→	**C**	→	**D**	→
	↓				↓			
1	**E**	→	**F**		**G**	→	**H**	

OK

$x_2\ x_1$

stato presente

	00	01	11	10	z
A	A	E	-	B	0
B	C	-	-	B	0
C	C	D	-	G	0
D	A	D	-	-	0
E	E	E	-	F	1
F	~~C~~ **B**	-	-	F	1
G	G	H	-	G	1
H	~~A~~ **D**	H	-	-	1

stato futuro

Tabella delle transizioni

$x_2\ x_1$

stato presente	00	01	11	10	z
A	A	E	-	B	0
B	C	-	-	B	0
C	C	D	-	G	0
D	A	D	-	-	0
E	E	E	-	F	1
F	~~C~~ B	-	-	F	1
G	G	H	-	G	1
H	~~A~~ D	H	-	-	1

stato futuro

$x_2\ x_1$

$y_2y_1y_0$	00	01	11	10	z
A=000	000	100	-	001	0
B=001	011	-	-	001	0
C=011	011	010	-	111	0
D=010	000	010	-	-	0
E=100	100	100	-	101	1
F=101	001	-	-	101	1
G=111	111	110	-	111	1
H=110	010	110	-	-	1

$Y_2Y_1Y_0$

$y_2 \backslash y_1y_0$	00		01		11		10	
0	**A**	→	**B**	→	**C**	→	**D**	→
	↓				↓			
1	**E**	→	**F**		**G**	→	**H**	

$$z\ (SP) = y_2$$

Sintesi combinatoria

$y_2 = 0$

$y_1 y_0$ \ $x_2 x_1$	00	01	11	10
00	0	1	-	0
01	0	-	-	0
11	0	0	-	1
10	0	0	-	-

$y_2 = 1$

$y_1 y_0$ \ $x_2 x_1$	00	01	11	10
00	1	1	-	1
01	0	-	-	1
11	1	1	-	1
10	0	1	-	-

Y_2

$$Y_2\,(SP) = y_2\,x_2 + y_1'\,x_1 + y_1\,x_2 + y_2\,x_1 + y_2\,y_1\,y_0 + y_2\,y_1'\,y_0'$$

Sintesi combinatoria

$x_2\,x_1$

y_1y_0	00	01	11	10
00	0	0	-	0
01	1	-	-	0
11	1	1	-	1
10	0	1	-	-

$y_2 = 0$

x_2x_1

	00	01	11	10
00	0	0	-	0
01	0	-	-	0
11	1	1	-	1
10	1	1	-	-

$y_2 = 1$

Y_1

$$Y_1\ (SP) = y_1y_0 + y_1x_1 + y_2y_1 + y_2'\,y_0\,x_2'$$

Sintesi combinatoria

$x_2\,x_1$

y_1y_0	00	01	11	10
00	0	0	-	1
01	1	-	-	1
11	1	0	-	1
10	0	0	-	-

$y_2 = 0$

x_2x_1

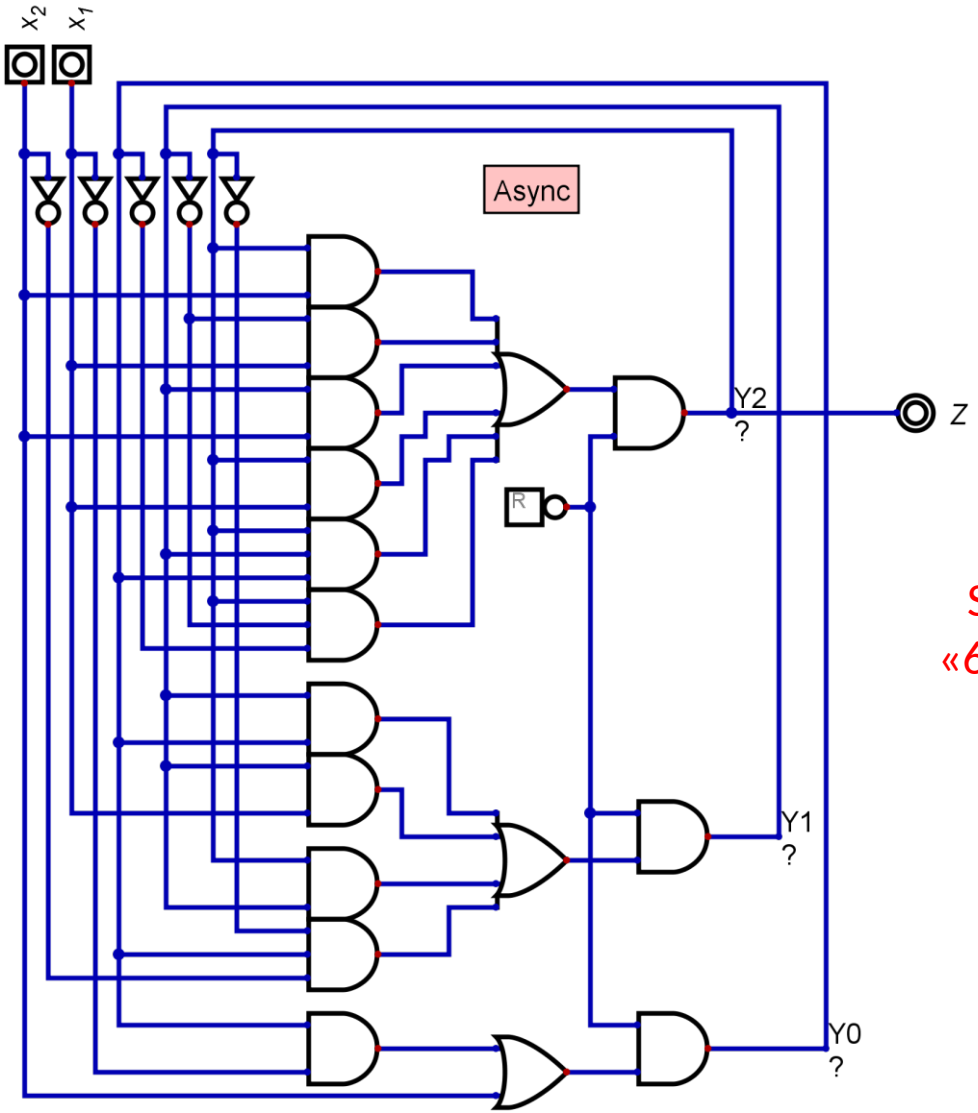
	00	01	11	10
00	0	0	-	1
01	1	-	-	1
11	1	0	-	1
10	0	0	-	-

$y_2 = 1$

Y_0

$$Y_0\ (SP)\ =\ y_0x_1'\ +\ x_2$$

Schema logico con rete di reset

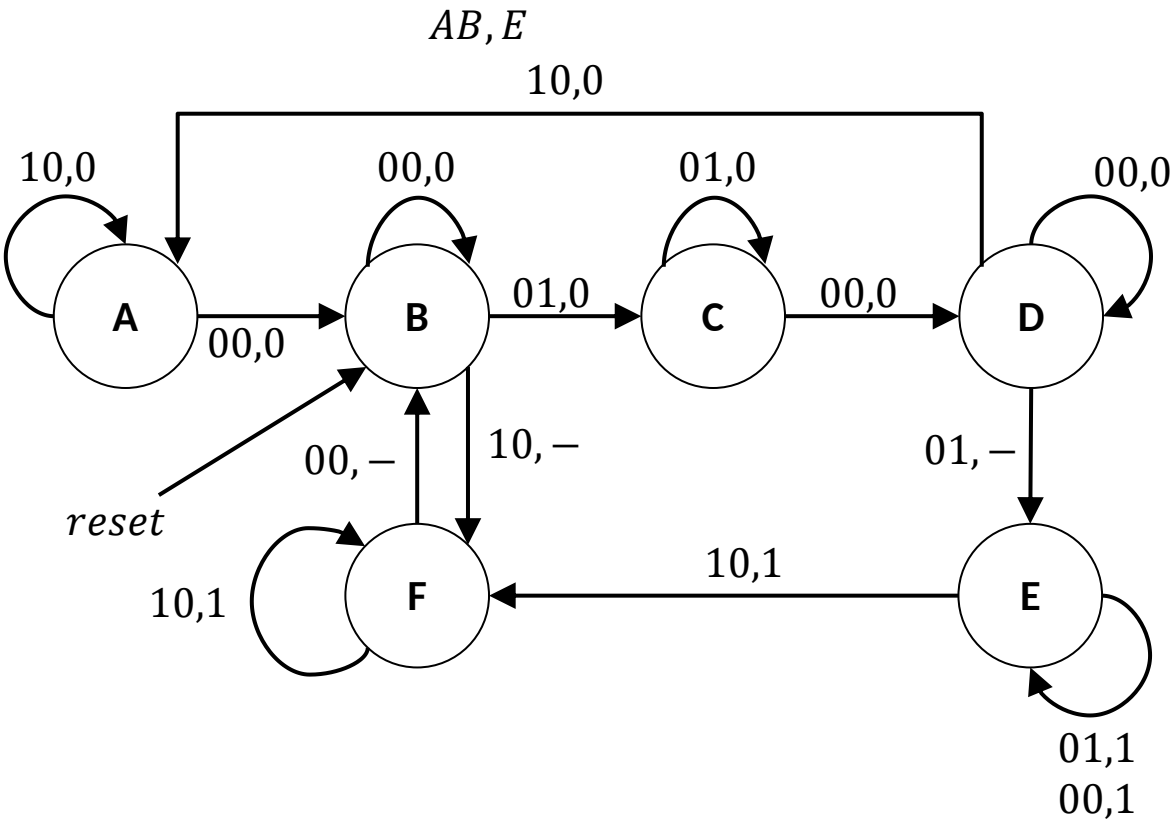


Simulare con Digital:
«6_RSA_example1.dig»

Esempio 2

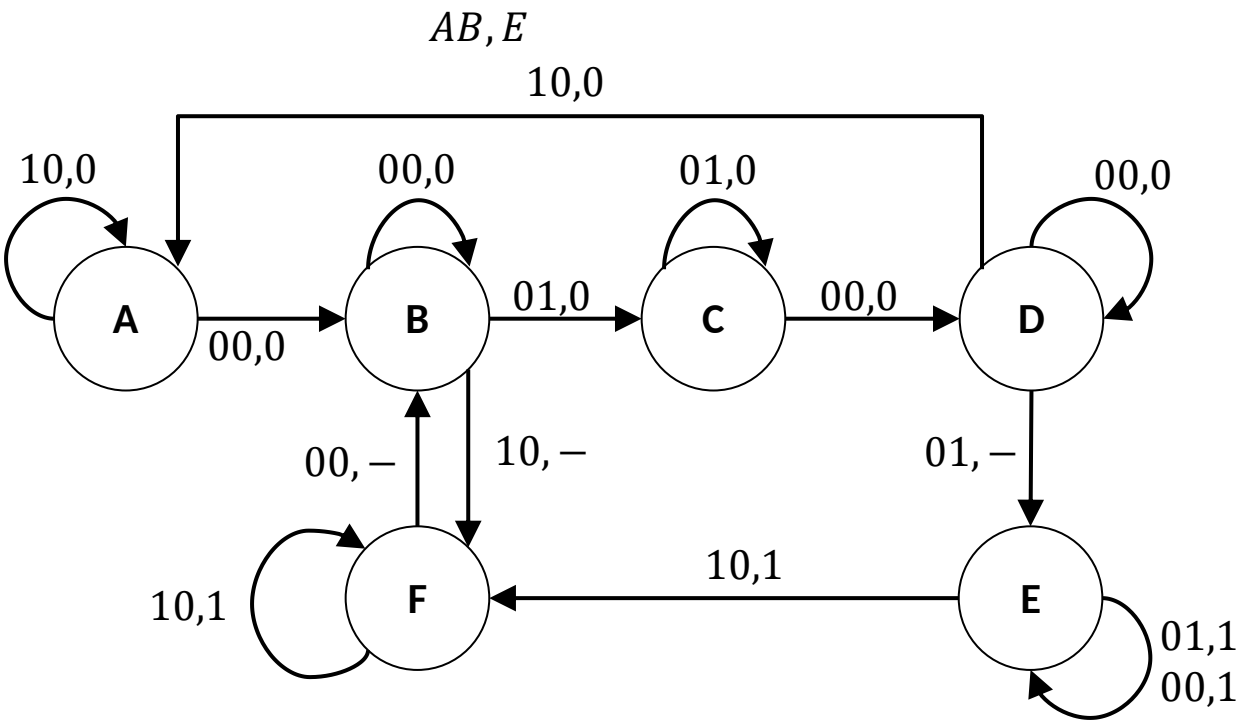
Una rete sequenziale asincrona deve monitorare una catena di produzione mediante due segnali di ingresso A, B. A assume il valore 1 quando il prodotto è in lavorazione, mentre vale 0 durante la successiva fase di analisi del prodotto. B assume sempre valore 0 durante la fase di lavorazione, mentre durante la fase di analisi può presentare variazioni. Se durante l'analisi B presenta esattamente un fronte di salita ed uno di discesa, il pezzo analizzato è stato correttamente lavorato e l'unica uscita E della rete (errore) assumerà valore 0. Se il prodotto non è stato correttamente lavorato, B può presentare 0 fronti o più di un fronte di salita e di discesa. In tal caso E dovrà assumere valore 1 non appena si abbia indicazione di un prodotto non correttamente lavorato e dovrà mantenere tale valore finché non inizia la fase di analisi del prodotto successivo. Si assuma che A e B non cambino mai contemporaneamente. All'avvio della macchina inizia sempre una fase di analisi. Sintetizzare secondo Mealy con espressioni SP di costo minimo per le funzioni combinatorie necessarie.

Grafo degli stati



Stato	Riassunto
A	«Lavorazione, prodotto precedente corretto»
B	«Analisi, nessuna variazione di B»
C	«Analisi, visto un fronte di salita di B»
D	«Analisi, visto un fronte di discesa di B»
E	«Analisi, visti più di 2 fronti di B, prodotto errato»
F	«Lavorazione, prodotto precedente errato»

Verifica


AB, E
10,0
10,0
00,0
01,0
00,0
A
B
C
D
00,0
01,0
00,0
00, –
10, –
01, –
10,1
10,1
F
E
01,1
00,1

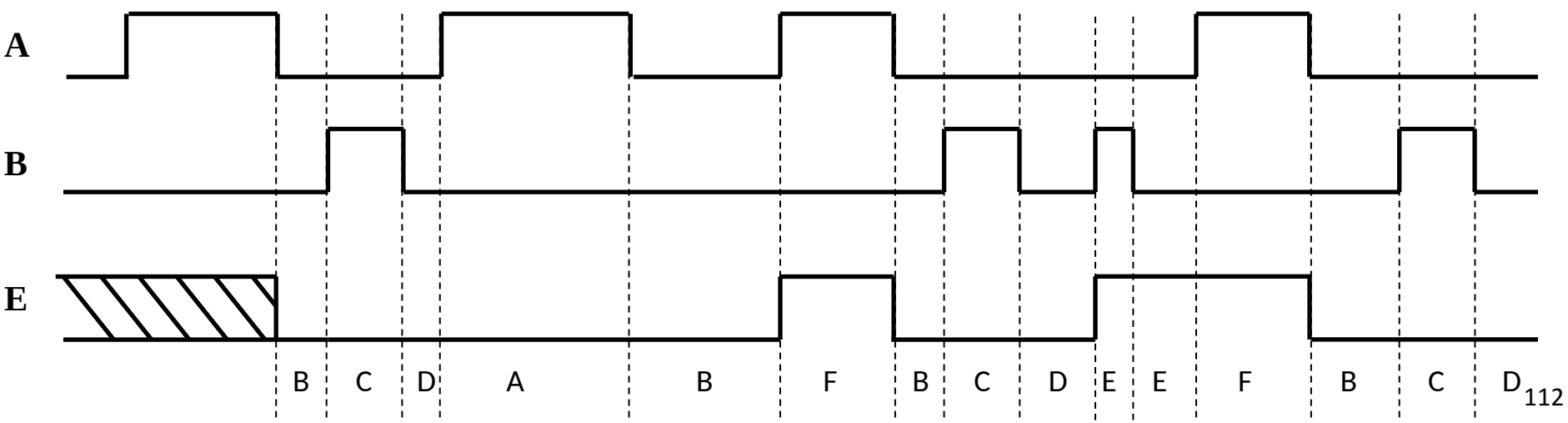
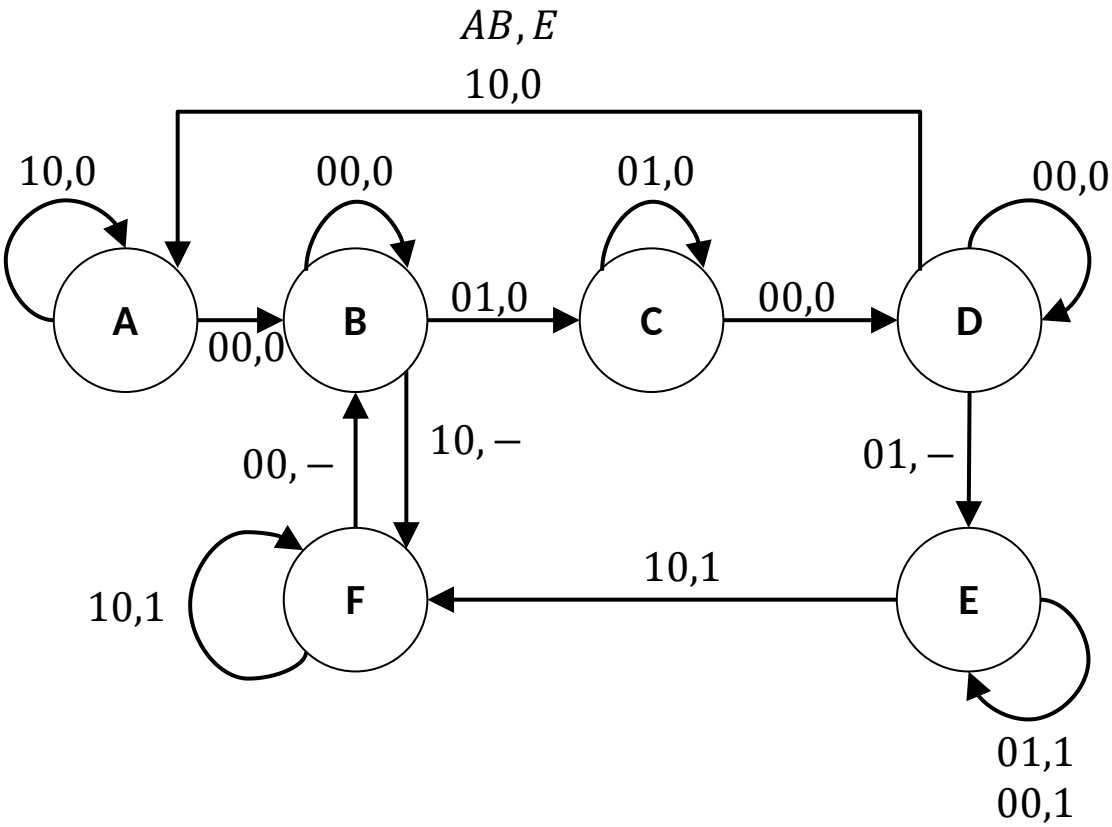

A
B
E
B C D A B F B C D E E F B C D

Tabella di flusso



stato presente

AB

	00	01	11	10
A	B,0	-	-	A,0
B	B,0	C,0	-	F,-
C	D,0	C,0	-	-
D	D,0	E,-	-	A,0
E	E,1	E,1	-	F,1
F	B,-	-	-	F,1

stato futuro, E

Adiacenze tra stati e codifica

- Non ci sono colonne con una sola stabilità, quindi non posso escluderne nessuna dall'analisi sulle adiacenze tra stati presenti e futuri
- Con l'ipotesi di codifica sottostante, ottenuta ricalcando il grafo, ho una corsa critica per ingresso 00, F→B, e una per ingresso 10, B→F.
- Non ho indifferenze da sfruttare nella colonna 00. Ho però un'altra transizione che punta già a B, ovvero A →B, con cui posso realizzare F→B come F→A→B
- Notare che se avessi messo F nella cella marcata da *, non sarebbe stato possibile raggiungere B passando per C, in quanto C non indica B come stato futuro, ma D.

$y_2 \backslash y_1 y_0$	00		01		11		10	
0	**A**	→	**B**	→	**C**	→	**D**	→
							↓	
1	F				*		**E**	→

AB

stato presente	00	01	11	10
A	B,0	-	-	A,0
B	B,0	C,0	-	101,- ~~F,~~
C	D,0	C,0	-	-
D	D,0	E,-	-	A,0
E	E,1	E,1	-	F,1
F	A,- ~~B,~~	-	-	F,1

stato futuro, E

Tabella delle transizioni

AB

stato presente	00	01	11	10
A	B,0	-	-	A,0
B	B,0	C,0	-	101,- ~~F,-~~
C	D,0	C,0	-	-
D	D,0	E,-	-	A,0
E	E,1	E,1	-	F,1
F	A,- ~~B,-~~	-	-	F,1

stato futuro, z

$y_2y_1y_0$

AB

	00	01	11	10
A=000	001,0	-	-	000,0
B=001	001,0	011,0	-	101,-
C=011	010,0	011,0	-	-
D=010	010,0	110,-	-	000,0
F=100	000,-	-	-	100,1
101	-	-	-	100,-
111	-	-	-	-
E=110	110,1	110,1	-	100,1

$Y_2Y_1Y_0, E$

$y_2 \backslash y_1y_0$	00		01		11		10	
0	**A**	→	**B**	→	**C**	→	**D**	→
							↓	
1	F						**E**	→

$$E\ (SP) = y_2$$

Sintesi combinatoria

$y_2 = 0$

$y_1 y_0$ \ AB	00	01	11	10
00	0	-	-	0
01	0	0	-	1
11	0	0	-	-
10	0	1	-	0

$y_2 = 1$

$y_1 y_0$ \ AB	00	01	11	10
00	0	-	-	1
01	-	-	-	1
11	-	-	-	-
10	1	1	-	1

Y_2

$$Y_2\,(SP) = y_2 A + y_2\, y_1 + y_0{}'B + y_0 A$$

Sintesi combinatoria

$y_1 y_0$ \ AB

	00	01	11	10
00	0	-	-	0
01	0	1	-	0
11	1	1	-	-
10	1	1	-	0

$y_2 = 0$

AB

	00	01	11	10
00	0	-	-	0
01	-	-	-	0
11	-	-	-	-
10	1	1	-	0

$y_2 = 1$

Y_1

$$Y_1\ (SP) = B + y_1 A'$$

Sintesi combinatoria

$y_2 = 0$

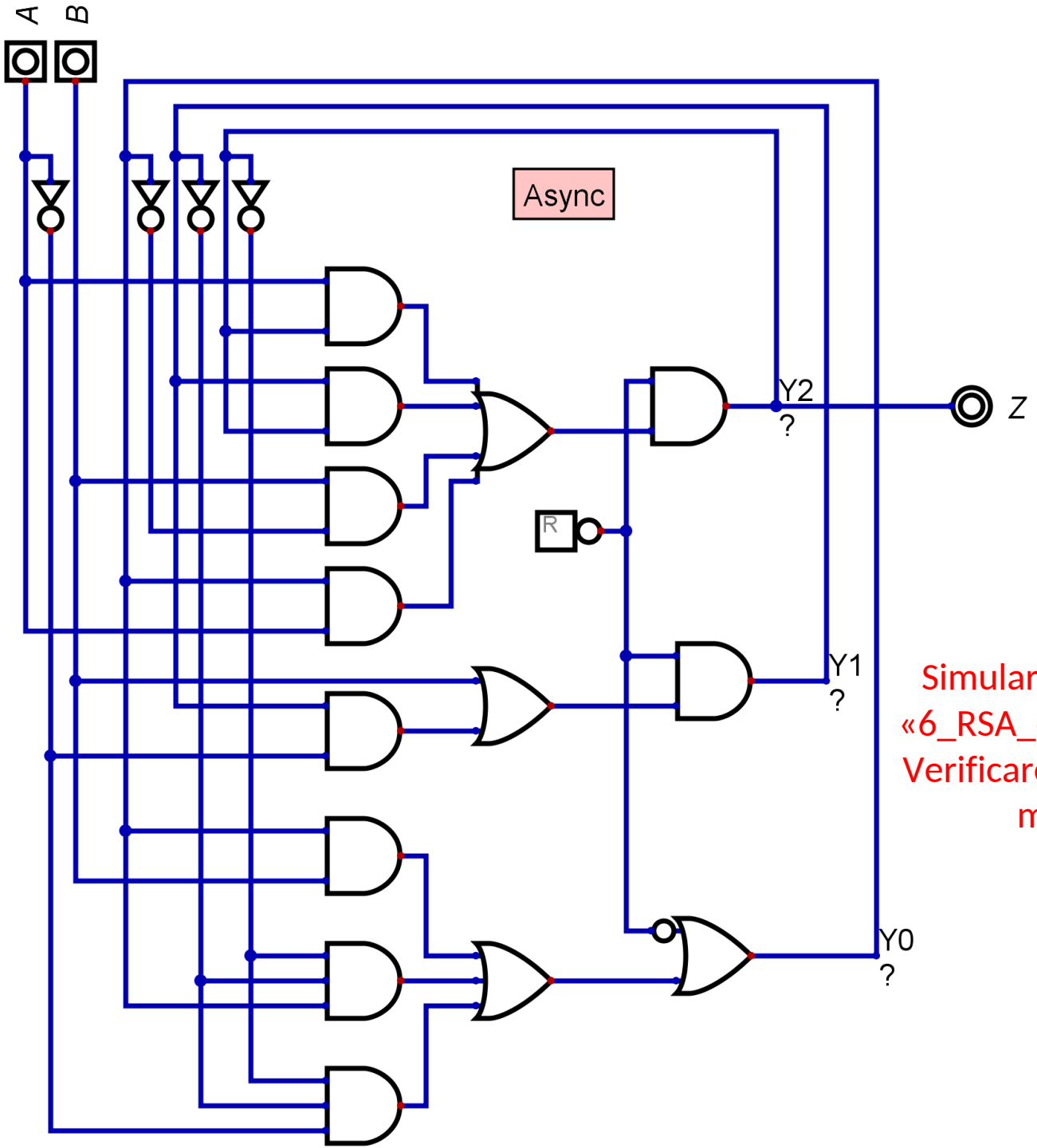
y_1y_0 \ AB	00	01	11	10
00	1	-	-	0
01	1	1	-	1
11	0	1	-	-
10	0	0	-	0

$y_2 = 1$

y_1y_0 \ AB	00	01	11	10
00	0	-	-	0
01	-	-	-	0
11	-	-	-	-
10	0	0	-	0

Y_0

$$Y_0\ (SP) = y_0B + y_2'\,y_1'y_0 + y_2'y_1'A'$$



Simulare con Digital:
«6_RSA_example2.dig»
Verificare le transizioni multiple

ANALISI

Il procedimento di analisi

Il procedimento di analisi di una rete sequenziale asincrona è formato da 5 passi e consente di dedurne il comportamento dallo schema logico:

1: individuazione delle variabili di stato,

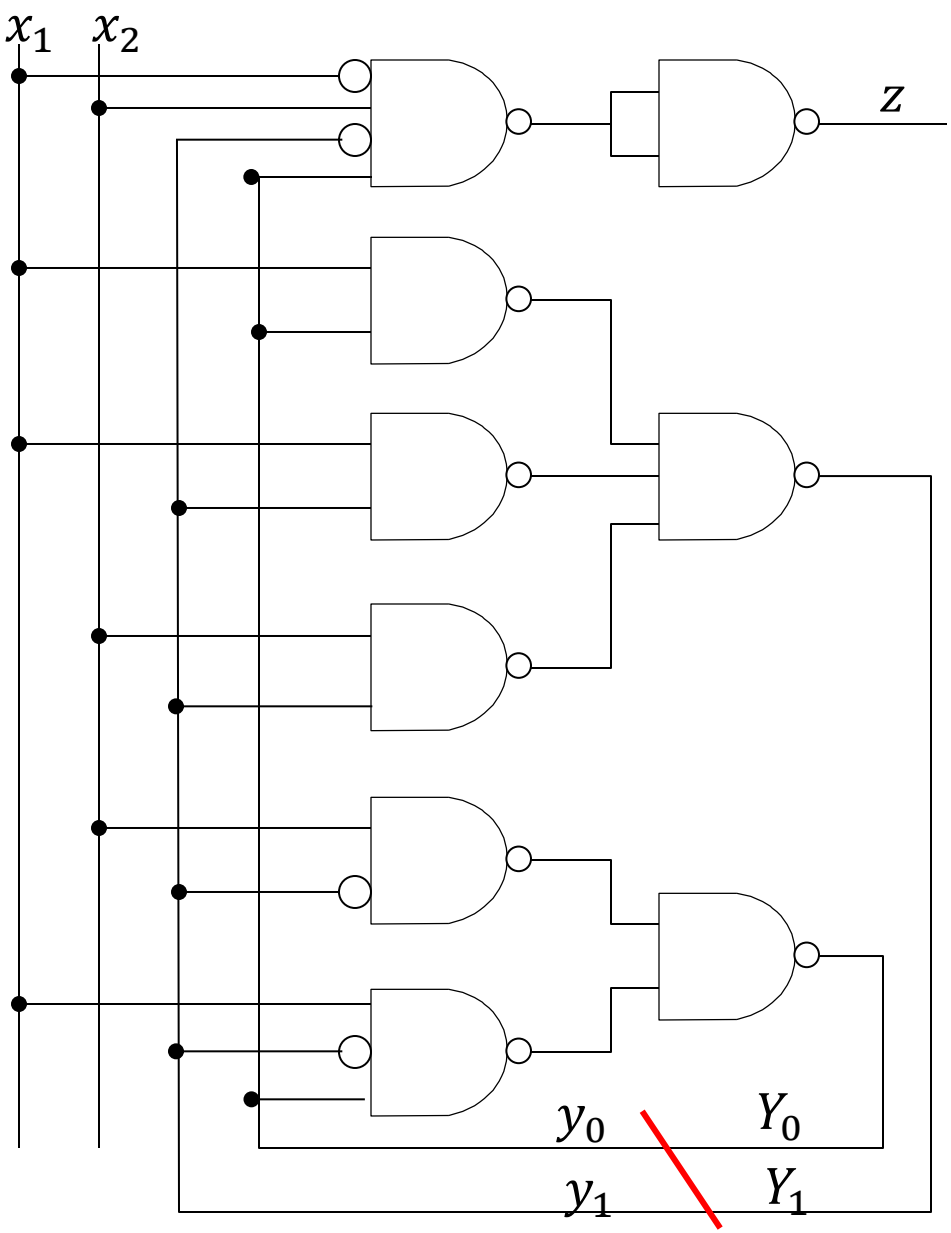
2: analisi della parte combinatoria,

3: individuazione della tabella delle transizioni,

4: studio delle condizioni di stabilità e individuazione della tabella di flusso,

5: passaggio al grafo degli stati e descrizione testuale del comportamento.

Esempio: rete a NAND con 2 retroazioni



La rete ha

- 2 ingressi (x_1, x_2)
- 2 variabili in retroazione
- 1 uscita (z)

Una volta individuate le retroazioni, si «tagliano» virtualmente per distinguere le **variabili di stato presente** (ingressi delle funzioni combinatorie) che indichiamo con y_1, y_0 dalle **variabili di stato futuro** (uscite), che indichiamo con Y_1, Y_0 . Scriviamo poi le espressioni delle uscite e delle variabili di stato futuro, trasformandole in SP/PS se necessario.

$$Y_1 = (x_1 \uparrow y_0) \uparrow (x_1 \uparrow y_1) \uparrow (x_2 \uparrow y_1)$$
$$= x_1 \cdot y_0 + x_1 \cdot y_1 + x_2 \cdot y_1$$

$$Y_0 = (x_2 \uparrow y_1') \uparrow (x_1 \uparrow y_1' \uparrow y_0)$$
$$= x_2 \cdot y_1' + x_1 \cdot y_1' \cdot y_0$$

$$z = x_1' \cdot x_2 \cdot y_1' \cdot y_0$$

Mappe di Karnaugh

- Dalle espressioni si ricostruiscono le mappe di Karnaugh delle singole variabili, inserendo valore «1»/«0» nelle celle appartenenti ai raggruppamenti rettangolari individuati dagli implicanti/implicati dell'espressione SP/PS e il valore complementare nelle celle al di fuori dei raggruppamenti.
- Verifico se sono state eliminate a priori tutte le alee statiche con la copertura ridondante

$$Y_1 = x_1 y_0 + x_1 y_1 + x_2 y_1$$

y_1y_0 \ x_1x_2	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	1	1
10	0	1	1	1

Y_1

$$Y_0 = x_2\, y_1' + x_1\, y_1'\, y_0$$

y_1y_0 \ x_1x_2	00	01	11	10
00	0	1	1	0
01	0	1	1	1
11	0	0	0	0
10	0	0	0	0

Y_0

$$z = x_1'\, x_2\, y_1'\, y_0$$

y_1y_0 \ x_1x_2	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	0	0	0
10	0	0	0	0

z

Tabella delle transizioni

- Unendo le mappe di Karnaugh si ottiene la tabella delle transizioni.
- N.B.: La tabella che ottengo è **completamente specificata**, poiché ogni sintesi di funzioni combinatorie lo è. Non è detto però che tutte le celle siano effettivamente usate dalla rete.

y_1y_0 \ x_1x_2	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	1	1
10	0	1	1	1

Y_1

y_1y_0 \ x_1x_2	00	01	11	10
00	0	1	1	0
01	0	1	1	1
11	0	0	0	0
10	0	0	0	0

Y_0

y_1y_0 \ x_1x_2	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	0	0	0
10	0	0	0	0

z

y_1y_0 \ x_1x_2	00	01	11	10
00	00,0	01,0	01,0	00,0
01	00,0	01,1	11,0	11,0
11	00,0	10,0	10,0	10,0
10	00,0	10,0	10,0	10,0

Y_1Y_0,z

Analisi stabilità e Tabella di Flusso

- Nel passare da tabella delle transizioni a tabella di flusso cerco di capire **quali celle sono necessarie per descrivere il comportamento della rete**:
  - **In colonne prive di stati stabili** pongo indifferenze su stato futuro e uscite di tutte le celle, in quanto ingressi non possibili per la rete (funzionamento in modo fondamentale).
  - **Rimuovo eventuali transizioni multiple**, ovvero in ogni cella attraversata devo inserire lo stato di arrivo finale della transizione.
  - **Pongo a indifferenza celle non adiacenti a celle stabili di una riga**, in quanto non utilizzabili per i vincoli di corretto funzionamento di un RSA (raggiungerle richiederebbe la modifica contemporanea di più di una variabile di ingresso)
  - **Elimino stati privi di celle stabili**, in quanto presenti solo nella tabella delle transizioni, ma non in quella di flusso.
  - **Se automa di Mealy, pongo indifferenze sulle uscite nelle celle appartenenti a transizioni in cui l'uscita varia**

x_1x_2

y_1y_0	00	01	11	10
00	00,0	01,0	01,0	00,0
01	00,0	01,1	11,0	11,0
11	00,0	10,0	10,0	10,0
10	00,0	10,0	10,0	10,0

Y_1Y_0,z

Transizioni multiple

Stato instabile

Non adiacente a unica cella stabile

x_1x_2

stato presente	00	01	11	10
A=00	A,0	B,-	C,0	A,0
B=01	A,-	B,1	C,-	-,-
C=10	A,0	C,0	C,0	C,0

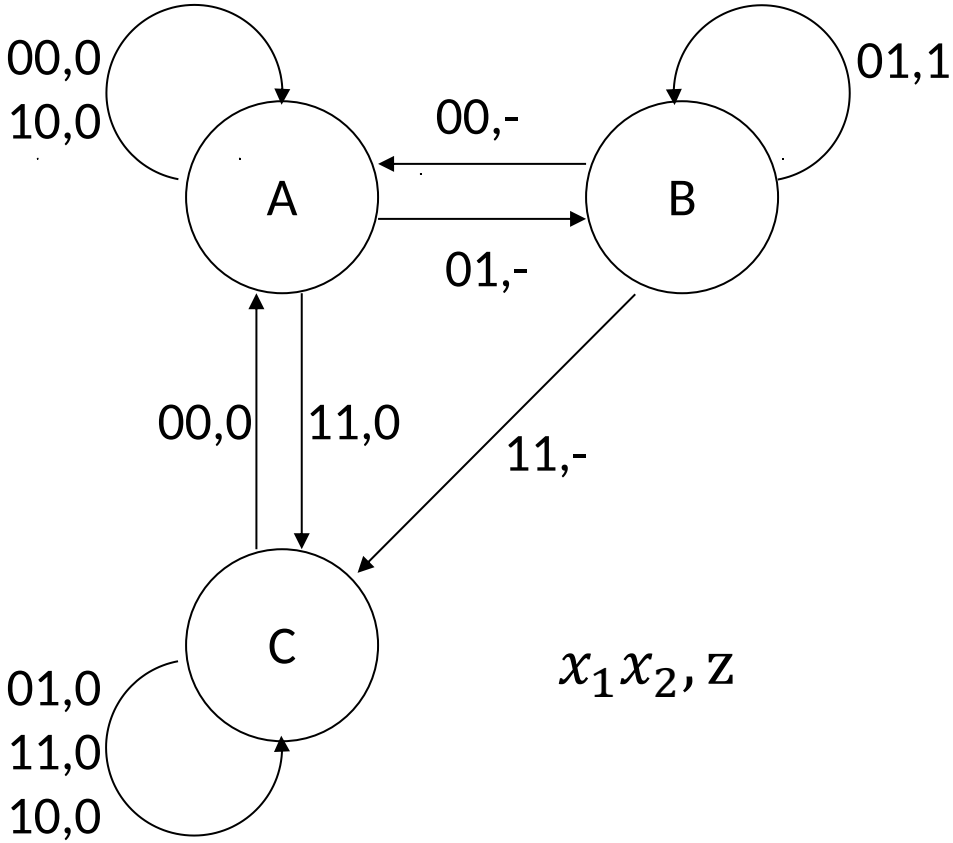
$stato\ futuro$, z

Grafo e comportamento

$x_1 x_2$

stato presente	00	01	11	10
A=00	A,0	B,-	C,0	A,0
B=01	A,-	B,1	C,-	-,-
C=10	A,0	C,0	C,0	C,0

stato futuro, z



Una possibile descrizione testuale: «La rete attiva l'uscita z quando x_2 ha un fronte di salita (=passa da «0» a «1») mentre x_1 vale «0» e la mantiene attiva finché non cambiano nuovamente gli ingressi».